Laboratorul 01 - PL/SQL Introducere

Obiective

- Introducere în PL/SQL
- Blocuri de instrucțiuni PL/SQL
- Rularea blocurilor PL/SQL în SQL*Plus
- Tipuri de variabile
- Tipuri de operatori
- Tipuri de simboluri
- Domeniul variabilelor și constantelor

Noțiuni teoretice

PL/SQL este un limbaj structurat pe blocuri. Are caracteristici de procesare procedurală și de tratare a erorilor. Blocurile pot fi de două tipuri:

- Blocuri anonime
- Subprograme(proceduri, funcții sau triggere)

Deoarece **PL/SQL** este o extensie pentru SQL, regulile sintactice de bază aplicabile în SQL sunt valabile și în PL/SQL:

- Instrucțiunile pot avea mai multe linii cu excepția cuvintelor cheie;
- Unitățile lexicale (identificatori, nume de operatori) pot fi separate de unul sau mai multe spații, sau alt delimitator care nu poate face parte din unități lexicale;
- Cuvintele rezervate nu pot fi folosite ca identificatori decât între ghilimele duble;
- Identificatorii (nume de variabile, functii, proceduri, etc.) trebuie să înceapă cu o literă și pot conține cel mult 30 caractere
- Șirurile trebuie să fie marcate cu ghilimele simple;
- Numerele pot fi reprezentate prin valoarea lor sau științifică;

PL/SQL permite o varietate de tipuri de date ce pot fi folosite pentru declararea de variabile și constante. Variabilele pot avea, opțional, atribuite valori la declarare și pot să-și schimbe valoarea prin atribuiri ulterioare în cadrul blocului. Constantele sunt identificatori care păstrează o valoare fixă ce trebuie atribuită în momentul declarării. Instructiunile PL/SQL nu sunt case sensitive. Setul de caractere este format din literele mari și mici, cifrele de la 0-9, spații, caractere de tabulare, carriage return și simboluri special.

Blocuri Anonime

Blocul anonim are cea mai simplă structură și este compus din trei părți:

- Declare - conține definirea tuturor variabilelor care vor fi folosite în procesul de execuție
- Execuție - conține instrucțiuni DML și bucle de execuție
- Excepții - conține rutinele de interceptare și tratare a erorilor

Fiecare bloc începe cu unul dintre cuvintele cheie **DECLARE** sau **BEGIN**. Cuvântul **BEGIN** este punctul de început al blocului de execuție. Înainte de executarea unui bloc se golește bufferul SQL.

Structura logică a unui bloc **PL/SQL** anonim este:

```
<< label >> (optional)
DECLARE    -- Declarative part (optional)
  -- Declarations of local types, variables, & subprograms

BEGIN      -- Executable part (required)
  -- Statements (which can use items declared in declarative part)

[EXCEPTION -- Exception-handling part (optional)
  -- Exception handlers for exceptions (errors) raised in executable part]
END;
```

Ex. 1. Să se selecteze numele si functia unui angajat introducand id-ul acestuia de la tastatura (testați cu 1, 100 și 200).

```
-- comanda SQL*Plus pentru afidarea de mesaje
set serveroutput on
DECLARE
  -- ampersantul este folosit
  -- pentru a citi o variabila de la tastatura
  idemp number(6) := &id;
  nume varchar2(50);
  -- %type este folosit pentru ca
  -- variabila functie sa ia tipul de date al coloanei
  -- job_title din tabelul jobs
  functie jobs.job_title%type;
BEGIN
  SELECT
    first_name||' '||last_name, job_title
  -- INTO este folosit pentru a spune comandei SELECT
  -- in ce variablie sa pastreze valorite returnate
  INTO
    nume, functie
  FROM
    employees NATURAL JOIN jobs
  WHERE
    employee_id = idemp;
  dbms_output.put_line('Numele angajatului este: '|| nume
    || ' si are functia '||functie);
EXCEPTION
  -- in cazul in care nu se gasesc date de catre comanda SELECT
  -- se intoarce eroarea predefinita no_data_found
  WHEN no_data_found THEN
    dbms_output.put_line('Nu am gasit angajat cu id-ul '||idemp);
END;
/
```

Observații:

- Pentru a afișa mesaje în **SQL*Plus** și **SQL Developer** se va folosi comanda: set serveroutput on
- Funcția care afișează mesaje în **SQL*Plus** și **SQL Developer** este: dbms_output.put_line(string);
- Concatenarea șirurilor de caractere se face folosind operatorul ||, e.g., **'Numele angajatului este ' || nume**
- Pentru a folosi comanda **SELECT** într-un bloc se va folosi o structură de forma: **SELECT COLOANA1, COLOANA2,... INTO VARIABILA1, VARIABILA2, ... FROM TABEL1,...**
- Variabilele VARIABILA1, VARIABILA2, ... sunt declarate în secțiunea **DECLARE**
- Blocurile DECLARE și EXCEPTION nu sunt obligatorii
- Pentru a da o variabilă de la tastatură se folosește **&**
- O variabliă poate avea acea celași data tip ca o altă variabilă sau o coloană dintr-un tabel (ex. functie jobs.job_title%type;).

- Știind că variabila **functie** va trebui să fie de același tip precum coloana **job_title** din tabelul **jobs**, am folosit tipul de variabilă corelat **%TYPE**
- Comentariile pe mai multe linii se delimitează cu **/*...*/**
- Comentariile pe o singură linie se marchează cu două linii orizontale --
- Comentatiile pe o singură linie nu înhibă funționalitatea caracterelor, în schimb cele pe mai multe linii o fac.
    - Daca folosesc următorul comentariu --acest **&** se va executa , atunci compilatorul va cere o valoare pentru variabila de substituție **&** din comentariu.
    - Dacă folosesc /* acest **&** nu se va executa */ , atunci compilatorul **nu** va cere o valoare pentru variabila de substituție **&** din comentariu.

Blocuri Înlănțuite

Blocurile anonime pot fi înlănțuite. Execuția se face secvențial. Declararea variabilelor pentru toate blocurile înlănțuite se face o singură data în secțiunea **DECLARE**.

Structura logică pentru un bloc înlănțuit este:

```
[<<label>>] -- etichetă bloc (opțional)
[DECLARE]  -- (opțional)
  -- variabile pentru blocurile partinte și copii
BEGIN -- start bloc parinte
    [DECLARE] -- (opțional)
      -- variabile pentru blocul copil_1
    BEGIN -- start blocul copil_1
      -- accesibile variabilele declarate în blocurile
      -- parinte și copil_1
    [EXEPTION] -- (opțional)
      -- interceptarea și tratarea excepțiilor blocul copil_1
    END; -- sfârșit bloc copil_1

    [DECLARE] -- (opțional)
      -- variabile pentru blocul copil_2
    BEGIN -- start blocul copil_2
      -- accesibile variabilele declarate în blocurile
      -- parinte si copil_2
    [EXCEPTION] -- (opțional)
      -- interceptarea și tratarea excepțiilor bloc 2
    END; -- sfârșit bloc copil_2
[EXCEPTION] -- (opțional)
  -- interceptarea și tratarea excepțiilor bloc principal
  -- si orice alta exceptie netratata în blocurile copii
END; -- sfârșit bloc parinte
```

Ex. 2. Să se insereze un nou angajat, astfel:

- se introduce de la tastatura un id de job, id-ul angajatului, prenumele și numele
- data angajarii este data curenta **sysdate**
- salariul ese 2123.85
- id-ul managerului este cel al managerului care are cei mai mulți angajați cu job introdus de la tastatura. Dacă sunt mai mulți manageri cu același număr de angajați se ia managerul care are id-ul cel mai mic.

După insert să se adauge pentru acest angajat un punctaj pentru comision de 0.5.

```
set serveroutput on
-- definire variabile la nivelul sesiunii
define id = 1000
define fname = 'ion'
define lname = 'ionescu'
define jid = 'it_prog'

DECLARE
  -- variabile pentru blocurile partinte si copii
  ecuson employees.employee_id%type := &id;
  jid employees.job_id%type := upper('&jid');
  mgr_id number(6);
BEGIN

  BEGIN
    SELECT manager_id INTO mgr_id
    FROM employees  WHERE job_id=upper(jid)
    GROUP BY manager_id
    HAVING count(*) = (
                        SELECT max(cnt)
                        FROM
                        (
                          SELECT count(*) cnt FROM employees
                          WHERE job_id=jid GROUP BY manager_id
                        )
                      )
    ORDER BY manager_id
    FETCH FIRST 1 ROWS ONLY;
  -- exceptii petru blocul copil_1
  EXCEPTION
    WHEN no_data_found THEN
      dbms_output.put_line('Nu gasit functia '||jid);
  END;

  DECLARE
    -- variabile pentru blocul copil_2
    fname employees.first_name%type := '&fname';
    lname employees.last_name%type := '&lname';
    email employees.email%type;
    sal number(8,2) := 2123.85;
  BEGIN
    email := UPPER(substr(fname, 0,1) || lname);
    INSERT INTO
      employees(employee_id, first_name, last_name, salary, email, hire_date, job_id)
    VALUES(ecuson, initcap(fname), initcap(lname), sal, email, sysdate, jid);
  END;

  DECLARE
    -- variabile pentru blocul copil_3
    comision employees.commission_pct%type := 0.5;
  BEGIN
    UPDATE employees
    SET commission_pct = comision
    WHERE employee_id = ecuson;
  END;
  /*
    exceptii pentru toate blocurile
    daca o exceptie nu este tratata in blocul copil
    se va face salt la blocul care trateaza exceptiile de la nivelul blocului parinte
  */
  EXCEPTION
    WHEN others THEN
      dbms_output.put_line('A aparut eroarea '||SQLERRM);
END;
/

-- stergerea variabilelor de la nivelul sesiunii
undefine id
undefine fname
undefine lname
undefine jid

-- verificare
SELECT * FROM employees WHERE employee_id=1000;

-- stergerea noului angajat
DELETE FROM employees WHERE employee_id=1000;
```

Blocuri Imbricate

Blocurile imbricate sunt blocuri care se execută în interiorul altui bloc. Se mai numesc și sub-blocuri. **Variabile pot fi transmise de la bloc la sub-bloc, nu și invers.**

Structura logică pentru un bloc imbricat este:

```
[<<label>>] -- etichetă bloc (opțional)
[DECLARE] -- (opțional)
  --  variabile bloc principal, bloc 1, bloc 2, bloc 3
BEGIN
```

```
    [DECLARE] -- (opțional)
      --  variabile bloc 1, bloc 2, bloc 3
    BEGIN -- bloc 1
        [DECLARE]  -- (opțional)
          --  variabile bloc 2, bloc 3
        BEGIN -- bloc 2
            [DECLARE]  -- (opțional)
              --  variabile bloc 3
            BEGIN -- bloc 3
            [EXCEPTION] -- (opțional)
              -- tratare erori bloc 3
            END; -- bloc 3
        [EXCEPTION] -- (opțional)
          -- tratare erori bloc 2
          -- tratarea erorilor propagate din blocul 3
        END; -- bloc 2
    [EXCEPTION] -- (opțional)
      -- tratare erori bloc 1
      -- tratarea erorilor propagate din blocul 2 și 3
    END; -- bloc 1
[EXCEPTION] -- (opțional)
  -- tratare erori bloc principal
  -- tratarea erorilor propagate din blocurile 1, 2 și 3
END;
```

Ex. 3. Să se rezolve exercițiul anterior folosind blocuri imbricate.

```plsql
set serveroutput on
-- definire variabile la nivelul sesiunii
define id = 1000
define fname = 'ion'
define lname = 'ionescu'
define jid = 'it_prog'

DECLARE
  -- variabile pentru blocurile partinte si copii
  jid employees.job_id%type := upper('&jid');
  mgr_id number(6);
BEGIN

  SELECT manager_id INTO mgr_id
  FROM employees  WHERE job_id=upper(jid)
  GROUP BY manager_id
  HAVING count(*) = (
                      SELECT max(cnt)
                      FROM
                      (
                        SELECT count(*) cnt FROM employees
                        WHERE job_id=jid GROUP BY manager_id
                      )
                    )
  ORDER BY manager_id
  FETCH FIRST 1 ROWS ONLY;

  DECLARE
    -- variabile pentru blocul imbricat_1
    fname employees.first_name%type := '&fname';
    lname employees.last_name%type := '&lname';
    email employees.email%type;
    ecuson employees.employee_id%type := &id;
    sal number(8,2) := 2123.85;
  BEGIN

    email := UPPER(substr(fname, 0,1) || lname);
    INSERT INTO
      employees(employee_id, first_name, last_name, salary, email, hire_date, job_id)
    VALUES(ecuson, initcap(fname), initcap(lname), sal, email, sysdate, jid);

    DECLARE
      -- variabile pentru blocul imbricat_2
      comision employees.commission_pct%type := 0.5;
    BEGIN
      UPDATE employees
      SET commission_pct = comision
      WHERE employee_id = ecuson;
    END;

  EXCEPTION
    /*
      exceptii pentru blocul imbricat_1
      daca apar exceptii netratate in blocul imbricat_2
      acestea mai intai se propaga aici
      daca nici aici nu sunt trate, se propaga mai departe la blocul parinte
    */
    WHEN others THEN
      dbms_output.put_line('A aparut eroarea '||SQLERRM);
  END;

  /*
    exceptii pentru toate blocurile
    daca o exceptie nu este tratata in blocul copil
    se va face salt la blocul care trateaza exceptiile de la nivelul blocului parinte
  */
  EXCEPTION
    WHEN no_data_found THEN
      dbms_output.put_line('Nu gasit functia '||jid);
END;
/

-- stergerea variabilelor de la nivelul sesiunii
undefine id
undefine fname
undefine lname
undefine jid

-- verificare
SELECT * FROM employees WHERE employee_id=1000;

-- stergerea noului angajat
DELETE FROM employees WHERE employee_id=1000;
```

Executarea blocurilor PL/SQL în SQL*Plus (Opțional)

În cazul în care nu aveți acces la un IDE, puteți să vă conectați la un Server ORACLE folosind utilitarul în linie de comandă **SQL*Plus**. (Mai multe dedalii în Laboratorul 2 - Baze de date [https://ocw.cs.pub.ro/courses/bd/laboratoare/02])

Pentru a vă conecta la SQL*Plus în consolă comanda care se folosește este:

```
sqlplus [username[/password[@bazadate]]]
```

Blocurile **PL/SQL** pot fi editate direct în **SQL*Plus** (nerecomandat) sau pot fi editate în orice editor de text . Pentru a executa un bloc în **SQL*Plus** folositi una din comenzile (comanda intreagă sau abrevierea):

Comandă	Abreviere	Utilizare
RUN	R	Afișează și execută blocul curent din buffer
	/	Execută blocul curent din buffer

În **SQL*Plus**

- Un bloc rămâne în buffer până la editarea următorului bloc.
- Un bloc poate fi scris pe mai multe rânduri (folosiți ENTER pentru a începe o nouă linie)

În **SQL*Plus** un bloc se poate executa astfel:

- Se copiază blocul în editor (Copy/Paste) și se execută cu / sau RUN
- Se execută tot scriptul (care poate să conțină mai multe blocuri) folosind:
  - SQL> @cale_fieier\nume_fisier.sql
  - SQL> start cale_fieier\nume_fisier.sql

Dacă executați sau copiați codul dintr-un fișer, se recomandă adaugarea unui forward slash (/) – comanda de executare a blocului – după ultimul **END**.

În linia de comandă se mai pot da și următoarele comenzi:

- SAVE file_path_and_name – permite salvarea într-un fișier a conținutului bufferului SQL*Plus;
- GET file_path_and_name – încarcă conținutul unui fișier salvat în prealabil în buffer;
- START file_path_and_name – lansează în execuție comenzile salvate în fișier;
- @file_path_and_name – execută comenzile din fișier;
- EXIT – ieșire din SQL*Plus.

Comenzi **SQL*Plus** utile pentru formatare

Comandă	Utilizare
set serveroutput on	Permite afișarea în SQL*Plus
set lines n	Permite printarea a n caractere pe verticală
set pages n	Permite printarea a n coloane

Tipuri variabile

PL/SQL acceptă toate tipurile de date din SQL. Nu sunt permise referințe anticipate, deci toate variabilele și constantele trebuie să fie declarate în secțiunea **DECLARE** înainte de utilizare. În secțiunea **DECLARE** trebuie declarate numele și tipul fiecărei variabile utilizate în cadrul blocului. Tipul specifică formatul de memorare, restricțiile și un domeniu valid de valori. Sintaxa declarării unei variabile este următoarea:

```
nume_var [CONSTANT]{tip_de_date|identificator%TYPE|identificator%ROWTYPE}[NOT NULL][{:=|DEFAULT} expresie_PL/SQL];
```

Tipuri de date scalare

Tipurile de date scalare nu au componente interne (conţin valori atomice).

Tipurile de date ce stochează valori numerice cuprind:

- tipul NUMBER cu subtipurile DEC, DECIMAL, DOUBLE PRECISION, FLOAT, INTEGER, INT, NUMERIC, REAL, SMALLINT;
- tipul BINARY_INTEGER cu subtipurile NATURAL, NATURALN, POSITIVE, POSITIVEN, SIGNTYPE;
- tipul PLS_INTEGER.

Tipurile de date ce stochează caractere cuprind:

- tipul VARCHAR2 cu subtipurile STRING, VARCHAR;
- tipul de date CHAR cu subtipul CHARACTER;
- tipurile LONG, RAW, LONG RAW, ROWID

Tipurile de date ce stochează data calendaristica si ora cuprind tipurile:

- DATE
- TIMESTAMP
- TIMESTAMP WITH TIME ZONE
- TIMESTAMP WITH LOCAL TIME ZONE
- INTERVAL YEAR TO MONTH
- INTERVAL DAY TO SECOND;

Tipurile de date ce stochează date în format unicode includ tipurile

- NCHAR
- NVARCHAR2;

Tipul de date BOOLEAN stochează valori logice (true, false sau null)

Exemple:

```
v_valoare NUMBER(15) NOT NULL := 0;
v_data_achizitie DATE DEFAULT SYSDATE;
v_material VARCHAR2(15) := 'Matase';
c_valoare CONSTANT NUMBER := 100000;
v_stare VARCHAR2(20) DEFAULT 'Buna';
v_clasificare BOOLEAN DEFAULT FALSE;
int_an_luna INTERVAL YEAR TO MONTH :=
INTERVAL '3-2' YEAR TO MONTH; --interval de 3 ani si 2 luni
```

Ex. 4. Să se scrie un bloc PL/SQL care primește de la tastatură un id de angajat și va afișa un mesaj cu forma: '**nume_angajat** are un venit orar de **venit_orar** și face parte din departamentul **denumire_departament**. A lucrat în firma un număr total de **zile_lucrate** zile'. În caz că angajatul nu există să se trateze eroarea.

```
set serveroutput on

DECLARE
  nume_ang nvarchar2(45);
  id_dep integer;
  dname varchar(20);
  dataAng date;
  idAng number(6) := &id;
  venit float;
  zileLucrate number(10);
  venitOrar real;
  zileLuna constant smallint := 21;
BEGIN

  SELECT
    department_id,
    first_name||' '||last_name,
    salary + nvl(commission_pct,0)*salary,
    hire_date
  INTO
    id_dep,
    nume_ang,
    venit,
    dataAng
  FROM employees
  WHERE employee_id = idAng;

  SELECT department_name
  INTO dname
  FROM departments
  WHERE department_id = id_dep;

  venitOrar := round(venit/(zileLuna*8), 2);
  zileLucrate := sysdate - dataAng;

  dbms_output.put_line(nume_ang || ' are un venit orar de ' ||
    venitOrar || ' si face parte din departamentul ' ||
    dname || '.' || chr(13) || chr(10) ||
    'A lucrat in firma un numar total de ' || zileLucrate || ' zile.' );

EXCEPTION
  WHEN no_data_found THEN
    dbms_output.put_line('Angajatul nu se afla in baza de date!');
END;
/
```

Tipuri de date corelate

Tipurile corelate sunt folosite pentru a declara variabile de tipul unui rând de tabel sau al unei coloane de tabel. Avantajul folosirii unor astfel de tipuri este că atunci când se modifică structura coloanelor dintr-un tabel nu mai trebuie să fie modificat și codul **PL/SQL**.

Tipuri corelate:

- %TYPE - variabilă de tipul unei coloane de tabel
- %ROWTYPE - vector de tipul unui rând de tabel

Ex. 5. Să se scrie un bloc PL/SQL care primește de la tastatură un id de angajat și afișează numele, funcția acestuia, departamentul și locația departamentului. Locația deopartamentului va cuprinde orașul, statul/județul, țara și regiunea.

```
set serveroutput on

DECLARE
  prenume_ang employees.first_name%type;
  nume_ang employees.last_name%type;
  venit employees.salary%type;
  id_dept employees.department_id%type;
  dept_row departments%rowtype;
  location_row locations%rowtype;
  country_row countries%rowtype;
  region_row regions%rowtype;
BEGIN

  SELECT
    department_id, first_name, last_name,
    salary+nvl(commission_pct,0) * salary
  INTO id_dept, prenume_ang, nume_ang, venit
  FROM employees
  WHERE employee_id=&id;

  SELECT d.* INTO dept_row FROM departments d
  WHERE department_id = id_dept;

  SELECT l.* INTO location_row FROM locations l
  WHERE l.location_id = dept_row.location_id;

  SELECT c.* INTO country_row FROM countries c
  WHERE c.country_id = location_row.country_id;

  SELECT r.* INTO region_row FROM regions r
  WHERE r.region_id = country_row.region_id;

  dbms_output.put_line(prenume_ang ||' '|| nume_ang ||
    ' face parte din departamentul ' || dept_row.department_name ||
    ' din '|| location_row.city || ', ' || location_row.state_province ||
    ', ' || country_row.country_name || ', ' || region_row.region_name);

  EXCEPTION
    WHEN no_data_found THEN
      dbms_output.put_line('Angajatul nu se gaseste in baza de date');
END;
/
```

Tipuri de date LOB

Tipuri LOB (Large OBject) sunt folosite pentru stocarea de date binare, text, imagini și fișiere video de dimensiuni mari (până la 4GB).

Data Type	Description
CLOB	A character large object containing single-byte or multibyte characters. Both fixed-width and variable-width character sets are supported, both using the database character set. Maximum size is (4 gigabytes - 1) * (database block size).
NLOB	A character large object containing Unicode characters. Both fixed-width and variable-width character sets are supported, both using the database national character set. Maximum size is (4 gigabytes - 1) * (database block size). Stores national character set data.
BLOB	A binary large object. Maximum size is (4 gigabytes - 1) * (database block size).
BFILE	Contains a locator to a large binary file stored outside the database. Enables byte stream I/O access to external LOBs residing on the database server. Maximum size is 4 gigabytes.

Tipuri de Operatori

Operatori aritmetici

Operatori aritmetici	Descriere
+	Adunare
-	Scădere

*	Multiplicare
/	Împărțire
**	Exponențial

Operatorii de comparație

Operatorii de comparație pot fi feluri: operatori logici și operatori SQL

Operator Logic	Semnificație
=	Egal
>	Mai mare decât
>=	Mai mare sau egal decât
<	Mai mic decât
<=	Mai mic sau egal decât

Operator SQL	Semnificație
`val BETWEEN val1 AND val2`	Verifică daca o valoare se află între două valori date (inclusiv). Echivalent cu `val >= val1 AND val <= val2`
`IN (list)`	Compară dacă valoarea se află în lista de valori list. `val IN (a,b,c)` este echivalent cu `val = a OR val = b OR val = c`
`LIKE string`	Compară cu un model de tip caracter/șir de caractere
`IS NULL`	Verifică dacă o valoare este `NULL`

Operatori de negație

Operatorii de negație pot fi de două feluri: operatori logici și operatori SQL.

Operator Logic	Semnificație
!=	Diferit de (Unix, Windows)
^=	Diferit de (IBM)
<>	Diferit de (Toate OS)
`NOT val { = | > | >= | < | <= }`	Neagă operatorul de comparație

Operator SQL	Semnificație
`NOT BETWEEN val1 AND val2`	Verifică daca o valoare nu se află între două valori date (inclusiv)
`NOT IN (list)`	Compară dacă valoarea nu se află în lista de valori list.
`NOT LIKE string`	Compară cu un model de tip caracter/șir de caractere
`IS NOT NULL`	Verifică dacă o valoare nu este `NULL`

Precedența operatorilor logici este următoarea:

1. Operatorii de comparație și operatorii SQL au precedență egală (`=,<,<=,>,>=,<>,!=,^=, BETWEEN, IN, LIKE, IS NULL`)
2. `NOT` (pentru inversarea rezultatului unei expresii logice)
3. `AND`
4. `OR`

Pentru a fi siguri de ordinea de execuție a operațiilor se recomandă folosirea parantezelor rotunde. Ordinea naturală a operațiilor în cadrul unei expresii este următoarea:

Ordine	Operație	Descriere
1	**	exponențial

2	+, -	identitate, negație
3	*, /	înmulțire, împărțire
4	+, -, ||	adunare, scădere, concatenare
5	=, !=, <, >, %⇐%, >=, IS NULL, LIKE, BETWEEN, IN	comparație
6	NOT	negație
7	AND	conjuncție
8	OR	incluziune

Tipuri de simboluri

Tipuri de simboluri simple

Simbol	Descriere
( )	Delimitator de listă sau expresii
;	Delimitator de instrucțiuni
,	Separator de obiect
.	Selector de componentă
@	Delimitator de acces la baza de date
'	Delimitator de șir
:	Delimitator de server
$	Format de afișare în USD
&	Variabilă substituită

Tipuri de simbiluri compuse

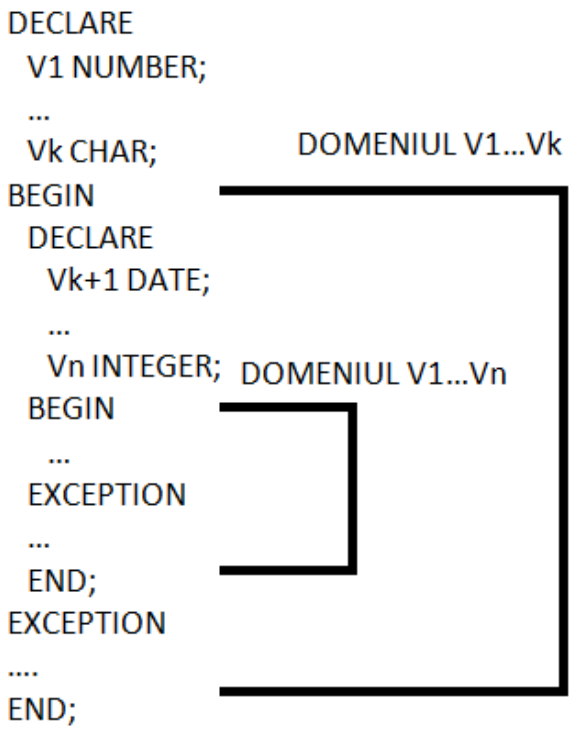
Simbol	Descriere
:=	Asignare
=>	Asociere
..	Rang
||	Concatenare
<< >>	Etichetă
--	Comentariu pe o singură linie
/* */	Comentatiu pe mai multe linii

Nu sunt permise spații între cele două caractere ale simbolurilor compuse.

Domeniul variabilelor și constantelor

Domeniul de valabilitate al variabilelor reprezintă zona unui program în care acestea pot fi folosite. Variabilele și constantele pot fi declarate în cadrul blocului în care au fost definite sau în orice sub-bloc inclus în acesta. Variabilele declarate într-un sub-bloc pot fi folosite doar în acesta sau în sub-blocurile acestuia. Variabilele se pot transmite de la un bloc la sub-bloc, dar nu și invers. Aceste constrângeri se aplică tuturor obiectelor declarate, inscusiv cursoare, constante și excepții definite de utilizator.

În sub-bloc se pot folosi atât variabilele definite local cât și cele din blocul superior. Dacă într-un sub-bloc se definește o variabilă cu același nume ca una dintr-un bloc superior atunci are prioritate cea locală, dezactivându-se variabila definită în blocul superior. Se recomandă să se evite astfel de situații pentru a nu crea confuzii.



Exerciții rezolvate

Ex. 6. Să se afișeze informațiile despre ultimul angajat care a venit în firmă în anul 2001.

```plsql
set serveroutput on

DECLARE
  nume_ang varchar2(20);
  functie string(30);
  max_data date;
  start_date date := '1-JAN-2001';
  end_date date := '31-DEC-2001';
BEGIN

  SELECT first_name||' '||last_name, job_title
  INTO nume_ang, functie
  FROM employees NATURAL JOIN jobs
  WHERE hire_date = (
                      SELECT max(hire_date)
                      FROM employees
                      WHERE hire_date BETWEEN start_date AND end_date
                    );

  dbms_output.put_line('Angajatul cautat este '
    || nume_ang ||' si are functia '|| functie);
EXCEPTION
  WHEN no_data_found THEN
    dbms_output.put_line('Nu a fost angajat nimeni in 2001.');
  WHEN too_many_rows THEN
    dbms_output.put_line('Sunt mai multe angajari pentru data maxima.');
END;
/
```

Ex. 7. Să se găsească care angajat are o vechime de peste 19 de ani si 5 luni in firmă.

```
set serveroutput on

DECLARE
  nume_ang varchar(45);
  functie jobs.job_title%type;
  data date;
  vechime interval year(2) to month;
BEGIN
  vechime := interval '19-5' year to month;
  dbms_output.put_line('Vechimea solicitata = '||vechime);

  SELECT hire_date into data
  FROM employees
  WHERE hire_date < sysdate - vechime;

  dbms_output.put_line('Data maxima '|| data);
  SELECT first_name||' '||last_name, job_title
  INTO nume_ang, functie
  FROM employees NATURAL JOIN jobs
  WHERE hire_date = data;

  dbms_output.put_line('Angajatul cu vechimea cautata este '||
    nume_ang||' si are functia '||functie);

EXCEPTION
  WHEN no_data_found THEN
    dbms_output.put_line('Nu exista angajati cu aceasta vechime.');
  WHEN too_many_rows THEN
    dbms_output.put_line('Sunt mai multi angajati cu aceasta vechime.');
END;
/
```

Ex. 8. Să se facă o verificare a numărului de angajări pentru un an introdus de la tastatură. Dacă există o angajare, atunci se va afișa mesajul 'In anul [an] au avut loc [numar angajari] angajare'. Dacă nu există nicio angajare, se va declanșa o excepție **no_data_found** și se va afișa mesajul 'Nu exista angajari in anul [an]'. Dacă sunt mai mulți angajați, se va declanșa excepția **too_many_rows** și se va afișa mesajul 'Sunt [numar angajari] angajari in anul [an]'.

```
DECLARE
  nume_ang varchar2(45);
  functie jobs.job_title%type;
  an number(4) := &an;
  angajari number;
BEGIN

  SELECT count(*)
  INTO angajari
  FROM employees NATURAL JOIN jobs
  WHERE extract(year from hire_date) = an;

  SELECT first_name, job_title
  INTO nume_ang, functie
  FROM employees NATURAL JOIN jobs
  WHERE extract(year from hire_date) = an;

  dbms_output.put_line('In anul '|| an || ' au avut loc '||
    angajari ||' angajare');

EXCEPTION
  WHEN no_data_found THEN
    dbms_output.put_line('Nu exista angajari in anul '|| an );
  WHEN too_many_rows THEN
    dbms_output.put_line('Sunt '|| angajari ||' angajari in anul '|| an);
END;
/
```

Ex. 9. Să să selecteze departamementul, numele, data angajării și comisionul unui angajat introducând id-ul angajatului de la tastatură. În cazul în care angajatul nu are departament, se afișează N/A. Testați cu valorile 100 si 178

```
set serveroutput on

DECLARE
  nume_dept departments.department_name%type;
  id_ang employees.employee_id%type;
  nume_ang varchar(50);
  comm number(10);
  data_ang date;
BEGIN
  id_ang := &idAng;
  BEGIN
    SELECT
      d.department_name, e.first_name||' '||e.last_name,
      e.hire_date, e.salary * nvl(e.commission_pct, 0)
    INTO nume_dept, nume_ang, data_ang, comm
    FROM employees e
    LEFT JOIN departments d ON e.department_id = d.department_id
    WHERE employee_id = id_ang;

    dbms_output.put_line(rpad('Nume Dept',30,' ')||rpad('Nume',30,' ')||
      rpad('Data Angajare',15,' ')||lpad('Comision', 10,' '));
    dbms_output.put_line(rpad(nvl(nume_dept, 'N/A'),30,' ')||rpad(nume_ang,30,' ')||
      rpad(data_ang,15,' ')||lpad(comm, 10,' '));
  EXCEPTION
    WHEN no_data_found THEN NULL;
  END;
END;
/
```

Ex. 10. Să se calculeze suma veniturilor angajaților care au aceeași funcție și lucrează în același departament. Testați cu

- id_dep = 10 și id_job = AD_ASST
- id_dep = 10 și id_job = IT_PROG

```
set serveroutput on

DECLARE
  nume_dep departments.department_name%type;
  sum_venit employees.salary%type;
  functie jobs.job_title%type;
BEGIN
  SELECT
    d.department_name, j.job_title,
    sum(e.salary + e.salary * nvl(e.commission_pct,0))
  INTO nume_dep, functie, sum_venit
  FROM employees e NATURAL JOIN jobs j
  RIGHT OUTER JOIN departments d ON e.department_id = d.department_id
  WHERE
    d.department_id=&id and job_id=upper('&jid')
  GROUP BY d.department_name, j.job_title;

  dbms_output.put_line('Suma veniturilor in departamentul '|| nume_dep
    ||' pentru functia ' || functie || ' este '||to_char(sum_venit));
EXCEPTION
  WHEN no_data_found THEN
    dbms_output.put_line('Nu s-au gasit date!');
END;
/
```

Laboratorul 02 - PL/SQL Structuri de control și tipuri de date

Obiective

- Instrucțiuni
- Tipuri compuse și colecții

Instrucțiuni

Limbajul PL/SQL combină puterea de procesare a datelor, specifică limbajelor procedurale, cu puterea de manipulare a datelor, specifică instrucțiunilor. Structurile de control reprezintă cea mai importantă completare adusă de PL/SQL limbajului SQL prin introducerea instrucțiunilor de control condiționale, iterative, secvențiale și necondiționale. Toate structurile acceptă în clauzele lor instrucțiuni imbricate.

Instrucțiuni condiționale

Instrucțiunea IF ... THEN

Sintaxa instrucțiunii este:

```
IF conditions THEN
    statements;
END IF;
```

Unde:

- conditions – reprezintă condițiile cerute pentru execuția instrucțiunilor
- statements – reprezintă secvența de instrucțiuni condiționale

Instrucțiunile care urmează după **THEN** se execută numai dacă toate condițiile din **conditions** sunt îndeplinite.

Ex. 1. Pentru un angajat cu funcția 'FI_MGR' care nu primește comision să se modifice comisionul la 10% (.1). Testați cu id-ul 108.

```
set serveroutput ON;
DECLARE
    numeangajat VARCHAR2(50);
    comangajat  NUMBER;
    functie     VARCHAR2(15);
    ecuson      employees.employee_id%TYPE;
BEGIN
    ecuson := &idAngajat; --108

    SELECT first_name
           || ' '
           || last_name,
           job_id,
           commission_pct
    INTO   numeangajat, functie, comangajat
    FROM   employees
    WHERE  employee_id = ecuson;

    IF Upper(functie) = 'FI_MGR'
       AND ( comangajat = 0
             OR comangajat IS NULL ) THEN
      UPDATE employees
      SET    commission_pct = 0.1
      WHERE  employee_id = ecuson;

      dbms_output.Put_line('Comisionul lui '
                           || numeangajat
                           ||' a fost modificat!');
    END IF;
EXCEPTION
    WHEN no_data_found THEN
      dbms_output.Put_line('Nu exista angajat cu employee_id = '
                           ||ecuson);
END;
```

Instrucțiunea IF ... THEN ... ELSE

Sintaxa instrucțiunii este:

```
IF conditions THEN
    statements_1;
ELSE
    statements_2;
END IF;
```

Unde:

- conditions – reprezintă condițiile cerute pentru execuția statements_1
- statements_1 – reprezintă secvența de instrucțiuni atunci când conditions este TRUE
- statements_2 – reprezintă secvența de instrucțiuni atunci când conditions este FALSE

Ex. 2. Să se acorde un comision de 10% din salariu angajaților care au o vechime de cel puțin 15 de ani în companie. (testati cu 108 si 149).

```
set serveroutput on;
declare
  numeAngajat varchar2(30);
  ecuson emp.empno%type;
  dataAngajare date;
begin

  ecuson := &idAng;

  select ename, hiredate into numeAngajat, dataAngajare
  from emp where empno = ecuson;

  if dataAngajare < add_months(sysdate, -384) then
    update emp set comm = 0.1 * sal where empno=ecuson;
    dbms_output.put_line('Angajatul '||numeAngajat||' a primit comision!');
  else
    dbms_output.put_line('Angajatul nu indeplineste conditia!');
  end if;

  exception
    when no_data_found then
      dbms_output.put_line('Nu exista angajat cu ecusonul='||ecuson);
end;
/
```

Instrucțiunea IF ... THEN ... ELSIF ... ELSE

Sintaxa instrucțiunii este:

```
IF conditions_1 THEN
  statements_1;
ELSIF conditions_2 THEN
  statements_2;
...
[ELSE statements_k;]
END IF;
```

Unde:

- conditions_1 – reprezintă setul de condiții cerute pentru execuția statements_1
- statements_1 – reprezintă secvența de instrucțiuni executate atunci când conditions_1 = TRUE
- conditions_2 – reprezintă setul de condiții cerute pentru execuția statements_2
- statements_2 – reprezintă secvența de instrucțiuni executate atunci când conditions_2 = TRUE
- statements_k – reprezintă secvența de instrucțiuni executate dacă niciuna dintre conditions_1..k-1 nu sunt verificate

Verificarea seturilor de condiții se face secvențial, în ordinea specificată și fiecare set poate conține condiții multiple.

Se acceptă mai multe clause ELSIF dar o singură clasuză ELSE.

Clauza ELSE este opțională, aceasta poate să lipsească din construcția instrucțiunii.

Ex. 3. Să se acorde un comision de 10%, conducerii companiei care nu a primit comision, dar președintele să nu primească comision.

```
set serveroutput ON;
DECLARE
    numeangajat VARCHAR2(50);
    comangajat  NUMBER;
    functie     VARCHAR2(15);
    ecuson      employees.employee_id%TYPE;
BEGIN
    ecuson := &idangajat; --100/122/110
    SELECT first_name
           || ' '
           || last_name,
           job_id,
           commission_pct
    INTO   numeangajat, functie, comangajat
    FROM   employees
    WHERE  employee_id = ecuson;

    IF Upper(functie) = 'AD_PRES' THEN
      dbms_output.Put_line('Presedintele nu primeste comision!');
    ELSIF ( Upper(functie) LIKE '%_MAN'
             OR Upper(functie) LIKE '%_MGR' )
          AND ( comangajat = 0
                 OR comangajat IS NULL ) THEN
      UPDATE employees
      SET    commission_pct = 0.1
      WHERE  employee_id = ecuson;

      dbms_output.Put_line('Comisionul lui '
                           || numeangajat
                           ||' a fost modificat!');
    ELSE
      dbms_output.Put_line('Angajatul nu face parte din conducere!');
    END IF;
EXCEPTION
    WHEN no_data_found THEN
      dbms_output.Put_line('Nu exista angajat cu empno = '
                           ||ecuson);
END;
```

Instrucțiunea CASE

Sintaxa instrucțiunii este:

```
[<<label_name>>]
CASE expression
WHEN value_1   THEN   statements_1;
WHEN value_2   THEN   statements_2;
...
[ELSE statements_k;]
END CASE [label_name];
```

Unde:

- expression – reprezintă expresia care se va evalua
- statements_1 – reprezintă secvența de instrucțiuni executate în cazul în care expression = value_1
- statements_2 – reprezintă secvența de instrucțiuni executate în cazul în care expression = value_2
- statements_k – reprezintă secvența de instrucțiuni executate dacă expression <> value_1, value_2, ...

Ex. 4. Să se calculeze impozitul pe salariu al unui angajat, după algoritmul:

- Din tabela JOBS se află salariul minim, respectiv maxim, pentru funcția pe care o deține;

- Dacă angajatul are salariul minim, atunci impozitul este de 10% din salariu.
- Dacă angajatul are salariul maxim, atunci impozitul este de 30% din salariu.
- Altfel impozitul este de 20% din salariu.
- Se afișează numele și impozitul angajatului.

```
 1 set serveroutput ON;
 2 DECLARE
 3     numeangajat VARCHAR2(50);
 4     salariu NUMBER;
 5     salariuMinim NUMBER;
 6     salariuMaxim NUMBER;
 7     functie VARCHAR2(10);
 8     ecuson employees.employee_id%TYPE;
 9     impozit NUMBER;
10 BEGIN
11     ecuson := &idangajat; --108 / 149
12     SELECT first_name
13            || ' '
14            || last_name,
15            job_id,
16            SALARY
17     INTO   numeangajat, functie, salariu
18     FROM   employees
19     WHERE  employee_id = ecuson;
20
21     SELECT MIN_SALARY,
22             MAX_SALARY
23     INTO salariuMinim, salariuMaxim
24     FROM JOBS WHERE JOB_ID = functie;
25
26
27     <<mycase>> --exemplu de eticheta (nu este obligatorie)
28     case salariu
29       when salariuMinim then impozit := salariu * 0.1;
30       when salariuMaxim then impozit := salariu * 0.3;
31     else
32       impozit := salariu * 0.2;
33     end case mycase;
34
35     dbms_output.Put_line('Impozitul lui '
36                          || numeangajat
37                          ||' este '
38                          || impozit);
39
40 EXCEPTION
41     WHEN no_data_found THEN
42       dbms_output.Put_line('Nu exista angajat cu employee_id = '
43                          ||ecuson);
44 END;
```

Instrucțiunea SEARCHED CASE

PL/SQL pune la dispoziția dezvoltatorilor **SEARCHED CASE**. Sintaxa instrucțiunii **SEARCHED CASE** este:

```
[<<label_name>>] CASE
WHEN search_condition_1 THEN sequence_of_statements_1;
WHEN search_condition_2 THEN sequence_of_statements_2;
...
WHEN search_condition_N THEN sequence_of_statements_N;
[ELSE sequence_of_statements_N+1;]
END CASE [label_name];
```

Ex. 5. Folosirea instrucțiunii searched CASE:

```
set serveroutput ON;
DECLARE
    grade FLOAT := 7.2;
BEGIN
    CASE
      WHEN grade >= 9 THEN
        dbms_output.Put_line('excellent');
      WHEN grade >= 8
           AND grade < 9 THEN
        dbms_output.Put_line('very good');
      WHEN grade >= 7
           AND grade < 8 THEN
        dbms_output.Put_line('good');
      WHEN grade >= 6
           AND grade < 7 THEN
        dbms_output.Put_line('fair');
      WHEN grade < 5 THEN
        dbms_output.Put_line('poor');
      ELSE
        dbms_output.Put_line('no such grade');
    END CASE;
END;
```

Ex. 6. Folosirea instrucțiunii CASE pentru inițializarea unei variabile și în clauza where a unei cereri:

```plsql
set serveroutput ON;
DECLARE
    alege       INTEGER;
    functie     employees.job_id%TYPE;
    numeangajat VARCHAR2(50);
BEGIN
    dbms_output.Put_line('Alege'||Chr(13)|| Chr(10)||'1 - IT_PROG'||Chr(13)|| Chr(10)||'2 -
FI_MGR'||Chr(13)|| Chr(10)||' - PU_MAN');
    alege := &numar; -- 1

    dbms_output.Put_line('Alege versiunea 1 sau 2');
    CASE &alegeversiune
      WHEN 1 THEN
        BEGIN
            dbms_output.Put_line('Ai ales versiunea 1');
            GOTO versiunea1;
        END;
      WHEN 2 THEN
        BEGIN
            dbms_output.Put_line('Ai ales versiunea 1');
            GOTO versiunea2;
        END;
      ELSE
        BEGIN
            dbms_output.Put_line('Nu ai ales nici o versiune!');
            GOTO endofprogram;
        END;
    END CASE;

    <<versiunea1>>
    functie := CASE
                 WHEN alege = 1 THEN 'IT_PROG'
                 WHEN alege = 2 THEN 'FI_MGR'
                 ELSE 'PU_MAN'
               END;
    SELECT first_name
           || ' '
           || last_name
    INTO   numeangajat
    FROM   employees
    WHERE  employee_id = ( CASE
                             WHEN functie = 'IT_PROG' THEN 103
                             WHEN functie = 'FI_MGR' THEN 108
                             WHEN functie = 'PU_MAN' THEN 114
                             ELSE 118
                           END );
    dbms_output.Put_line(numeangajat
                         ||' are functia '
                         ||functie);
    GOTO endofprogram;

    <<versiunea2>>
    functie := CASE alege
                 WHEN '1' THEN 'IT_PROG'
                 WHEN '2' THEN 'CLERK'
                 ELSE 'PU_MAN'
               END;
    SELECT first_name
           || ' '
           || last_name
    INTO   numeangajat
    FROM   employees
    WHERE  employee_id = ( CASE functie
                             WHEN 'IT_PROG' THEN 103
                             WHEN 'FI_MGR' THEN 108
                             WHEN 'PU_MAN' THEN 114
                             ELSE 118
                           END );
    dbms_output.Put_line(numeangajat
                         ||' are functia '
                         ||functie);
    <<endofprogram>>

    dbms_output.Put_line('Sfarsit program');
END;
```

Ex. 7. Exemplu de folosire a instrucțiunii CASE în clauza SELECT.

```
set serveroutput ON;
DECLARE
    numeangajat VARCHAR2(50);
    functie     VARCHAR2(15);
BEGIN
    SELECT e.first_name
           || ' '
           || e.last_name,
           ( CASE e.job_id
               WHEN 'FI_MGR' THEN 'Finance Manager'
               WHEN 'FI_ACCOUNT' THEN 'Accountant'
               WHEN 'AC_ACCOUNT' THEN 'Public Accountant'
               WHEN 'SA_MAN' THEN 'Sales Manager'
               WHEN 'ST_MAN' THEN 'Stock Manager'
               ELSE ' #nuAvetiPermisiuni'
             END )
    INTO   numeangajat, functie
    FROM   employees e
    WHERE  employee_id = &idemp; -- 108
    dbms_output.Put_line(numeangajat
                         ||' are functia '
                         ||functie);
END;
```

Observații:

- Clauza **ELSE** poate să lipsească din instrucțiunea **CASE**;
- Dacă lipsește clauza **ELSE** și niciuna dintre clauzele **WHEN** nu se execută atunci apare eroarea **CASE_NOT_FOUND**.

Instrucțiuni de repetiție

Instrucțiunea LOOP

Instrucțiunea LOOP este folosită pentru execuția ciclică a unei secvențe de instrucțiuni.

Sintaxa instrucțiunii este:

```
[<<label_name>>]
LOOP
    statements;
    [EXIT [WHEN conditions];]
    [CONTINUE [WHEN conditions];]
END LOOP [label_name];
```

Unde:

- statements – reprezintă secvența de instrcțiuni execuate în cadrul buclei

- label_name – este eticheta următoarei instrucțiuni la care se forțează saltul, care poate să lipsească
- conditions – reprezintă setul de condiții la care se forțează ieșirea din buclă și nu este obligatoriu, iar dacă lipsește atunci bucla are o singură iterație

Observații

- După fiecare procesare a unui ciclu, controlul revine la începutul ciclului și se reia execuția instrucțiunilor
- Instrucțiunea **EXIT** este folosită pentru ieșirea din ciclu, cu sau fără verificare de condiție
- **EXIT** poate fi executată fie ca o acțiune dintr-un **IF** în cadrul blocului **LOOP**, fie ca o instrucțiune individuală în cadrul unui ciclu
- O clauză **WHEN** poate fi înlocuită de instrucțiunea **IF**, cum se vede în construcția următoare:

```
LOOP
    statements;
    IF conditions THEN
        statements_in_if;
        EXIT;  -- exit loop immediately
    END IF;
END LOOP;
```

Ex. 8. În exemplul următor se dorește să se facă o listă cu data angajării și veniturile angajaților care fac parte dintr-un departament specificat.

```
set serveroutput ON;
DECLARE
    deptid          INTEGER;
    contor          INTEGER := 100;
    numeangajat     VARCHAR2(30);
    venit           NUMBER(10);
    dataangajare    DATE;
    numedepartament departments.department_name%TYPE;
BEGIN
    deptid := &iddept; --90

    SELECT department_name
    INTO   numedepartament
    FROM   departments
    WHERE  department_id = deptid;

    dbms_output.Put_line(Rpad('Nume', 30, '=')
                         || Rpad('Data Angajare', 15, '=')
                         ||Lpad('Venit', 10, '='));

    LOOP
        contor := contor + 1;

        BEGIN
            SELECT first_name
                   || ' '
                   || last_name,
                   hire_date,
                   salary + ( Nvl(commission_pct, 0) * salary )
            INTO   numeangajat, dataangajare, venit
            FROM   employees
            WHERE  department_id = deptid
                   AND employee_id = contor;

            dbms_output.Put_line(Rpad(numeangajat, 30, '=')
                                 ||Rpad(dataangajare, 15, '=')
                                 ||Lpad(venit, 10, '='));
        EXCEPTION -- nu poate fi folosit decat intr-un bloc
            WHEN no_data_found THEN
              NULL;
        END;

        exit WHEN contor = 206;
    END LOOP;
EXCEPTION
    WHEN no_data_found THEN
      dbms_output.Put_line('Nu sunt angajati in departamentul '
                           ||deptid);
END;
```

Instrucțiunea WHILE

Instructiunea WHILE este folosită pentru realizarea unei structuri repetitive condiționate.

Sintaxa instrucțiunii este:

```
WHILE conditions
LOOP
    statements;
END LOOP;
```

Unde:

- conditions – reprezintă setul de condiții pentru execuția buclei
- statements – reptezintă secvența de instrucțiuni executată în cadrul buclei

Ex. 9. Să se modifice exercițiul precedent astfel încât să folosească instrucțiunea WHILE.

```
set serveroutput ON;
DECLARE
    deptid          INTEGER;
    contor          INTEGER := 100;
    numeangajat     VARCHAR2(30);
    venit           NUMBER(10);
    dataangajare    DATE;
    numedepartament departments.department_name%TYPE;
BEGIN
    deptid := &iddept; --90

    SELECT department_name
    INTO   numedepartament
    FROM   departments
    WHERE  department_id = deptid;

    dbms_output.Put_line(Rpad('Nume', 30, '=')
                         || Rpad('Data Angajare', 15, '=')
                         ||Lpad('Venit', 10, '='));

    WHILE contor <= 206
    LOOP
        contor := contor + 1;

        BEGIN
            SELECT first_name
                   || ' '
                   || last_name,
                   hire_date,
                   salary + ( Nvl(commission_pct, 0) * salary )
            INTO   numeangajat, dataangajare, venit
            FROM   employees
            WHERE  department_id = deptid
                   AND employee_id = contor;

            dbms_output.Put_line(Rpad(numeangajat, 30, '=')
                                 ||Rpad(dataangajare, 15, '=')
                                 ||Lpad(venit, 10, '='));
        EXCEPTION -- nu poate fi folosit decat intr-un bloc
            WHEN no_data_found THEN
              NULL;
        END;
    END LOOP;
EXCEPTION
    WHEN no_data_found THEN
      dbms_output.Put_line('Nu sunt angajati in departamentul '
                           ||deptid);
END;
```

Instrucțiunea FOR

Instrucțiunea FOR este folosită pentru realizarea unei structuri repetitive.

Sintaxa instrucțiunii este:

```
FOR contor IN [REVERSE] value1 .. value2
LOOP
    statements
END LOOP;
```

Unde:

- contor – este contorul folosit pentru incrementarea (decrementarea) iterațiilor
- value1 .. value2 – reprezintă valorile minime și maxime ale contorului (value1 ⇐ value2)
- statements – reprezintă secvența de instrucțiuni executate în cadrul buclei

Observații:

- Întotdeauna trebuie să se îndeplinească condiția **value1 < value2** pentru a se intra în bucla **FOR**, indiferent dacă se folosește **IN** sau **IN REVERSE**
- Incrementarea (cazul cu **IN**) sau decrementarea (cazul **IN REVERSE**) contorului este făcută automat cu 1 (cu data tipul PLS_INTEGER)

Ex. 10. Să se modifice exercițiul precedent astfel încât să folosească instrucțiunea FOR.

```sql
set serveroutput ON;
DECLARE
    deptid          INTEGER;
    contor          INTEGER := 100;
    numeangajat     VARCHAR2(30);
    venit           NUMBER(10);
    dataangajare    DATE;
    numedepartament departments.department_name%TYPE;
BEGIN
    deptid := &iddept; --90

    SELECT department_name
    INTO   numedepartament
    FROM   departments
    WHERE  department_id = deptid;

    dbms_output.Put_line(Rpad('Nume', 30, '=')
                         || Rpad('Data Angajare', 15, '=')
                         ||Lpad('Venit', 10, '='));

    FOR contor IN 100 .. 206
    LOOP
        BEGIN
            SELECT first_name
                   || ' '
                   || last_name,
                   hire_date,
                   salary + ( Nvl(commission_pct, 0) * salary )
            INTO   numeangajat, dataangajare, venit
            FROM   employees
            WHERE  department_id = deptid
                   AND employee_id = contor;

            dbms_output.Put_line(Rpad(numeangajat, 30, '=')
                                 ||Rpad(dataangajare, 15, '=')
                                 ||Lpad(venit, 10, '='));
        EXCEPTION -- nu poate fi folosit decat intr-un bloc
            WHEN no_data_found THEN
              NULL;
        END;
    END LOOP;
EXCEPTION
    WHEN no_data_found THEN
      dbms_output.Put_line('Nu sunt angajati in departamentul '
                           ||deptid);
END;
```

Observații:

- Buclele pot fi imbricate pe mai multe niveluri
- În general, o buclă internă nu termină execuția unui bucle care o include, exceptând cazul când este generată o excepție
- Pentru salturi de instrucțiuni, se folosește instrucțiunea **GOTO**
- **GO** poate fi folosit în oricare dintre instrucțiunile prezentate
- Instrucțiunea **GOTO** poate fi folosită și să se iasă dintr-o buclă

Ex. 11.Exemplu de folosire a instrucțiunii GOTO.

```
set serveroutput ON;
DECLARE
    contor INTEGER;
BEGIN
    FOR contor IN 1..10 LOOP
        IF contor = 5 THEN
          GOTO label1;
        ELSE
          GOTO label2;
        END IF;

        <<label1>>
        dbms_output.Put_line('Contorul este la jumatate!');

        <<label2>>
        dbms_output.Put_line('Valoare contorului este '
                             ||contor);
    END LOOP;
END;
```

Ex. 12. Exeplu de ieșise dintr-o buclă folosind GOTO.

```
set serveroutput ON;
DECLARE
    contor INTEGER;
BEGIN
    FOR contor IN 1..10 LOOP
        IF contor = 5 THEN
          GOTO label1;
        ELSE
          GOTO label2;
        END IF;

        <<label2>>
        dbms_output.Put_line('Valoarea contorului este '
                             ||contor);
    END LOOP;

    <<label1>>
    dbms_output.Put_line('Am iesit din bucla!');
END;
```

Ex. 13. Să se scrie un bloc PL/SQL care face o listă cu numărul angajaților care au veniturile mai mici de 4000 (considerate venituri mici) și mai mari de 4000 (considerate venituri mari), pentru fiecare departament. Se va face o construcție folosind structuri de control și cicluri. Pentru departamentele care nu au niciun angajat se va afișa un mesaj.

```sql
set serveroutput ON;
DECLARE
    contor           INTEGER := 0;
    contor1          NUMBER;
    contor2          NUMBER;
    suma1            NUMBER;
    suma2            NUMBER;
    numedepartament departments.department_name%TYPE;
BEGIN
    dbms_output.Put_line(Rpad('Nume Departament', 20)
                         || Rpad('Nr. Sal. Mici', 20)
                         ||Rpad('Suma Sal. Mici', 20)
                         ||Rpad('Nr. Sal. Mari', 20)
                         ||Rpad('Suma Sal. Mari', 20));

    LOOP
        BEGIN
            contor := contor + 10;

            SELECT department_name
            INTO   numedepartament
            FROM   departments
            WHERE  department_id = contor;

            SELECT Count(*),
                   SUM(salary + ( Nvl(commission_pct, 0) * salary ))
            INTO   contor1, suma1
            FROM   employees
            WHERE  department_id = contor
                   AND ( salary + ( Nvl(commission_pct, 0) * salary ) ) < 4000;

            SELECT Count(*),
                   SUM(salary + ( Nvl(commission_pct, 0) * salary ))
            INTO   contor2, suma2
            FROM   employees
            WHERE  department_id = contor
                   AND ( salary + ( Nvl(commission_pct, 0) * salary ) ) >= 4000;

            dbms_output.Put_line(Rpad(numedepartament, 20)
                                 ||Rpad(contor1, 20)
                                 ||Rpad(Nvl(suma1, 0), 20)
                                 ||Rpad(contor2, 20)
                                 ||Rpad(Nvl(suma2, 0), 20));

            IF contor1 = 0
               AND contor2 = 0 THEN
              dbms_output.Put_line('Departamentul '
                                   ||numedepartament
                                   ||' nu are angajati.');
            END IF;
        EXCEPTION
            WHEN no_data_found THEN
              IF contor > 260 THEN
                EXIT;
              ELSE
                NULL;
              END IF;
        END;
    END LOOP;
END;
```

Tipuri compuse și colecție

Tipul RECORD

Tipurile scalare nu au componente interne, în timp ce tipurile compuse au componente interne care se pot manipula.

RECORD este un tip compus:

- Folosit pentru a accesa liniile tabelelor sub formă de matrice
- Asemănător cu o înregistrare a unei tabele cu excepția faptului că numele coloanelor sunt proprii fiecărei înregistrări

Declararea tipului RECORD se face conform următoarei sintaxe:

```
TYPE nume_tip IS RECORD (
   field_1 {datatype | variable%TYPE | TABLE.column%TYPE | table%ROWTYPE} [ [NOT NULL] [{:= | DEFAULT} expression_1] ],
   field_2 {datatype | variable%TYPE | TABLE.column%TYPE | table%ROWTYPE} [ [NOT NULL] [{:= | DEFAULT} expression_2] ],
   ...
   field_n {datatype | variable%TYPE | TABLE.column%TYPE | table%ROWTYPE} [ [NOT NULL] [{:= | DEFAULT} expression_n] ]
);
```

Începând cu Oracle9i, pentru tipul RECORD sunt introduse câteva facilităţi:

- Se poate insera (INSERT) o linie într-un tabel utilizând tipul RECORD
- Se poate actualiza (UPDATE) o linie într-un tabel utilizând tipul RECORD (cu sintaxa SET ROW)
- Se poate regăsi şi returna sau şterge informaţia din clauza RETURNING a comenzilor UPDATE sau DELETE
- Dacă în comenzile UPDATE sau DELETE se modifică mai multe linii, atunci pot fi utilizate colecţii de înregistrări cu sintaxa BULK COLLECT INTO

Ex. 14. Un exemplu de utilizare a tipului RECORD.

```
set serveroutput ON;
DECLARE
    TYPE deptrecord IS RECORD (
      iddept NUMBER,
      numedept VARCHAR2(30),
      idmanager NUMBER,
      locatie NUMBER );
    recdept  DEPTRECORD;
    deptinfo departments%ROWTYPE;
BEGIN
    SELECT *
    INTO   deptinfo
    FROM   departments
    WHERE  department_id = 10;

    recdept := deptinfo;

    dbms_output.Put_line('Denumirea departemanetului '
                         ||recdept.iddept
                         ||' este '
                         ||recdept.numedept);
END;
```

Ex. 15. Ex. 15. Folosindu-se tipul compus RECORD:

- Să se creeze un record de forma tabelului dept
- Să se insereze un nou departament folosind o variabilă de tip record
- Să se selecteze înregistrarea inserată într-un record
- Să se updateze înregistrarea inserată folosindu-se record și să se returneze într-un record informații
- Să se șteargă departamentul creat și să se returneze într-un record informații.

```
set serveroutput ON;
DECLARE
    TYPE myrecord IS RECORD (
      iddepartament departments.department_id%TYPE,
      numedepartament departments.department_name%TYPE,
      idmanager departments.manager_id%TYPE,
      idlocatie departments.location_id%TYPE );
    deptrecord       MYRECORD;
    deptrecordx      MYRECORD;
    deptrecordupdate MYRECORD;
    deptrecordinsert MYRECORD;
BEGIN
    deptrecordinsert.iddepartament := 8001;
    deptrecordinsert.numedepartament := 'IT Department';
    deptrecordinsert.idlocatie := 1700;
    deptrecordinsert.idmanager := NULL;

    -- insert folosind record
    INSERT INTO departments
    VALUES     (deptrecordinsert.iddepartament,
                deptrecordinsert.numedepartament,
                deptrecordinsert.idmanager,
                deptrecordinsert.idlocatie);

    -- select folosind record
    SELECT *
    INTO   deptrecord
    FROM   departments
    WHERE  department_id = deptrecordinsert.iddepartament;

    dbms_output.Put_line('Informatii dupa insert '
                         ||deptrecord.iddepartament
                         ||' '
                         ||deptrecord.numedepartament
                         ||' '
                         ||deptrecord.idmanager
                         ||' '
                         ||deptrecord.idlocatie);

    deptrecordupdate.iddepartament := deptrecordinsert.iddepartament;
    deptrecordupdate.numedepartament := 'Operatii';
    deptrecordupdate.idlocatie := 2400;

    -- update frolosind record
    UPDATE departments
    SET    ROW = deptrecordupdate
    WHERE  department_id = deptrecordinsert.iddepartament
    returning department_id, department_name, manager_id, location_id INTO
    deptrecord;

    dbms_output.Put_line('Informatii dupa update '
                         ||deptrecord.iddepartament
                         ||' '
                         ||deptrecord.numedepartament
                         ||' '
                         ||deptrecord.idmanager
                         ||' '
                         ||deptrecord.idlocatie);
```

```
59
60      -- stergere inregistrare
61      DELETE FROM departments
62      WHERE  department_id = deptrecordupdate.iddepartament
63      returning department_id, department_name, manager_id, location_id INTO
64      deptrecordx;
65
66      dbms_output.Put_line('Informatii dupa delete '
67                           ||deptrecordx.iddepartament
68                           ||' '
69                           ||deptrecordx.numedepartament
70                           ||' '
71                           ||deptrecordx.idlocatie);
72  END;
```

Tipuri colecție

Tipurile colecție sunt:

- Varrays
- Index-by table
- Nested tables

Tipurile colecție au **atribute** și **metode**. (În funcție de tip, atributele se comporta diferit).

Atribute și metode	Descriere
COUNT	Numărul de componente din colecţie
FIRST	Indicele primului element din tablou
LAST	Indicele ultimului element din colecție
EXISTS	Întoarce TRUE dacă există în coleție componenta cu indexul specificat
NEXT	Întoarce indicele următoarei componente
PRIOR	Întoarce indicele componentei anterioare
DELETE	Șterge una sau mai multe componente
EXTEND	Adaugă elemente la sfârşitul colecției
LIMIT	Întoarce numărul maxim de elemente al unei colecţii (pentru vectori), null pentru tablouri imbricate
TRIM	Șterge elementele de la sfârşitul unei colecţii

Tipul VARRAY

Vectorii (varray) sunt structuri asemănătoare vectorilor din limbajele C sau Java. Vectorii au o dimensiune maximă (constantă) stabilită la declarare. Se utilizează pentru modelarea relaţiilor one-to-many, atunci când numărul maxim de elemente din partea "many" este cunoscut şi ordinea elementelor este importantă. Fiecare element are un index. **Indexarea începe de la 1**.

Tipul de date vector este declarat utilizând sintaxa:

```
TYPE name IS {VARRAY | VARYING ARRAY} (max_length) OF element_type [NOT NULL];
```

Ex. 16. Exemplul folosire varray.

```
set serveroutput ON;
DECLARE
    TYPE secventa IS varray(5) OF VARCHAR2(10);
    v_sec SECVENTA := Secventa ('alb', 'negru', 'rosu', 'verde');
    t     BOOLEAN;
BEGIN
    dbms_output.Put_line(V_sec(1)
                         ||' '
                         ||V_sec(2)
                         ||' '
                         ||V_sec(3)
                         ||' '
                         ||V_sec(4));

    V_sec(4) := 'rosu';

    dbms_output.Put_line(v_sec.limit);

    dbms_output.Put_line(V_sec(1)
                         ||' '
                         ||V_sec(2)
                         ||' '
                         ||V_sec(3)
                         ||' '
                         ||V_sec(4));

    -- desi s-au declarat 5 elemente pentru vector
    -- acest print va da eroare pentru ca vextorul
    -- s-a initializat doar cu 4
    -- dbms_output.put_line(v_sec(5));
    -- pentru a nu da eroare trebuie extins
    v_sec.extend; -- adauga un element null

    dbms_output.Put_line(V_sec(1)
                         ||' '
                         ||V_sec(2)
                         ||' '
                         ||V_sec(3)
                         ||' '
                         ||V_sec(4)
                         ||' '
                         ||V_sec(5));

    V_sec(5) := 'albastru';

    dbms_output.Put_line(V_sec(1)
                         ||' '
                         ||V_sec(2)
                         ||' '
                         ||V_sec(3)
                         ||' '
                         ||V_sec(4)
```

```
53                                  ||' '
54                                  ||V_sec(5));
55
56      dbms_output.Put_line(v_sec.limit);
57  -- extinderea la 6 elemente va genera o eroarea
58  -- deoarece vectorul a fost declarat cu lungimea maxima 5
59  -- v_sec.extend;
60  END;
```

Tipul ASSOCIATIVE ARRAY

Tipul ASSOCIATIVE ARRAY se mai numește și INDEX BY TABLE (tabel indexat).

Un table indexat în PL/SQL are două componente:

- ~~Coloană ce cuprinde cheia primară pentru a avea acces la liniile tabelului~~
- ~~O coloană care include valoarea efectivă a elementelor tabelului~~
- Indexul pentru a avea acces la valoare
- Valoarea efectivă a elementului

Declararea tipului ASSOCIATIVE ARRAY se face cu următoarea sintaxă:

```
TYPE name IS TABLE OF {data_type | variable%TYPE | TABLE.column%TYPE [NOT NULL] | table%ROWTYPE} INDEX BY data_type_index;
```

Unde:

- data_type_index poate fi String sau Integer

Observații:

- Elementele unui ASSOCIATIVE ARRAY nu sunt într-o ordine particulară şi pot fi inserate cu indecși arbitrar aleși
- Deoarece nu există constrângeri de dimensiune, dimensiunea tabelului se modifică dinamic
- Un ASSOCIATIVE ARRAY nu poate fi iniţializat în declararea sa
- Un ASSOCIATIVE ARRAY neiniţializat este vid (nu conţine nici valori, nici indecși)
- Un element al unui ASSOCIATIVE ARRAY este nedefinit atâta timp cât nu are atribuită o valoare efectivă
- Dacă se face referire la o linie care nu există, atunci se produce excepţia **NO_DATA_FOUND**.

Ex. 17. Exemplu de folosire ASSOCIATIVE ARRAY.

```
set serveroutput ON;
DECLARE
    TYPE tabindex
      IS TABLE OF departments%ROWTYPE INDEX BY BINARY_INTEGER;
    mytabindex TABINDEX;
    rowdept    departments%ROWTYPE;
BEGIN
    SELECT *
    INTO   rowdept
    FROM   departments
    WHERE  department_id = 10;

    Mytabindex(1) := rowdept;

    SELECT *
    INTO   Mytabindex(2)
    FROM   departments
    WHERE  department_id = 20;

    dbms_output.Put_line(Rpad('Id', 20)
                         ||Rpad('Nume', 20)
                         ||Lpad('IDLocatie', 20));

    dbms_output.Put_line(Rpad(Mytabindex(1).department_id, 20)
                         ||Rpad(Mytabindex(1).department_name, 20)
                         ||Lpad(Mytabindex(1).location_id, 20));

    dbms_output.Put_line(Rpad(Mytabindex(2).department_id, 20)
                         ||Rpad(Mytabindex(2).department_name, 20)
                         ||Lpad(Mytabindex(2).location_id, 20));
END;
```

Tipul Nested Table

Un tablou imbricat este o mulţime neordonată de elemente de acelaşi tip. Valorile de acest tip:

- Pot fi stocate în baza de date
- Pot fi prelucrate direct în instrucţiuni SQL
- Au excepţii predefinite proprii

Comanda de declarare a tipului de date tablou imbricat are sintaxa:

```
TYPE name IS TABLE OF {data_type | variable%TYPE | TABLE.column%TYPE [NOT NULL] | table%ROWTYPE} [NOT NULL];
```

Observații:

- Pentru adăugarea de linii intr-un tablou imbricat, acesta trebuie sa fie inițializat cu ajutorul constructorului
- PL/SQL apelează un constructor numai în mod explicit
- Tabelele indexate nu au constructori

- Constructorul primeşte ca argumente o listă de valori numerotate în ordine, de la 1 la numărul de valori date ca parametrii constructorului
- Dimensiunea iniţială a colecţiei este egală cu numărul de argumente date în constructor, când aceasta este iniţializată
- Pentru vectori nu poate fi depăşită dimensiunea maximă precizată la declarare
- Atunci când constructorul este fără argumente va crea o colecție fără niciun element (vida), dar care are valoarea not null

Ex. 18. Exemplu de folosire al unui tabel imbricat.

```
set serveroutput ON;
DECLARE
    TYPE tabimbricat
      IS TABLE OF VARCHAR2(30);
    curs TABIMBRICAT;
BEGIN
    curs := Tabimbricat('Baze de date', 'Proiectare Baze de Date');

    dbms_output.Put_line('Cursul echivalent pentru BD2 este '
                         ||Curs(2));
END;
```

Colecții stocate

Tipurile colecție pot fi definite direct în baza de date.

Collection Type	Number of Elements	Subscript Type	Dense or Sparse	Where Created	Can Be Object Type Attribute
Associative array (or index-by table)	Unbounded	String or PLS_INTEGER/BINARY_INTEGER	Either	Only in PL/SQL block	No
Nested table	Unbounded	Integer	Starts dense, can become sparse	Either in PL/SQL block or at schema level	Yes
Variable-size array (varray)	Bounded	Integer	Always dense	Either in PL/SQL block or at schema level	Yes

Ex. 19. Definirea tipurilor direct în baza de date.

```sql
-- folosind vector
CREATE OR REPLACE TYPE listaproiecte AS varray(20) OF VARCHAR(20);
/

CREATE TABLE proiecte
  (
     iddept   NUMBER(2),
     denumire VARCHAR2(20),
     buget    NUMBER(11, 2),
     proiect  listaproiecte
  );

--folosind nested table
CREATE OR REPLACE TYPE listaproiecte_nt
  AS TABLE OF VARCHAR2(30);
/

CREATE TABLE proiecte_nt
  (
     iddept   NUMBER(2),
     denumire VARCHAR2(20),
     buget    NUMBER(11, 2),
     proiect  LISTAPROIECTE_NT
  ) NESTED TABLE proiect STORE AS proiect_store;

set serveroutput ON;
DECLARE
    proiect listaproiecte;
BEGIN
    proiect := listaproiecte('E-commerce', 'Carduri bancare');

    INSERT INTO proiecte
    VALUES     (20,
                'Proiectare',
                165580,
                proiect);

    dbms_output.put_line('Proiectul bancar se numeste '
                         ||proiect(2));
END;
/
```

Atunci când o coloană dintr-un tabel are tipul NESTED TABLE, Oracle stochează informația din această coloană în aceași zonă de stocare. De aceea trebuie să specificăm unde se zona de stocare la crearea tabelului folosind `NESTED TABLE proiect STORE AS proiect_store;`

BULK COLLECT

Pentru a insera mai multe valori într-o variabilă de tip colecție se poate folosi BULK COLLECT INTO.

Ex. 20. Folosirea BULK COLLECT INTO cu varray.

```
set serveroutput ON;
DECLARE
    TYPE vector IS varray(27) OF departments.department_id%TYPE;
    myvector VECTOR;
    contor   INTEGER;
BEGIN
    dbms_output.Put_line('folosire bulk collect into varray');

    SELECT department_id
    bulk   collect INTO myvector
    FROM   departments;

    FOR contor IN myvector.first .. myvector.last LOOP
        dbms_output.Put_line('myVector('
                             ||contor
                             ||')='
                             ||Myvector(contor));
    END LOOP;
END;
```

Ex. 21. Folosirea BULK COLLECT INTO cu index-by table.

```
set serveroutput ON;
DECLARE
    TYPE tableindexat
      IS TABLE OF departments%ROWTYPE INDEX BY BINARY_INTEGER;
    contor     INTEGER;
    tblindexat TABLEINDEXAT;
BEGIN
    dbms_output.Put_line('folosire bulk collect into index-by table');

    SELECT *
    bulk   collect INTO tblindexat
    FROM   departments;

    FOR contor IN tblindexat.first .. tblindexat.last LOOP
        dbms_output.Put_line('contor='
                             ||contor
                             ||' tblIndexat.department_id='
                             ||Tblindexat(contor).department_id
                             ||' tblIndexat.department_name='
                             ||Tblindexat(contor).department_name
                             ||' tblIndexat.location_id='
                             ||Tblindexat(contor).location_id);
    END LOOP;
END;
```

Ex. 22. Folosirea BULK COLLECT INTO cu nested table.

```
set serveroutput ON;
DECLARE
    TYPE tablenested
      IS TABLE OF employees%ROWTYPE;
    contor    INTEGER;
    tblnested TABLENESTED;
BEGIN
    dbms_output.Put_line('folosire bulk collect into nested table');

    SELECT *
    bulk   collect INTO tblnested
    FROM   employees
    WHERE  department_id = 10;

    FOR contor IN tblnested.first .. tblnested.last LOOP
        dbms_output.Put_line('contor='
                             ||contor
                             ||' tblNested.EMPLOYEE_ID='
                             ||Tblnested(contor).employee_id
                             ||' tblNested.FullName='
                             ||Tblnested(contor).first_name
                             ||' '
                             || Tblnested(contor).last_name
                             ||' tblNested.EMAIL='
                             ||Tblnested(contor).email
                             ||' tblNested.PHONE_NUMBER='
                             ||Tblnested(contor).phone_number
                             ||' tblNested.HIRE_DATE='
                             ||Tblnested(contor).hire_date
                             ||' tblNested.JOB_ID='
                             ||Tblnested(contor).job_id
                             ||' tblNested.SALARY='
                             ||Tblnested(contor).salary
                             ||' tblNested.COMMISSION_PCT='
                             ||Tblnested(contor).commission_pct
                             ||' tblNested.MANAGER_ID='
                             ||Tblnested(contor).manager_id
                             ||' tblNested.DEPARTMENT_ID='
                             ||Tblnested(contor).department_id);
    END LOOP;
END;
```

Colecții de înregistrări

Se poate crea un “tabel” care sa aibă coloane definite de dezvoltatori folosind un table(index-by sau nested) care să fie de tipul RECORD.

Ex. 23. Crearea unui tabel cu coloanele definite de utilizator.

```
set serveroutput ON;
DECLARE
    TYPE myrecord IS RECORD(
      numeangajat VARCHAR2(50),
      numedepartament departments.department_name%TYPE );
    TYPE mytabel
      IS TABLE OF MYRECORD;
    tbl    MYTABEL;
    contor INTEGER;
BEGIN
    SELECT e.first_name
           || ' '
           || e.last_name,
           d.department_name
    bulk   collect INTO tbl
    FROM   employees e
           INNER join departments d ON e.department_id = d.department_id;

    FOR contor IN tbl.first .. tbl.last LOOP
        dbms_output.Put_line(Tbl(contor).numeangajat
                             ||' lucreaza in departamentul '
                             ||Tbl(contor).numedepartament);
    END LOOP;
END;
```

bdd/laboratoare/02.txt · Last modified: 2024/10/08 01:43 by alex.petrescu

Laboratorul 03 - PL/SQL Tratarea excepțiilor

Conținut

- Excepții predefinite
- Excepții nedefinite
- Excepții definite

Tratarea excepțiilor

Excepțiile în PL/SQL au asociate coduri de eroare și pot apare în timpul execuției unui bloc.

Dacă excepția nu este tratată, adică nu se specifică secvența de cod care trebuie executată în cazul apariției, sistemul de gestiune a bazei de date va încheia automat execuția blocul respectiv.

Sunt trei tipuri de excepții:

- Predefinite = sunt excepții tratate automat de către sistemul de gestiune;
- Nedefinite = sunt excepții tratate de către sistemul de gestiune, au coduri de eroare tip ORA-….
  Aceste erori pot fi interceptate și tratate de programator daca li se atașează un nume;
- Definite = sunt excepții definite și tratate de o secvență de program specificată de programator.

Excepții predefinite

Sunt excepțiile cele mai uzuale care sunt interpretate și tratate automat de SGBD.

Interpretarea excepțiilor se realizează în secțiunea EXCEPTION.

Sintaxa generală are forma:

```
DECLARE
        ...
BEGIN
        ...
EXCEPTION
        WHEN exception_1 [OR exception_2 ...] THEN statements_1;
        ...
        WHEN exception_k [OR exception_k+1 ...] THEN statements_k;
        [WHEN OTHERS THEN statements_n;]
END;
```

Unde:

- exception_i reprezintă numele excepției
- statements_i reprezintă secvența de instrucțiuni care tratează excepția.

Clauza WHEN OTHERS se specifică întotdeauna ultima, pentru ca ramura să preia toate excepțiile netratate.

Atunci când este tratată o excepție, o singură ramură WHEN din secțiunea EXCEPTION este tratată.

Găsiți toate excepțiile la următoarele adresa: http://docs.oracle.com/database/121/ERRMG/toc.htm [http://docs.oracle.com/database/121/ERRMG/toc.htm]

Ex. 1 Să se introducă un ID de departament de la tastatură și să se afișeze la terminal numele acestuia:

```sql
SET serveroutput ON;

DECLARE
    idDepartament NUMBER;
    numeDepartament departments.department_name%TYPE;
BEGIN
    idDepartament := &idDepartament;

    SELECT department_name INTO numeDepartament
        FROM departments WHERE department_id = idDepartament;

    dbms_output.put_line('Numele departamentului '
        || idDepartament || ' este ' || numeDepartament);
END;
/
```

Se observă că pentru idDepartament = 10 blocul funcționează normal.

```
Numele departamentului 10 este Administration


PL/SQL procedure successfully completed.
```

Pentru idDepartament = 1 se generează o eroare ORA-01403 deoarece în tabela departments nu există acest departament.

```
Error report -
ORA-01403: no data found
ORA-06512: at line 7
01403. 00000 -  "no data found"
*Cause:    No data was found from the objects.
*Action:   There was no data from the objects which may be due to end of fetch.
```

Pentru a evita o astfel de situație trebuie să fie tratată excepția no_data_found.

Ex. 2 Tratarea excepție no_data_found pentru blocul precedent:

```
SET serveroutput ON;

DECLARE
    idDepartament NUMBER;
    numeDepartament departments.department_name%TYPE;
BEGIN
    idDepartament := &idDepartament;

    SELECT department_name INTO numeDepartament
        FROM departments WHERE department_id = idDepartament;

    dbms_output.put_line('Numele departamentului '
        || idDepartament || ' este ' || numeDepartament);
EXCEPTION
    WHEN no_data_found THEN
        dbms_output.put_line('Departamentul ' || idDepartament
            || ' nu exista in baza de date.');
END;
/
```

Rezultatul execuției pentru idDepartament = 1 este:

```
Departamentul 1 nu exista in baza de date.


PL/SQL procedure successfully completed.
```

Excepții nedefinite

Sunt excepții pe care nu le tratează automat sistemul de gestiune dar pot fi tratate prin interceptarea codului de eroare ORA-….. și executarea unui cod de program dorit.

Sintaxa erorilor nedefinite este:

```
DECLARE
        ...
        exception_name EXCEPTION;
        PRAGMA EXCEPTION_INIT(exception_name,                  exception_code);
        ...
BEGIN
        ...
EXCEPTION
        ...
        WHEN exception_name THEN statements;
        ...
END;
```

Unde:

- exception_name reprezintă numele excepției
- error_code reprezintă codul de eroare pentru care se declanșează excepția
- statements reprezintă secvența de instrucțiuni care tratează excepția

În cazul acestor excepții trebuie asociat codul de eroare cu numele excepției în secțiunea DECLARE a blocului prin cuvintele cheie rezervate PRAGMA EXCEPTION_INIT.

Pentru interpretarea erorilor se folosește funcția de sistem SQLCODE, care returnează următoarele valori:

- 0 – dacă nu s-a întâlnit o eroare
- 1 – excepție definită de utilizator
- +100 – excepția NO_DATA_FOUND
- Valori negative – pentru alte erori ale bazei

Mesajul erorii se afișează cu funcția SQLERRM.

Ex. 3 În următorul exemple, se dorește să se facă o inserare în tabelul employees a unui nou angajat, dar fără să i se atribuie un employee_id, care este o coloană definită de tip not null.

```
SET serveroutput ON;

DECLARE
    insert_angajat EXCEPTION;
    PRAGMA exception_init(insert_angajat, -01400);
BEGIN
    INSERT INTO employees(first_name, last_name, hire_date, salary)
        VALUES ('Ion', 'Ionescu', '14-AUG-2020', 1450);
EXCEPTION
    WHEN insert_angajat THEN
        dbms_output.put_line('Nu se accepta inregistrari noi ' ||
            'in tabela employees fara employee_id');
        dbms_output.put_line('Cod eroare SQL ' || SQLCODE);
        dbms_output.put_line('Mesaj ' || SQLERRM);
END;
/
```

Rezultatul execuției este:

```
Nu se accepta inregistrari noi in tabela employees fara employee_id
Cod eroare SQL -1400
Mesaj ORA-01400: cannot insert NULL into ("SYSTEM"."EMPLOYEES"."EMPLOYEE_ID")


PL/SQL procedure successfully completed.
```

Se observă că funcția SQLCODE a generat eroarea ORA-01400 cu mesajul cannot insert NULL into (“SYSTEM”.”EMPLOYEES”.”EMPLOYEE_ID”).

Codul de eroare a fost interceptat de excepția insert_angajat, tratarea acesteia fiind înfăptuită prin afișarea mesajului ‘Nu se accepta inregistrari noi in tabela employees fara employee_id’.

Motorul PL/SQL generează propriile coduri de eroare în formatul PLS-xxxxx, unde xxxxx este codul de eroare. Acest cod este diferit de codul ORA-yyyyy generat de SGBD. Un cod de eroare PL/SQL este prefațat de literele PLS și urmat de un număr unic. Cu ajutorul acestui număr se poate identifica definiția erorii folosindu-se documentația ORACLE.

Ex. 4 Se modifică exemplul 3 adăugându-se în secțiunea EXCEPTION tratarea unei excepții care nu a fost declarată.

```plsql
SET serveroutput ON;

DECLARE
    insert_angajat EXCEPTION;
    PRAGMA exception_init(insert_angajat, -01400);
BEGIN
    INSERT INTO employees(first_name, last_name, hire_date, salary)
        VALUES ('Ion', 'Ionescu', '14-AUG-2020', 1450);
EXCEPTION
    WHEN insert_angajat THEN
        dbms_output.put_line('Nu se accepta inregistrari noi ' ||
            'in tabela employees fara employee_id');
        dbms_output.put_line('Cod eroare SQL ' || SQLCODE);
        dbms_output.put_line('Mesaj ' || SQLERRM);
    WHEN exceptie_eronata THEN null;
END;
/
```

Rezultatul execuției este:

```
Error report -
ORA-06550: line 13, column 10:
PLS-00201: identifier 'EXCEPTIE_ERONATA' must be declared
ORA-06550: line 0, column 0:
PL/SQL: Compilation unit analysis terminated
```

Un program PL/SQL își încetează execuția la detectarea unei erori interne de execuție, cu excepția cazului în care sunt tratate excepțiile în alt mod de catre programator. În anumite cazuri, programul nu trebuie să-și înceteze execuția atunci când apare o eroare. Se pot include secvențe de program care să trateze erorile astfel încât procesarea să continue și după detectarea erorilor de execuție.

Excepții definite

Sunt excepții care nu se bazează pe codurile de eroare generate de sistem și au următoarea sintaxă:

```
DECLARE
        ...
        exception_name EXCEPTION
        ...
BEGIN
        ...
        IF condition THEN
                RAISE exception_name
        END IF;
        ...
EXCEPTION
        ...
        WHEN exception_name THEN statements;
        ...
END;
```

Unde:

- exception_name – reprezintă numele atribuit excepției
- RAISE – reprezintă instrucțiunea de interpretare și generare a excepției;
- statements – reprezintă secvența de instrucțiuni care tratează excepția;

Limbajul PL/SQL permite astfel programatorului să-și definească propriile excepții. Excepțiile definite de programator trebuie declarate în secțiunea DECLARE și trebuie generate în mod explicit cu instrucțiunea RAISE.

Declararea și interceptarea excepțiilor definite de programator impun următoarele acțiuni:

- Declararea excepției în secțiunea DECLARE.
- Utilizarea instrucțiunii RAISE pentru generarea excepției în secțiunea de execuție a blocului: RAISE exception_name
- Tratarea excepției în secțiunea EXCEPTION.

Ex. 5 Pentru angajații dintr-un departament, al cărui ID se introduce de la tastatură, care au comision null, să se genereze o excepție definită.

```
SET serveroutput ON;

DECLARE
    idDepartament NUMBER;
    contor NUMBER := 100;
    numeAngajat employees.first_name%TYPE;
    comision employees.commission_pct%TYPE;
    dataAngajare employees.hire_date%TYPE;
    lipsaComision EXCEPTION;
BEGIN
    idDepartament := &deptId;
    dbms_output.put_line(
        rpad('Nume', 15, ' ') ||
        rpad('Data angajare', 20, ' ') ||
        lpad('Comision', 15, ' ')
    );
    dbms_output.put_line(
        rpad('=', 15, '=') ||
        rpad('=', 20, '=') ||
        lpad('=', 15, '=')
    );

    LOOP
        contor := contor + 1;
        BEGIN
            SELECT first_name || ' ' || last_name, hire_date, commission_pct
            INTO numeAngajat, dataAngajare, comision
            FROM employees
            WHERE department_id = idDepartament AND employee_id = contor;

            IF comision IS NULL THEN
                RAISE lipsaComision;
            END IF;

            dbms_output.put_line(
                rpad(numeAngajat, 15, ' ') ||
                rpad(dataAngajare, 20, ' ') ||
                lpad(comision, 15, ' ')
            );
        EXCEPTION
            WHEN lipsaComision THEN
                dbms_output.put_line(
                    rpad(numeAngajat, 15, ' ') ||
                    rpad(dataAngajare, 20, ' ') ||
                    lpad('lipsa comision', 15, ' ')
                );
            WHEN no_data_found THEN NULL;
            WHEN OTHERS THEN NULL;
        END;
        EXIT WHEN contor = 300;
    END LOOP;
END;
/
```

Rezultatul execuției pentru idDepartament = 20 este:

```
Nume            Data angajare               Comision
==================================================
Michael Hartstel17-FEB-04             lipsa comision
Pat Fay         17-AUG-05             lipsa comision
```

Excepția definită se numește lipsaComision și se tratează prin afișarea mesajului 'lipsa comision'. Se observă că excepția nu se declanșează pe o eroare ci se verifică în secvența IF dacă comisionul angajatului este null. Dacă o excepție este generată într-un bloc, iar acesta nu are secțiunea EXCEPTION, sau nu se

dorește tratarea ei în blocul în care este generată, atunci excepția poate fi tratată în blocul superior sau în blocul apelant.

Trebuie tratate cu atenție excepțiile deoarece funcționalitatea programului se poate schimba în funcție de locul unde este tratată o excepție.

Ex. 6 Să se trateze excepția lipsaComision din exercițiul 5 în blocul principal.

```
SET serveroutput ON;

DECLARE
    idDepartament NUMBER;
    contor NUMBER := 100;
    numeAngajat employees.first_name%TYPE;
    comision employees.commission_pct%TYPE;
    dataAngajare employees.hire_date%TYPE;
    lipsaComision EXCEPTION;
BEGIN
    idDepartament := &deptId;
    dbms_output.put_line(
        rpad('Nume', 15, ' ') ||
        rpad('Data angajare', 20, ' ') ||
        lpad('Comision', 15, ' ')
    );
    dbms_output.put_line(
        rpad('=', 15, '=') ||
        rpad('=', 20, '=') ||
        lpad('=', 15, '=')
    );

    LOOP
        contor := contor + 1;
        BEGIN
            SELECT first_name || ' ' || last_name, hire_date, commission_pct
            INTO numeAngajat, dataAngajare, comision
            FROM employees
            WHERE department_id = idDepartament AND employee_id = contor;

            IF comision IS NULL THEN
                RAISE lipsaComision;
            END IF;

            dbms_output.put_line(
                rpad(numeAngajat, 15, ' ') ||
                rpad(dataAngajare, 20, ' ') ||
                lpad(comision, 15, ' ')
            );
        EXCEPTION
            WHEN no_data_found THEN NULL;
        END;
        EXIT WHEN contor = 300;
    END LOOP;
EXCEPTION
    WHEN lipsaComision THEN
        dbms_output.put_line(
            rpad(numeAngajat, 15, ' ') ||
            rpad(dataAngajare, 20, ' ') ||
            lpad('lipsa comision', 15, ' ')
        );
END;
/
```

Rezultatul execuției pentru idDepartament = 20 este:

```
Nume            Data angajare              Comision
=================================================
Michael Hartstel7-FEB-04            lipsa comision
```

Se observă că execuția se oprește în momentul în care apare primul angajat fără comision. Această funcționalitate anormală apare din cauza faptului că excepția care apare în sub-bloc este tratată în blocul principal, astfel se face o ieșire forțată din buclă.

Pentru a intercepta erorile de sistem există procedura stocată RAISE_APPLICATION_ERROR.

Această procedură permite transmiterea erorilor din proceduri stocate, dar poate fi folosită și în alte situații pentru transmiterea excepțiilor către procedurile apelate. Sintaxa este:

```
raise_application_error(error_number, message [, {TRUE | FALSE}])
```

Unde:

- error_number reprezintă codul ORA atribuit excepției (diferit de cele alocatei bazei de date, cu valori între -20000 și -20999);
- message este mesajul aferent excepției cu lungimea maximă de 2048 caractere;
- TRUE|FALSE este opțional și permite adăugarea mesajului la stiva de mesaje (pentru TRUE) alocată excepției (se adună mesajele de la mai multe niveluri. Implicit opțiunea este FALSE și în stivă rămâne mesajul singular pentru eroare, fără a mai prelua mesaje de la alte niveluri.

Ex. 7 Pentru un angajat care nu are comision să se definească o eroare care este tratată în momentul generării excepției și care are codul de eroare -20100.

```
SET serveroutput ON;

DECLARE
    ecuson employees.employee_id%TYPE;
    comision employees.commission_pct%TYPE;
    lipsaComision EXCEPTION;
    PRAGMA exception_init(lipsaComision, -20100);
BEGIN
    ecuson := &idAngajat;

    SELECT commission_pct
    INTO comision
    FROM employees
    WHERE employee_id = ecuson;

    IF comision IS NULL THEN
        raise_application_error(-20100, 'Angajatul nu are comision!');
    END IF;
EXCEPTION
    WHEN lipsaComision THEN
        dbms_output.put_line('Angajatul cu ID-ul introdus nu are comision');
        dbms_output.put_line('Cod eroare SQL: ' || SQLCODE);
        dbms_output.put_line('Mesaj eroare: ' || SQLERRM);
END;
/
```

Rezultatul execuției pentru ecuson = 100 este:

```
Angajatul cu ID-ul introdus nu are comision
Cod eroare SQL: -20100
Mesaj eroare: ORA-20100: Angajatul nu are comision!


PL/SQL procedure successfully completed.
```

Sunt situații când un program PL/SQL poate avea o funcționalitate diferită, în privința excepțiilor, din cauza erorilor generate de catre sistemul de gestiune.

Ex. 8 Să se calculeze suma veniturilor angajaților dintr-un departament care au o anumită funcție.

```sql
SET serveroutput ON;

DECLARE
    idDepartament departments.department_id%TYPE;
    numeDepartament departments.department_name%TYPE;
    salariu employees.salary%TYPE;
    functie employees.job_id%TYPE;
BEGIN
    idDepartament := &idDepartament;
    functie := '&numeFunctie';

    SELECT SUM(salary + salary * NVL(commission_pct, 0))
    INTO salariu
    FROM employees
    WHERE department_id = idDepartament
        AND Upper(job_id) = Upper(functie)
    GROUP BY department_id;

    SELECT department_name
    INTO numeDepartament
    FROM departments
    WHERE department_id = idDepartament;

    dbms_output.put_line(
        'Suma veniturilor in departamentul ' ||
        numeDepartament || ' este ' || to_char(salariu)
    );
EXCEPTION
    WHEN no_data_found THEN
        dbms_output.put_line(
            'In departamentul ' || idDepartament ||
            ' nu exista angajati cu functia ' || functie
        );
END;
/
```

Rezultatul execuției pentru idDepartament = 80 și functie = SA_REP este:

```
Suma veniturilor in departamentul Sales este 297490


PL/SQL procedure successfully completed.
```

Rezultatul execuției pentru idDepartament = 10 și functie = SA_REP este:

```
In departamentul 10 nu exista angajati cu functia SA_REP


PL/SQL procedure successfully completed.
```

Rezultatul execuției pentru idDepartament = 1 și functie = SA_REP este:

```
In departamentul 1 nu exista angajati cu functia SA_REP


PL/SQL procedure successfully completed.
```

Dacă din prima cerere SELECT se scoate clauza GROUP BY, adică:

```sql
SET serveroutput ON;

DECLARE
    idDepartament departments.department_id%TYPE;
    numeDepartament departments.department_name%TYPE;
    salariu employees.salary%TYPE;
    functie employees.job_id%TYPE;
BEGIN
    idDepartament := &idDepartament;
    functie := '&numeFunctie';

    SELECT SUM(salary + salary * NVL(commission_pct, 0))
    INTO salariu
    FROM employees
    WHERE department_id = idDepartament
        AND Upper(job_id) = Upper(functie);

    SELECT department_name
    INTO numeDepartament
    FROM departments
    WHERE department_id = idDepartament;

    dbms_output.put_line(
        'Suma veniturilor in departamentul ' ||
        numeDepartament || ' este ' || to_char(salariu)
    );
EXCEPTION
    WHEN no_data_found THEN
        dbms_output.put_line(
            'In departamentul ' || idDepartament ||
            ' nu exista angajati cu functia ' || functie
        );
END;
/
```

Rezultatul execuției blocului fără cererea group by pentru idDepartament = 80 și functie = SA_REP este:

```
Suma veniturilor in departamentul Sales este 297490


PL/SQL procedure successfully completed.
```

Rezultatul execuției blocului fără cererea group by pentru idDepartament = 10 și functie = SA_REP este:

```
Suma veniturilor in departamentul Administration este


PL/SQL procedure successfully completed.
```

Rezultatul execuției blocului fără cererea group by pentru idDepartament = 1 și functie = SA_REP este:

```
In departamentul 1 nu exista angajati cu functia SA_REP


PL/SQL procedure successfully completed.
```

Se observă că de data aceasta excepția s-a declanșat numai pentru departamentul 1, cu toate că departamentul 10 nu are niciun angajat cu funcția SA_REP. În mod normal, SELECT ar trebui să declanșeze

o excepție, dar funcția SUM întoarce o valoare null, astfel încât nu se declanșează excepția predefinită no_data_found.

Anomalia apare datorită modului de funcționare a cererii SELECT, din cauza aceasta, trebuie să se construiacă cu mare atenție o cerere DML.

Bibliografie

http://docs.oracle.com/database/121/LNPLS/errors.htm#LNPLS007
[http://docs.oracle.com/database/121/LNPLS/errors.htm#LNPLS007]

bdd/laboratoare/03.txt · Last modified: 2024/10/07 14:33 by ciprian.truica

Laboratorul 04 - PL/SQL Cursoare

Exercitii date spre rezolvare: Enunt/Rezolvare pentru Luni EG305 10-14

Conținut

- Cursoare implicite
- Cursoare explicite
- Cursoare cu parametri
- Tipuri de variabile REF CURSOR, SYS_REFCURSOR și RECORD
- Eficiența cursoarelor

Cursoare

Un cursor este o construcție PL/SQL prin care se pot controla și accesa informațiile dintr-o zonă de memorie alocată pentru a stoca datele procesate prin instrucțiuni SQL.

Există două tipuri de cursoare:

- Implicite – create implicit de catre sistemul de gestiune pentru toate comenzile DML, chiar și pentru interogările care returnează o singură înregistrare;
- Explicite – create de dezvoltatori pentru controlul comenzilor DML care returnează mai multe înregistrări.

Cursoare Implicite

În PL/SQL se creează implicit un cursor pentru fiecare instrucțiune DML (SELECT, INSERT, DELETE, UPDATE).

Pentru un utilizator acest lucru este total transparent și nu poate să controleze modul de execuție al unui cursor implicit, acesta poate numai să prelucreze datele returnate de cursorul implicit. Trebuie să se acorde mare atenție la modul în care se creează o cerere DML pentru că pot să apară probleme la execuția unui bloc. Trebuie să se acorde atenție și la modul în care sunt puse condițiile în clauza WHERE.

De exemplu, dacă nu se folosește o variabilă de tip colecție(vector, indexby table, nested table) cu un BULK COLLECT INTO într-un SELECT care returnează mai multe înregistrări și se va folosi o variabilă simplă atunci blocul o să dea o eroare deoarece se încearcă să se insereze mai multe valori intr-o singură variabilă.

Ex. 1 Exemplu greșit de folosire a unui cursor implicit (returnează mai mute valori care nu pot să fie inserate în variabila salariu):

```
1  SET serveroutput ON;
2
3  DECLARE
4      salariu NUMBER;
5  BEGIN
6      SELECT salary
7      INTO salariu
8      FROM employees
9      WHERE department_id = 80;
10 END;
```

Ex. 2 Exemplu corect de folosire a unui cursor implicit (clauza where conține o condiție care garantează că se va întoarce o singură valoare):

```
SET serveroutput ON;

DECLARE
    salariu NUMBER;
BEGIN
    SELECT salary
    INTO salariu
    FROM employees
    WHERE employee_id = 100;
END;
```

Ex. 3 Exemplu corect de folosire a unui cursor implicit care returnează mai multe înregistrări (se folosește un nested table în care se inserează salariile angajaților din departamentul 80):

```
SET serveroutput ON;

DECLARE
    TYPE nested_table IS TABLE OF employees.salary%TYPE;
    salariu nested_table;
BEGIN
    SELECT salary
    BULK COLLECT INTO salariu
    FROM employees
    WHERE department_id = 80;
END;
```

Cursoare Explicite

Cursoarele explicite sunt declarate de catre utilizator și sunt folosite pentru prelucrarea rezultatelor interogărilor care returnează linii multiple.

Limitări:

- Există un parametru de sistem OPEN_CURSOR care setează numărul maxim de cursoare care se pot deschide pe parcursul unei sesiuni. Pentru a vedea sau modifica acest parametru trebuie să ai privilegiu de DBA și se folosesc comenzile: “select name, value from v$parameter where name like 'open_cursors'”” sau “ALTER SYSTEM SET open_cursors = 400 SCOPE=BOTH”.
- O altă limitare a numărului de cursoare este dată de memoria disponibilă pentru gestionarea lor.

În secțiunea DECLARE se definește numele cursorului și structura interogării care va fi efectuată cu el. În această secțiune, interogarea este compilată dar nu este executată, semnalându-se eventualele erori de compilare.

Sintaxa este:

```
DECLARE
        CURSOR cursor_name [(<parameter> <parameter_type>)] IS
                SELECT column_manes
                FROM table_names
                WHERE conditions [FOR UPDATE [OF col1_name[, OF col2_name[, …]]]];

        variable cursor_name%rowtype;
```

Unde:

- cursore_name – numele atribuit cursorului
- column_names – numele coloanelor returnate de cursor
- table_names – numele tabelelor folosite de interogare
- conditions – condițiile de filtrare sau de join
- parameter – numele parametrilor cursorului și este opțional
- parameter_type – tipul parametrilor
- variable – este o variabilă de tipul unei linii din cursor
- for update – este opțiunea de blocare a liniilor (lock) selectate de cursor în baza de date

Observații:

- Interogarea SELECT poate să fie de oricetip, poate să conțină joinuri și poate să conțină subcereri
- Există o convenție în Oracle care recomandă ca numele cursoarelor să înceapă cu litera c; folosind un astfel de nume va fi clar că numele din secțiunea de declarare se referă la un cursor
- O înregistrare care este bazată pe cursor (cursor-based) este o înregistrare a cărei structuri este de tipul unei linii a cursorului
- Pentru a declara o variabilă bazată pe cursor se folosește atributul %ROWTYPE
- Referirea unui element al variabilei de tip cursor se face adăugând ca prefix numele variabilei (asemănător cu referirea unei coloane din tabel): value := variable_name.element_name
- Numele atribuit unui cursor nu este o variabilă PL/SQL, ci un identificator, deci nu se pot atribui valori numelui unui cursor și nici folosi în expresii
- Rezultatele interogării cursorului devine setul activ de date al cursorului

În secțiunea OPEN se deschide cursorul și se face execuția efectivă a interogării, obținându-se ca rezultat liniile care vor constitui setul activ de date.

Sintaxa este:

```
OPEN cursor_name [(<parameter> <parameter_type>)];
```

Unde:

- cursor_name - numele atribuit cursorului
- parameter – numele parametrilor formali ai cursorului
- parameter_type – tipul parametrilor

Cursorul se deschide cu valorile actuale pentru fiecare parametru formal specificat în declarația cursorului, în caz contrat parametrii care apar în declarația cursorului vor avea valori prestabilite. După ce s-a deschis cursorul se pot atribui parametrii actuali la parametrii formali folosind operatorul de atribuire.

Atunci când se execută comanda OPEN, cursorul identifică numai acele linii care satisfac condiția de interogare, dar liniile nu sunt citite până când nu se face prelucrarea de către cursor.

Cursorul se inițializează la prima linie a setului activ.

Fiecare cursor explicit are patru atribute care sunt setate automat de catre sistemul de gestiune și conțin informații despre starea lor:

- %ISOPEN = Returnează TRUE dacă cursorul este deschis și FALSE dacă cursorul este închis
- %FOUND = Returnează: -INVALID_CURSOR dacă cursorul este declarat dar nu este deschis sau dacă cursorul a fost închis; -NULL dacă cursorul este deschis dar nu a fost executat; -TRUE dacă atribuirea s-a efectuat cu succes; -FALSE dacă nu a fost returnat niciun rând.
- %NOTFOUND = Returnează: -INVALID_CURSOR dacă cursorul este declarat dar nu este deschis sau dacă cursorul a fost închis; -NULL dacă cursorul este deschis dar nu a fost executat; -FALSE dacă

atribuirea s-a efectuat cu succes; -TRUE dacă nu a fost returnat niciun rând.
- %ROWCOUNT = Returnează: -INVALID_CURSOR dacă cursorul este declarat dar nu este deschis sau dacă cursorul a fost închis; - Numărul de linii selectate și atribuite.

În secțiunea FETCH se memorează valorile din linia curentă a cursorului într-o variabilă. Sintaxa este:

```
LOOP
        FETCH cursor_name
        INTO variable_names;
        EXIT WHEN condition;
        ...
END LOOP;
```

Unde:

- cursore_name – numele atribuit cursorului
- variable_names – numele variabilelor în care se memorează valorile din linia curentă a cursorului
- condition – condiția de ieșire din ciclul LOOP

Instrucțiunea FETCH preia liniile din setul activ una câte una și sortează setul activ, dacă este cazul.

Cursorul avansează automat pe următoarea linie la executarea comenzii FETCH

În secțiunea CLOSE se închide setul de linii returnate de catre cursor după deschidere. Sintaxa este:

```
CLOSE cursor_name;
```

Unde:

- cursor_name – numele cursorului

După închiderea unui cursor rezultatul interogării se pierde. Pentru a accesa mulțimea datelor asociate unui cursor închis trebuie redeschis cursorul.

Structura completă a unui cursor explicit este următoarea:

```
DECLARE
        CURSOR cursor_name IS SELECT ...;
        variable cursor_name%rowtype;
BEGIN
        OPEN cursor_name;
        LOOP
                FETCH cursor_name
                INTO variable;
                EXIT WHEN condition
                ...
        END LOOP;
        CLOSE cursor_name;
END
```

În general, condition este cursor_name%NOTFOUND.

Ex. 4 Să se declare un cursor care selectează denumirea departamentului, numele angajatului, salariul și data angajării pentru acei angajati care au venit în companie în 2003.

```sql
SET serveroutput ON;

DECLARE
    CURSOR c_angajati IS
        SELECT d.department_name, e.employee_id,
            e.first_name || ' ' || e.last_name AS name, e.salary, e.hire_date
        FROM employees e INNER JOIN departments d ON e.department_id = d.department_id
        WHERE hire_date LIKE '%03'
        ORDER BY hire_date;
    v_angajat c_angajati%ROWTYPE;
BEGIN
    OPEN c_angajati;
    LOOP
        FETCH c_angajati INTO v_angajat;
        EXIT WHEN c_angajati%NOTFOUND;

        dbms_output.put_line(v_angajat.department_name || ' ' ||
                             v_angajat.employee_id || ' ' ||
                             v_angajat.name || ' ' ||
                             v_angajat.salary || ' ' ||
                             v_angajat.hire_date);
    END LOOP;
    CLOSE c_angajati;
END;
```

Ex. 5 Să se folosească un cursor pentru a face o listă cu veniturile managerilor din companie.

```sql
SET serveroutput ON;

DECLARE
    CURSOR c_angajati IS
        SELECT d.department_name, e.employee_id,
            e.first_name || ' ' || e.last_name AS name, e.salary, e.commission_pct
        FROM employees e INNER JOIN departments d ON e.department_id = d.department_id
        WHERE
            upper(e.job_id) LIKE '%MGR' OR
            upper(e.job_id) LIKE '%MAN' OR
            upper(e.job_id) LIKE '%PRES';
    angajat c_angajati%ROWTYPE;
    venit NUMBER := 0;
BEGIN
    OPEN c_angajati;

    dbms_output.put_line(
        rpad('ECUSON', 10) ||
        rpad('NUME', 30) ||
        rpad('DEPARTAMENT', 20) ||
        lpad('VENIT', 10)
    );
    dbms_output.put_line(
        rpad('=', 10, '=') ||
        rpad('=', 30, '=') ||
        rpad('=', 20, '=') ||
        lpad('=', 10, '=')
    );

    LOOP
        FETCH c_angajati INTO angajat;
        EXIT WHEN c_angajati%NOTFOUND;

        venit := round(angajat.salary + nvl(angajat.commission_pct,0) * angajat.salary);

        dbms_output.put_line(
            rpad(angajat.employee_id, 10) ||
            rpad(angajat.name, 30) ||
            rpad(angajat.department_name, 20) ||
            lpad(venit, 10)
        );
        venit := 0;
    END LOOP;
    CLOSE c_angajati;
END;
```

Setul activ de date definite de cursorul c_angajati este procesat linie cu linie. Ciclul iterativ de citire și procesare a liniilor individuale continuă până la procesarea tuturor liniilor din setul activ.

Ieșirea din buclă se face testând condiția %NOTFOUND care se setează pe true după ce cursorul procesează toate liniile. După închiderea cursorului datele din setul activ sunt resetate.

Ciclul FOR este folosit pentru simplificarea sintaxei de prelucrare a liniilor dintr-un cursor. Cursorul însuși este cel care determină momentul în care se părăsește ciclul. Sintaxa este:

```
DECLARE
        CURSOR cursor_name IS SELECT ...;
        variable cursor_name%rowtype;
BEGIN
        FOR variable IN cursor_name
        LOOP
                ...
        END LOOP;
END;
```

Ciclul FOR simplifică codul programului și efectuează următoarele operații:

- La inițializarea ciclului se execută în mod implicit o instrucțiune OPEN
- La fiecare parcurgere se execută în mod implicit o instrucțiune FETCH
- La părăsirea ciclului se execută în mod implicit o instrucțiune CLOSE

Ex. 6 Rescriere exemplu 5 folosind FOR.

```sql
SET serveroutput ON;

DECLARE
    CURSOR c_angajati IS
        SELECT d.department_name, e.employee_id,
            e.first_name || ' ' || e.last_name AS name, e.salary, e.commission_pct
        FROM employees e INNER JOIN departments d ON e.department_id = d.department_id
        WHERE
            upper(e.job_id) LIKE '%MGR' OR
            upper(e.job_id) LIKE '%MAN' OR
            upper(e.job_id) LIKE '%PRES';
    angajat c_angajati%ROWTYPE;
    venit NUMBER := 0;
BEGIN
    dbms_output.put_line(
        rpad('ECUSON', 10) ||
        rpad('NUME', 30) ||
        rpad('DEPARTAMENT', 20) ||
        lpad('VENIT', 10)
    );
    dbms_output.put_line(
        rpad('=', 10, '=') ||
        rpad('=', 30, '=') ||
        rpad('=', 20, '=') ||
        lpad('=', 10, '=')
    );

    FOR angajat IN c_angajati
    LOOP
        venit := round(angajat.salary + nvl(angajat.commission_pct,0) * angajat.salary);

        dbms_output.put_line(
            rpad(angajat.employee_id, 10) ||
            rpad(angajat.name, 30) ||
            rpad(angajat.department_name, 20) ||
            lpad(venit, 10)
        );
        venit := 0;
    END LOOP;
END;
```

Într-un ciclu FOR se poate folosi un SELECT care este tot un cursor, dar care nu trebuie să mai fie declarat în secțiunea DECLARE. Sintaxa este:

```
BEGIN
        FOR variable IN (SELECT column_manes FROM table_names WHERE conditions)
        LOOP
                ...
        END LOOP;
END;
```

Ex. 7 Să se rescrie exercițiul 6 folosind un SELECT în ciclul FOR.

```
SET serveroutput ON;

DECLARE
    venit NUMBER := 0;
BEGIN
    dbms_output.put_line(
        rpad('ECUSON', 10) ||
        rpad('NUME', 30) ||
        rpad('DEPARTAMENT', 20) ||
        lpad('VENIT', 10)
    );
    dbms_output.put_line(
        rpad('=', 10, '=') ||
        rpad('=', 30, '=') ||
        rpad('=', 20, '=') ||
        lpad('=', 10, '=')
    );

    FOR angajat IN (
        SELECT d.department_name, e.employee_id,
            e.first_name || ' ' || e.last_name AS name, e.salary, e.commission_pct
        FROM employees e INNER JOIN departments d ON e.department_id = d.department_id
        WHERE
            upper(e.job_id) LIKE '%MGR' OR
            upper(e.job_id) LIKE '%MAN' OR
            upper(e.job_id) LIKE '%PRES'
    )
    LOOP
        venit := round(angajat.salary + nvl(angajat.commission_pct,0) * angajat.salary);

        dbms_output.put_line(
            rpad(angajat.employee_id, 10) ||
            rpad(angajat.name, 30) ||
            rpad(angajat.department_name, 20) ||
            lpad(venit, 10)
        );
        venit := 0;
    END LOOP;
END;
```

Când ne referim la rândul curent dintr-un cursor explicit comenzile SQL pot folosi clauza WHERE CURRENT OF cursor_name. Această clauză permite actualizarea sau ștergerea în punctul în care se află cursorul, fără a fi necesară folosirea condițiilor în clauza WHERE pentru identificarea unică a liniei. Trebuie să se folosească clauza FOR UPDATE în definirea cursorului pentru a se bloca rândurile la deschidere (se face un lock).

Clauza WHERE CURRENT OF cursor_name se poate folosi si in comenzile UPDATE și DELETE.

Ex. 8 Să se modifice comisionul cu 10% din salariu pentru angajații care au peste 18 ani vechime în companie.

```sql
SET serveroutput ON;

DECLARE
    CURSOR c_angajati IS
        SELECT d.department_name, e.employee_id, e.commission_pct, e.hire_date,
            e.first_name || ' ' || e.last_name AS name, e.salary
        FROM employees e INNER JOIN departments d ON e.department_id = d.department_id
        FOR UPDATE OF commission_pct;
    angajat c_angajati%ROWTYPE;
    comision_nou NUMBER DEFAULT 0;
BEGIN
    OPEN c_angajati;
    LOOP
        FETCH c_angajati INTO angajat;
        EXIT WHEN c_angajati%NOTFOUND;

        IF add_months(angajat.hire_date, 216) < SYSDATE THEN
            comision_nou := nvl(angajat.commission_pct,0) + 0.1;
            UPDATE employees SET commission_pct = comision_nou
            WHERE CURRENT OF c_angajati;
        END IF;
    END LOOP;
    CLOSE c_angajati;
END;
```

Ex. 9 Să se șteargă din tabela employees toți angajații care nu au comision.

```sql
SET serveroutput ON;

BEGIN
    FOR angajat IN (SELECT employee_id FROM employees WHERE commission_pct IS NULL)
    LOOP
        DELETE FROM employees
        WHERE employee_id = angajat.employee_id;
    END LOOP;
END;
```

Parametrii permit transmiterea unor valori efective unui cursor când acesta este deschis. Parametrii formali se definesc în momentul declarării cursorului. Tipurile parametrilor sunt aceleași cu ale variabilelor scalare, dar nu primesc dimensiuni. Parametrii sunt folosiți pentru referirea în cadrul expresiei de definire din cadrul cursorului și pot fi tratați și ca variabile PL/SQL.

Ex. 10 Să se facă o listă cu angajații care fac parte dintr-un departament specificat, au o anumită funcție și au venit în companie la o anumită dată specificată. Aceste condiții să fie transmise ca parametri unui cursor.

```sql
SET serveroutput ON;

DECLARE
    CURSOR c_angajati(departmentId NUMBER, jobId CHAR, hireDate DATE) IS
        SELECT department_id, first_name || ' ' || last_name AS name, job_id, hire_date
        FROM employees
        WHERE
            department_id = departmentId AND
            lower(job_id) = lower(jobId) AND
            hire_date > hireDate;
    angajat c_angajati%ROWTYPE;
BEGIN
    dbms_output.put_line('Cursor 1');
    OPEN c_angajati(20, 'MK_REP', '1-JUN-02');
    LOOP
        FETCH c_angajati INTO angajat;
        EXIT WHEN c_angajati%NOTFOUND;

        dbms_output.put_line(
            rpad(angajat.department_id, 10) ||
            rpad(angajat.name, 30) ||
            rpad(angajat.job_id, 15) ||
            lpad(angajat.hire_date, 20)
        );
    END LOOP;
    CLOSE c_angajati;

    dbms_output.put_line('Cursor 2');
    OPEN c_angajati(departmentId => 30, jobId => 'PU_CLERK', hireDate => '1-JUN-02');
    LOOP
        FETCH c_angajati INTO angajat;
        EXIT WHEN c_angajati%NOTFOUND;

        dbms_output.put_line(
            rpad(angajat.department_id, 10) ||
            rpad(angajat.name, 30) ||
            rpad(angajat.job_id, 15) ||
            lpad(angajat.hire_date, 20)
        );
    END LOOP;
    CLOSE c_angajati;

    dbms_output.put_line('Cursor 3');
    OPEN c_angajati(jobId => 'MK_REP', hireDate => '1-JUN-02', departmentId => 20);
    LOOP
        FETCH c_angajati INTO angajat;
        EXIT WHEN c_angajati%NOTFOUND;

        dbms_output.put_line(
            rpad(angajat.department_id, 10) ||
            rpad(angajat.name, 30) ||
            rpad(angajat.job_id, 15) ||
            lpad(angajat.hire_date, 20)
        );
    END LOOP;
    CLOSE c_angajati;

    dbms_output.put_line('Cursor 4');
    OPEN c_angajati(30, jobId => 'PU_CLERK', hireDate => '1-JUN-02');
    LOOP
        FETCH c_angajati INTO angajat;
        EXIT WHEN c_angajati%NOTFOUND;

        dbms_output.put_line(
            rpad(angajat.department_id, 10) ||
            rpad(angajat.name, 30) ||
            rpad(angajat.job_id, 15) ||
            lpad(angajat.hire_date, 20)
        );
    END LOOP;
    CLOSE c_angajati;

    dbms_output.put_line('Cursor 5');
    OPEN c_angajati(30, 'PU_CLERK', hireDate => '1-JUN-02');
    LOOP
        FETCH c_angajati INTO angajat;
        EXIT WHEN c_angajati%NOTFOUND;

        dbms_output.put_line(
            rpad(angajat.department_id, 10) ||
            rpad(angajat.name, 30) ||
            rpad(angajat.job_id, 15) ||
```

```
            lpad(angajat.hire_date, 20)
        );
    END LOOP;
    CLOSE c_angajati;
END;
```

Tipuri de variabile REF CURSOR, SYS_REFCURSOR și RECORD

Se poate asocia o interogare unui cursor astfel:

- Se declară un tip de date REF CURSOR și se declară o variabilă cu acest tip
- Se declară o variabilă cu tipul de date SYS_REFCURSOR

Acest mod de a declara un cursor este util în momentul în care se dorește să se transmită seturi de date la subprograme(proceduri, funcții, pachete). Utilizatorul poate să-și definească propriul tip de date pentru a păstra o linie dintr-un cursor folosind o variabilă de tip RECORD.

Sintaxa REF CURSOR:

```
DECLARE
        TYPE ref_cursor IS REF CURSOR;
        c_variable ref_cursor;
        variable variable_type;
BEGIN
        OPEN c_variable
                FOR SELECT column_manes
                FROM table_names
                WHERE conditions  [FOR UPDATE [OF col1_name[, col2_name, …]]];
        LOOP
                FETCH c_variable INTO variable;
                EXIT WHEN condition
                ...
        END LOOP;
        CLOSE c_variable;
END;
```

Unde:

- ref_cursor – numele variabilei de tip REF CURSOR
- c_variable – numele variabilei curso
- variable – numele unei variabile normale
- conditions – condițiile de ieșire din buclă puse pentru c_variable

Sintaxa SYS_REFCURSOR:

```
DECLARE
        c_variable SYS_REFCURSOR;
        variable variable_type;
BEGIN
        OPEN c_variable
                FOR SELECT column_manes
                FROM table_names
                WHERE conditions  [FOR UPDATE [OF col1_name[, col2_name, …]]];
        LOOP
                FETCH c_variable INTO variable;
                EXIT WHEN condition
                ...
        END LOOP;
        CLOSE c_variable;
END;
```

Ex. 11 Să se facă o listă cu toate departamentele.

```
SET serveroutput ON;

DECLARE
    TYPE r_cursor IS REF CURSOR;
    c_departamente r_cursor;
    departament departments.department_name%TYPE;
BEGIN
    OPEN c_departamente FOR SELECT department_name FROM departments;
    LOOP
        FETCH c_departamente INTO departament;
        EXIT WHEN c_departamente%NOTFOUND;
        dbms_output.put_line(departament);
    END LOOP;
    CLOSE c_departamente;
END;
```

Ex. 12 Să se listeze toți angajatii din tabela employees.

```
SET serveroutput ON;

DECLARE
    TYPE r_cursor IS REF CURSOR;
    c_angajati r_cursor;
    angajat employees%ROWTYPE;
BEGIN
    OPEN c_angajati FOR SELECT * FROM employees;
    LOOP
        FETCH c_angajati INTO angajat;
        EXIT WHEN c_angajati%NOTFOUND;
        dbms_output.put_line(
            angajat.first_name || ' ' ||
            angajat.last_name || ' ' ||
            angajat.job_id || ' ' ||
            angajat.salary
        );
    END LOOP;
    CLOSE c_angajati;
END;
```

Ex. 13 Să se folosească un record pentru a păstra numele, funcția și salariul pentru toți angajații.

```plsql
SET serveroutput ON;

DECLARE
    TYPE r_cursor IS REF CURSOR;
    TYPE employee_record IS RECORD(
        name employees.first_name%TYPE,
        job employees.job_id%TYPE,
        salary employees.salary%TYPE
    );
    c_angajati r_cursor;
    angajat employee_record;
BEGIN
    OPEN c_angajati FOR
        SELECT first_name || ' ' || last_name, job_id, salary FROM employees;
    LOOP
        FETCH c_angajati INTO angajat;
        EXIT WHEN c_angajati%NOTFOUND;
        dbms_output.put_line(
            angajat.name || ' ' ||
            angajat.job || ' ' ||
            angajat.salary
        );
    END LOOP;
    CLOSE c_angajati;
END;
```

Eficiența cursoarelor

- Pentru a mări eficiența cursoarelor trebuie să se țină cont de următoarele facilități:
- Crearea de indecși pe coloanele folosite în clauzele WHERE și JOIN
- Prelucrarea procedurală permite un control puternic și flexibil al liniilor din baza de date, dar fiecare INSERT sau UPDATE provoacă o reorganizare a indecșilor
- Efectuarea, pe cât posibil, a calculelor în cererea SELECT
- Prefixarea coloanelor cu numele userului și a tabelei
- În cazul unui JOIN indexat, să se facă referire la tabela cea mai mică la sfârșit
- Să se folosească IF pentru comparație, dacă toate datele de prelucrat sunt stocate în variabile
- Să se evite SELECT … FROM SYS.DUAL deoarece produce deschiderea unui cursor și transmiterea unor cereri către RDBMS
- Cursoarele explicite evită un FETCH ulterior față de cele implicite care execută două cereri(testează și linia următoare)
- Dacă se prelucrează mai multe linii dintr-o tabelă, este bine să se facă într-un singur pas, evitând trecerile repetate prin tabelă

Performanțele cursoarelor pot fi îmbunătățite prin ajustarea parametrului de sistem CURSOR_SPACE_FOR_TIME care precizează momentul în care zona partajată (shared pool) poate fi dealocată pentru a face loc unei noi comenzi SQL și are valoarea implicită FALSE. Aceasta înseamnă că SGBD-ul poate dealoca zona partajată chiar dacă este deschis un cursor al unei aplicații. Valoarea TRUE înseamnă că o zonă partajată poate fi dealocată numai atunci când toate cursoarele aplicației sunt închise, cu condiția ca zona să fie suficient de mare pentru ca toate cursoarele să fie deschise simultan. Setarea parametrului pe TRUE face ca SGBD-ul să realizeze o economie de timp și resurse și poate îmbunătăți performanțele apelurilor de execuție.

Nu se recomandă totdeauna setarea pe TRUE deoarece parametrul trebuie corelat cu alți parametri de sistem pentru dimesionarea zonei partajate și a numărului maxim de cursoare deschise simultan.

Ex. 14 Să se scrie un cursor explicit PL/SQL care face o listă pentru acordarea de comisioane tuturor șefilor de departament(care se identifică în coloana mgr din tabela EMP), după următorul algoritm:

- Dacă salariu_sef < salariu_mediu_departament atunci comision_sef = 10% * salariu_mediu_departament
- Dacă salariu_sef >= salariu_mediu_departament atunci comision_sef = 20% * salariu_mediu_departament

```
SET serveroutput ON;

DECLARE
    CURSOR c_managers IS
        SELECT DISTINCT manager_id FROM employees WHERE manager_id IS NOT NULL;
    emp_name employees.first_name%TYPE;
    dep_id employees.department_id%TYPE;
    dep_name departments.department_name%TYPE;
    salary employees.salary%TYPE;
    emp_id employees.employee_id%TYPE;
    avg_salary NUMBER;
    commission NUMBER;
BEGIN
    dbms_output.put_line(
        rpad('Departament', 20) ||
        rpad('Salariu mediu', 15) ||
        rpad('Nume', 30) ||
        lpad('Salariu', 10) ||
        lpad('Comision', 10)
    );
    dbms_output.put_line(
        rpad('-', 20, '-') ||
        rpad('-', 15, '-') ||
        rpad('-', 30, '-') ||
        lpad('-', 10, '-') ||
        lpad('-', 10, '-')
    );

    FOR rec IN c_managers
    LOOP
        SELECT first_name || ' ' || last_name, department_id, salary, employee_id
        INTO emp_name, dep_id, salary, emp_id
        FROM employees
        WHERE employee_id = rec.manager_id;

        SELECT round(avg(salary))
        INTO avg_salary
        FROM employees
        WHERE department_id = dep_id;

        SELECT department_name
        INTO dep_name
        FROM departments
        WHERE department_id = dep_id;

        IF salary < avg_salary THEN
            commission := round(0.1 * avg_salary);
        ELSE
            commission := round(0.2 * avg_salary);
        END IF;

        -- comision ca procent din salariu
        commission := commission / salary;

        dbms_output.put_line(
            rpad(dep_name, 20) ||
            rpad(avg_salary, 15) ||
            rpad(emp_name, 30) ||
            lpad(salary, 10) ||
            lpad(to_char(commission, '99.99'), 10)
        );
    END LOOP;
END;
```

Laboratorul 05 - PL/SQL Subprograme

Conținut

- Subprograme
- Proceduri PL/SQL
- Funcții PL/SQL
- Considerente asupra procedurilor și funcțiilor
- Informații din dicționarul bazei de date

Subprograme

Un subprogram este un bloc PL/SQL cu nume (spre deosebire de blocurile anonime) care poate primi parametri și poate fi invocat dintr-un anumit mediu (e.g., SQL*Plus, Oracle Forms, Oracle Reports etc.).

Subprogramele sunt bazate pe structura de bloc PL/SQL. Similar unui bloc PL/SQL ele conțin o parte declarativă facultativă, o parte executabilă obligatorie și o parte de tratare excepțiilor facultativă.

Exista 2 tipuri de subprograme:

- Proceduri
- Funcții (trebuie să conțină cel puțin o comandă RETURN);

Subprogramele pot fi:

- Locale (în cadrul altui bloc PL/SQL sau subprogram)
- Stocate (create cu comanda CREATE)

Odată create, procedurile și funcțiile sunt stocate în baza de date de aceea ele se numesc subprograme stocate.

Parametrii unui subprogram

Parametrii sunt variabile disponibile atât pentru programul principal (apelant) cât și pentru subprogram. Parametrii determină funcționalitatea și rezultatele procesării datelor. Parametrii unui subprgram sunt opționali și sunt declarați în momentul creării subprogramului. Un nume de parametru trebuie să înceapă cu o literă, să nu conțină spații, și să aibă lungimea maximă de 30 de caractere. Parametrii actuali reprezintă valorile efective transmise subprogramului în momentul apelului sau rezultatele întoarse de subprogram (în cazul definirii unor parametri **OUT** sau **IN OUT**), în timp ce parametrii formali sunt cei care recepționează valorile parametrilor actuali, fiind referiți conform logicii subprogramului.

Sintaxa:

```
parameter_name [IN | OUT [NOCOPY] | IN OUT [NOCOPY]] parameter_type  [ {:=  | DEFAULT } {expresion | VALUE}]
```

Unde:

- **parameter_name** – numele parametrului
- **parameter_name** – tipul parametrului
- **NOCOPY** – specifică că parametrul se transmite prin referință (adresă) nu prin valoare și este valabil doar pentru OUT și IN OUT care se transmit implicit prin valoare
- **:= | DEFAULT** – pentru a da o valoare implicită
- **expresion | value** – se pot atribui expresii sau valori parametrilor

Parametrii unui subprogram pot fi transmiși în două moduri, prin referință sau prin valoare. În primul caz un pointer către parametrul actual este transmis parametrului formal corespunzător și nu se copiază efectiv, ceea ce duce la creșterea performanțelor mai ales la parametrii de tip colecție. La transmiterea prin valoare are loc copierea valorii parametrului actual la cel formal. Până la versiunea 8i doar parametrii de intrare (**IN**) erau transmiși prin referință (se întâmplă în continuare acest lucru), iar cei de ieșire (**OUT**) prin valoare. Folosindu-se NOCOPY se pot transmite parametrii de ieșire (**OUT**) și cei de intrare-ieșire (**IN OUT**) prin referință. Când un parametru este transmis prin referință, orice modificare asupra parametrului actual va modifica și parametrul formal, deoarece ambii au pointeri către aceeași valoare.

Există trei tipuri de parametri:

- IN – parametrul poate fi referit în interiorul subprogramului, dar nu poate fi modificat;
- OUT – parametrul nu poate fi referit în interiorul subprogramului, dar poate fi modificat și poate fi referit în afara subprogramului (în programul apelant);
- IN OUT – parameterul poate fi referit în interiorul subprogramului, poate fi modificat și poate fi referit în afara subprogramului (în programul apelant).

Parametrul **IN** este tipul implicit. Parametrul **OUT** este inițializat cu **NULL**, iar subprogramul atribuie parametrului o valoare care poate fi referită în afara ei. Un parametru **IN OUT** poate avea (sau nu) o valoare inițială, valoarea inițială poate fi modificată în subprogram și orice modificare poate fi returnată în programul apelant.

Parametrii sunt declarați ca tipuri de date dar fără lungime de tip sau de precizie, adică un parametru poate fi declarat ca VARCHAR2 dar fără o componentă de lungime(de ex. VARCHAR2(30) nu este permis). Parametrii pot avea o valoare implicită similară variabilelor, deci se poate folosi operatorul de atribuire (**:=**) sau opțiunea DEFAULT. Dacă un parametru are o valoare implicită nu este nevoie să se includă în apel. Se poate folosi operatorul de asociere (**=>**) pentru a trimite valori efective unui parametru, în acest caz ordinea nu contează. Se indică folosirea operatorului de asociere în cazul subprogramelor cu mulți parametri mai ales pentru că o astfel de scriere este mult mai clară și nu există posibilitatea strecurării unei erori în momentul când se apelează subprogramul.

Proceduri PL/SQL

O procedură este un subprogram care execută un set de instrucțiuni și nu returnează direct o valoare către programul apelant. Rezultatele obținute prin procesarea datelor pot fi folosite în programul apelant, în funcție de modul de declarare a parametrilor.

Proceduri declarate în cadrul unui bloc

O procedură stocată poate fi declarată în cadrul unui bloc PL/SQL. Durata de viață al unei astfel de proceduri este numai pe perioada existenței blocului. După terminarea execuției blocului procedura se pierde, deoarece este stocată doar în memorie, nu este stocată permanent în SGBD.

Sintaxa

```
DECLARE
  [block_variables;]
  PROCEDURE procedure_name [(parameter_name [IN|OUT|IN OUT] parameter_type,...)]
  {IS|AS}
    [procedure_declaration_section]
  BEGIN
    procedure_executable_section;
  [EXCEPTION
    procedure_exception_section]
  END [procedure_name];
  ...
BEGIN
  lock_executable_section;
  procedure_name [(parameters)];
  block_executable_section;
[EXCEPTION
```

```
  block_exception_section]
END;
```

Unde:

- **procedure_name** – numele procedurii
- **parameter_name** – numele unui parametru formal din lista de parametri
- **parameter_type** – reprezintă tipul parametrului formal, se dă fără precizie
  - GREȘIT: nume_param varchar2(30)
  - CORECT: nume_param varchar2
- **procedure_declaration_section** – secțiunea de declarare a variabilelor locale folosite în procedură, poate să lipsească
- **procedure_executable_section** – secțiunea executabilă a procedurii
- **procedure_exception_section** – secțiune de tratare a excepțiilor din cadrul procedurii, poate să lipsească
- **block_variables** – secțiunea de declarare a variabilelor locale folosite în bloc
- **block_executable_section** – secțiunea executabilă a blocului
- **block_exception_section** – secțiunea de tratare a excepțiilor din cadrul blocului, este opțională
- **IN | OUT | IN OUT** – specifică dacă parametrul poate fi referit sau modificat în interiorul sau exteriorul procedurii

Ex. 1. Exemplu de procedură declarată într-un bloc.

```
set verify off
set serveroutput on
DECLARE
  PROCEDURE procFaraParam
  AS
  BEGIN
    dbms_output.put_line('Am apelat o procedura fara parametri');
  END procFaraParam;
BEGIN
  -- nu este nevoie de un bloc imbricat fiindca nu am o sectiune de declaratii
  -- daca blocul necesita variabile,
  -- atunci TREBUIE sa se foloseasca un bloc imbricat
  procFaraParam();
  -- pentru proceduri care NU au parametri parantezele nu sunt obligarotii
  --procFaraParam;
END;
/
```

Ex. 2. Să se scrie o procedură declarată în cadrul unui bloc care întoarce salariu maxim pentru un ID de departament și o funcție introduse de la tastatură. Salariu maxim să fie returnat folosindu-se o variabilă scalară. Să se traducă joburile în limba română, în cadrul procedurii. Aveți grijă la cum sunt repartizate job-urile în departamente.

```plsql
set serveroutput ON;
DECLARE
    numedepartament departments.department_name%TYPE;
    iddept          employees.department_id%TYPE := &iddepartament;
    PROCEDURE Salariu (deptid       IN NUMBER,
                       functie      IN OUT VARCHAR2,
                       salariumaxim OUT NUMBER)
    IS --merge si AS in loc de IS
      salmax NUMBER; -- imediar dupa IS sau AS se fac declaratiile
    BEGIN
        SELECT Max(salary)
        INTO   salmax
        FROM   employees
        WHERE  department_id = deptid
               AND Lower(job_id) = Lower(functie)
        GROUP  BY department_id;

        salariumaxim := salmax;

        -- se putea insera direct in variabila salariuMaxim trimisa ca parametru
        --SELECT max(SALARY) INTO salariuMaxim FROM EMPLOYEES
        --  WHERE DEPARTMENT_ID = deptId AND lower(JOB_ID) = lower(functie)
        --  GROUP BY DEPTNO;
        functie := CASE
                     WHEN ( Upper(functie) = 'FI_ACCOUNT' ) THEN 'Contabil'
                     WHEN ( Upper(functie) = 'IT_PROG' ) THEN 'Programator'
                     WHEN ( Upper(functie) LIKE '%_CLERK' ) THEN 'Functionar'
                     WHEN ( Upper(functie) = 'AD_PRES' ) THEN 'Presedinte'
                     WHEN ( Upper(functie) LIKE '%_MAN' ) THEN 'Manager'
                     ELSE 'Nu avem functia'
                   END;
    EXCEPTION
      WHEN no_data_found THEN
                dbms_output.Put_line('Nu a fost gasit nici o inregistrare');
    END;
BEGIN
    DECLARE
        functie VARCHAR(40) := '&numeFunctie';
        salmax  NUMBER;
    BEGIN
        SELECT department_name
        INTO   numedepartament
        FROM   departments
        WHERE  department_id = iddept;

        Salariu(iddept, functie, salmax);

        -- folosind operatorul de asociere (discutat la cursoare, merge la fel)
        --salariu(deptId => idDept, salariuMaxim => salMax, functie => functie);
        --salariu(idDept, salariuMaxim => salMax, functie => functie);
        --salariu(idDept, functie, salariuMaxim => salMax);
        dbms_output.Put_line('In departamentul '
                             || numedepartament
                             ||' salariu maxim pentru functia '
                             || functie
                             ||' este '
                             ||salmax);
    EXCEPTION
        WHEN no_data_found THEN
          dbms_output.Put_line('Departament inexistent');
    END;
END;
/
```

Proceduri stocate

O procedură stocată este creată ca un obiect în dicționarul de date de la nivelul SGBD-ului și poate fi folosită oricând.

Sintaxa:

```
CREATE [OR REPLACE] PROCEDURE procedure_name [(parameter_name   [IN|OUT|IN OUT] parameter_type, ... )]
  [AUTHID {DEFINER|CURRENT_USER}]
  [PRAGMA AUTONOMOUS_TRANSACTION]
{IS|AS}
  [declaration_section]
BEGIN
  execution_section;
[EXCEPTION
  exception_section;]
END [procedure_name];
```

Unde:

- **procedure_name** – numele procedurii
- **parameter_name** – numele unui parametru formal din lista de parametri
- **parameter_type** – reprezintă tipul parametrului formal, se specifica fără precizie
- **declaration_section** – secțiunea de declarare a variabilelor locale folosite în procedură
- **executable_section** – secțiunea executabilă a procedurii
- **exception_section** – secțiune de tratare a excepțiilor din cadrul procedurii, poate să lipsească
- **IN | OUT | IN OUT** – specifică dacă parametrul poate fi referit sau modificat în interiorul sau exteriorul procedurii
- **[AUTHID {DEFINER | CURRENT_USER}]** – specifică dacă o procedură stocată se execută cu drepturile celui care a creat-o (valoarea implicită) sau ale utilizatorului curent
- **[PRAGMA AUTONOMOUS_TRANSACTION]** – specifică că execuția procedurii suspendă tranzacția curentă care se reia după terminarea execuției procedurii, adică într-o tranzacție imbricăm o altă tranzacție cu propriile sale comenzi TCL (COMMIT și ROLLBACK)

Ex. 3. Să se creeze o procedură stocată care calculează veniturile angajaților cu o vechime de peste 10 de ani în firmă, dintr-un anumit departament.

```plsql
CREATE OR REPLACE PROCEDURE Sp_venit(iddept  IN NUMBER,
                                     venit   IN OUT NUMBER,
                                     dataref IN DATE DEFAULT SYSDATE)
IS
  numedept departments.department_name%TYPE;
BEGIN
    SELECT department_name
    INTO   numedept
    FROM   departments
    WHERE  department_id = iddept;

    SELECT SUM(salary * ( 1 + Nvl(commission_pct, 0) ))
    INTO   venit
    FROM   employees
    WHERE  department_id = iddept
           AND Add_months(hire_date, 120) < dataref;

    dbms_output.Put_line(Rpad(numedept, 20)
                         ||Rpad(venit, 20));
EXCEPTION
  WHEN no_data_found THEN
             dbms_output.Put_line('Nu a fost gasita nicio inregistrare!');
END sp_venit;
/

set verify OFF
set serveroutput ON
DECLARE
    dataref DATE DEFAULT SYSDATE;
    venit   NUMBER := 0;
    total   NUMBER := 0;
BEGIN
    FOR rand IN (SELECT DISTINCT department_id
                 FROM   departments
                 ORDER  BY department_id) LOOP
        Sp_venit(rand.department_id, venit, dataref);

        -- merge si fara sa dam data fiindca are default in lista
        -- de parametri pe care ii primeste procedura stocata
        -- sp_venit(rand.DEPARTMENT_ID, venit);
        -- sp_venit(idDept => rand.DEPARTMENT_ID, venit => venit);
        -- sp_venit(venit => venit, idDept => rand.DEPARTMENT_ID);
        -- sp_venit(rand.DEPARTMENT_ID, venit => venit);
        total := total + Nvl(venit, 0);
    END LOOP;

    dbms_output.Put_line('Total '
                         ||total);
END;
/
```

Observații:

- Procedura se va crea prima și dacă nu are erori de compilare se va afișa mesajul: **Procedure created**
- În caz contrar se va afișa mesajul: **Warning: Procedure created with compilation errors**

- Pentru a vedea erorile de compilare se va folosi comanda **show errors**
- Dacă în dicționarul de date există o procedură cu un nume, atunci nu se mai poate crea alta cu același nume
- Pentru a suprascrie o procedură existentă se folosește comanda **CREATE OR REPLACE**, folosiți comanda aceasta cu atenție deoarece puteți să înlocuiți din greșeală o procedură deja existentă, scrisă de un alt dezvoltator, care are același nume dar are altă funcționalitate
- Pentru a șterge o procedură stocată se folosește comanda DDL drop: **DROP PROCEDURE procedure_name**

Cursoare în proceduri

Cursoarele declarate într-o procedură urmează aceleași reguli ca într-un bloc obișnuit. O procedură poate conține mai multe cursoare înlănțuite sau imbricate.

Ex. 4. Să se scrie o procedură nestocată, care utilizează un cursor, pentru calculul numărului de zile de concediu pentru toți angajații, după următorul algoritm:

1. manageri de departament:
    - Vechimea < 13 de ani primesc 20 zile de concediu
    - Vechimea >= 13 de ani primesc 22 zile de concediu
2. angajații care nu sunt șefi:
    - Vechimea < 13 de ani primesc 15 zile de concediu
    - Vechimea >= 13 de ani primesc 22 zile de concediu

```
set serveroutput ON;
DECLARE
    PROCEDURE Concediu
    AS
      CURSOR c_concediu IS
        SELECT *
        FROM   employees
        WHERE  department_id IS NOT NULL
        ORDER  BY department_id;
      randconcediu c_concediu%ROWTYPE;
      TYPE r_concediu IS RECORD (
        numedept departments.department_name%TYPE,
        numeang VARCHAR2(50),
        dataang employees.hire_date%TYPE,
        sef VARCHAR2(2),
        anivechime NUMBER,
        zileconcediu NUMBER );
      concediu     R_CONCEDIU;
      manager      employees.manager_id%TYPE;
    BEGIN
        OPEN c_concediu;

        LOOP
            FETCH c_concediu INTO randconcediu;

            EXIT WHEN c_concediu%NOTFOUND;

            SELECT department_name
            INTO   concediu.numedept
            FROM   departments
            WHERE  department_id = randconcediu.department_id;

            Concediu.dataang := randconcediu.hire_date;

            Concediu.numeang := randconcediu.first_name
                                || ' '
                                || randconcediu.last_name;

            Concediu.anivechime := Trunc(
            Months_between(SYSDATE, randconcediu.hire_date) / 12);

            BEGIN
                SELECT manager_id
                INTO   manager
                FROM   employees
                WHERE  manager_id = randconcediu.employee_id;

                concediu.sef := 'DA';
            EXCEPTION
                WHEN no_data_found THEN
                  concediu.sef := 'NU';
                WHEN too_many_rows THEN
                  concediu.sef := 'DA';
            END;
```

```
55
56              IF Concediu.anivechime < 13
57                 AND Concediu.sef = 'DA' THEN
58                Concediu.zileconcediu := 20;
59              ELSIF concediu.anivechime >= 13
60                    AND concediu.sef = 'DA' THEN
61                Concediu.zileconcediu := 22;
62              ELSIF concediu.anivechime < 13
63                    AND concediu.sef = 'NU' THEN
64                Concediu.zileconcediu := 15;
65              ELSE
66                Concediu.zileconcediu := 17;
67              END IF;
68
69              dbms_output.Put_line(Rpad(Concediu.numedept, 20)
70                                   ||Rpad(Concediu.numeang, 20)
71                                   ||Rpad(Concediu.dataang, 20)
72                                   ||Rpad(Concediu.sef, 20)
73                                   ||Rpad(Concediu.anivechime, 20)
74                                   ||Rpad(Concediu.zileconcediu, 20));
75          END LOOP;
76      END;
77  BEGIN
78      concediu;
79  END;
```

Ex. 5. Să se scrie un program PL/SQL cu o procedură care distribuie salariul șefului de departament la subalternii lui, în funcție de vechimea în companie, astfel:

- grupa 1 – subalternii cu o vechime ⇐ 31 de ani să primească un un fond de premiere egal cu 30% din salariul șefului
- grupa 2 – subalternii cu o vechiime > 31 de ani să primească un fond de premiere egal cu 70% din salariul șefului

Fondul de premiere se distribuie în mod egal la toți subalternii dintr-o grupă.

```
set serveroutput ON;
DECLARE
    PROCEDURE Distributie(salary  IN NUMBER,
                          vechime IN NUMBER,
                          nrang1  IN NUMBER,
                          nrang2  IN NUMBER,
                          prima   OUT NUMBER,
                          grupa   OUT NUMBER)
    IS
    BEGIN
        IF vechime <= 31 THEN
          prima := Round(0.3 * salary / nrang1);

          grupa := 1;
        ELSE
          prima := Round(0.7 * salary / nrang2);

          grupa := 2;
        END IF;
    END;
BEGIN
    DECLARE
        nrang1  NUMBER := 0;
        nrang2  NUMBER := 0;
        dataang DATE;
        salsef  employees.salary%TYPE;
        prima   NUMBER;
        grupa   NUMBER;
        numesef VARCHAR2(20);
        vechime NUMBER;
    BEGIN
        dbms_output.Put_line(Rpad('Nume sef', 50)
                             ||Rpad('Salariu sef', 20)
                             ||Rpad('Nume subaltern', 50)
                             ||Rpad('Vechime', 20)
                             ||Rpad('Prima', 20)
                             ||Rpad('Grupa', 20));

        FOR sef IN (SELECT DISTINCT manager_id
                    FROM   employees
                    WHERE  manager_id IS NOT NULL) LOOP
            nrang1 := 0;

            nrang2 := 0;

            SELECT first_name
                   || ' '
                   || last_name,
                   salary
            INTO   numesef, salsef
            FROM   employees
            WHERE  employee_id = sef.manager_id;

            FOR angajat IN (SELECT DISTINCT employee_id
                            FROM   employees
                            WHERE  manager_id = sef.manager_id) LOOP
                SELECT Trunc(Months_between(SYSDATE, hire_date) / 12)
                INTO   vechime
                FROM   employees
```

```
60                 WHERE  employee_id = angajat.employee_id;
61
62                 IF vechime <= 31 THEN
63                   nrang1 := nrang1 + 1;
64                 ELSE
65                   nrang2 := nrang2 + 1;
66                 END IF;
67             END LOOP;
68
69             FOR angajat IN (SELECT DISTINCT first_name
70                                          || ' '
71                                          || last_name
72                                          fullname,
73                                          Trunc(Months_between(SYSDATE,
74                                                hire_date)
75                                                /
76                                                12)
77                                          vechime
78                             FROM   employees
79                             WHERE  manager_id = sef.manager_id) LOOP
80                 Distributie(salsef, angajat.vechime, nrang1, nrang2, prima,
81                 grupa);
82
83                 dbms_output.Put_line(Rpad(numesef, 50)
84                                      || Rpad(salsef, 20)
85                                      || Rpad(angajat.fullname, 50)
86                                      || Rpad(angajat.vechime, 20)
87                                      || Rpad(prima, 20)
88                                      ||Rpad(grupa, 20));
89             END LOOP;
90         END LOOP;
91     END;
92 END;
93 /
```

Tipul Colecție ca parametri

Pentru a folosi un tip colecție ca parametru pentru o procedură stocată acesta trebuie să fie definit la nivelul bazei de date folosindu-se instrucțiunea DDL create. Tipurile colecție care pot fi declarate la nivelul bazei de date sunt:

- Varray
- Nested table

Ex. 6. Să se creeze un vector fie la nivelul dicționarului de date, fie local. Într-un bloc, să se definească și inițializeze o variabilă de tipul nou creat și să se apeleze o procedură care afișează valorile elementelor. Să se șteargă tipul din dicționarul de date.

```sql
-- versiune varray stocat
CREATE OR REPLACE TYPE v_culori IS VARRAY(6) OF VARCHAR2(20);
/

DECLARE
  PROCEDURE itereaza(culori IN v_culori)
  IS
  BEGIN
    FOR contor IN culori.FIRST .. culori.LAST
    LOOP
      dbms_output.put_line(culori(contor));
    END LOOP;
  END;
BEGIN
  DECLARE
    culori v_culori;
  BEGIN
    culori := v_culori('Rosu', 'Galben', 'Albastru', 'Verde', 'Maro', 'Mov');
    itereaza(culori);
  END;
END;
/

DROP TYPE v_culori;
```

```sql
-- vesriune varray local
DECLARE
  TYPE v_culori IS VARRAY(6) OF VARCHAR2(20);
  PROCEDURE itereaza(culori IN v_culori)
  IS
  BEGIN
    FOR contor IN culori.FIRST .. culori.LAST
    LOOP
      dbms_output.put_line(culori(contor));
    END LOOP;
  END;
BEGIN
  DECLARE
    culori v_culori;
  BEGIN
    culori := v_culori('Rosu', 'Galben', 'Albastru', 'Verde', 'Maro', 'Mov');
    itereaza(culori);
  END;
END;
/
```

```sql
-- vesiune nested table stocat
CREATE OR REPLACE TYPE t_culori IS TABLE OF VARCHAR2(20);
/

DECLARE
  PROCEDURE itereaza(culori IN t_culori)
  IS
  BEGIN
    FOR contor IN culori.FIRST .. culori.LAST
    LOOP
      dbms_output.put_line(culori(contor));
    END LOOP;
  END;
BEGIN
  DECLARE
    culori t_culori;
  BEGIN
    culori := t_culori('Rosu', 'Galben', 'Albastru', 'Verde', 'Maro', 'Mov');
    itereaza(culori);
  END;
END;
/

DROP TYPE t_culori;
```

```sql
-- vesiune nested table local
DECLARE
  TYPE t_culori IS TABLE OF VARCHAR2(20);
  PROCEDURE itereaza(culori IN t_culori)
  IS
  BEGIN
    FOR contor IN culori.FIRST .. culori.LAST
    LOOP
      dbms_output.put_line(culori(contor));
    END LOOP;
  END;
BEGIN
  DECLARE
    culori t_culori;
  BEGIN
    culori := t_culori('Rosu', 'Galben', 'Albastru', 'Verde', 'Maro', 'Mov');
    itereaza(culori);
  END;
END;
/
```

```plsql
-- vesiune associative array
DECLARE
  TYPE t_culori IS TABLE OF VARCHAR2(20) INDEX BY PLS_INTEGER;
  PROCEDURE itereaza(culori IN t_culori)
  IS
  BEGIN
    FOR contor IN culori.FIRST .. culori.LAST
    LOOP
      dbms_output.put_line(culori(contor));
    END LOOP;
  END;
BEGIN
  DECLARE
    culori t_culori;
  BEGIN
    culori := t_culori(1=>'Rosu', 2=>'Galben', 3=>'Albastru',
                       4=>'Verde', 5=>'Maro', 6=>'Mov');
    itereaza(culori);
  END;
END;
/
```

O procedură stocată poate să primească ca parametru un cursor. Acesta trebuie să fie de tipul SYS_REFCURSOR.

Ex. 7. Exemplu de cursor ca parametru.

```
DECLARE
    PROCEDURE Test_ref (c_dept IN SYS_REFCURSOR)
    IS
      TYPE r_dept IS RECORD (
        department_name departments.department_name%TYPE,
        loc locations.city%TYPE);
      dept_rec R_DEPT;
    BEGIN
        dbms_output.Put_line(Rpad('Nume', 20)
                             ||Rpad('Locatie', 20));

        LOOP
            FETCH c_dept INTO dept_rec;

            EXIT WHEN c_dept%NOTFOUND;

            dbms_output.Put_line(Rpad(dept_rec.department_name, 20)
                                 || Rpad(dept_rec.loc, 20));
        END LOOP;
    END;
BEGIN
    DECLARE
        TYPE dept_ref IS ref CURSOR;
        c_dept DEPT_REF;
    BEGIN
        OPEN c_dept FOR
          SELECT department_name,
                 locations.city
          FROM   departments
                 inner join locations
                         ON locations.location_id = departments.location_id;

        Test_ref(c_dept);

        IF c_dept%isopen THEN
          dbms_output.Put_line('Cursorul este deschis, trebuie inchis!');

          CLOSE c_dept;
        END IF;
    END;
END;
```

```plsql
DECLARE
    PROCEDURE Test_ref (c_dept OUT SYS_REFCURSOR)
    IS
    BEGIN
        OPEN c_dept FOR
          SELECT department_name,
                 locations.city
          FROM   departments
                 inner join locations
                         ON locations.location_id = departments.location_id;
    END;
BEGIN
    DECLARE
        TYPE dept_ref IS ref CURSOR;
        c_dept   DEPT_REF;
        TYPE r_dept IS RECORD (
          department_name departments.department_name%TYPE,
          loc locations.city%TYPE);
        dept_rec R_DEPT;
    BEGIN
        Test_ref(c_dept);

        dbms_output.Put_line(Rpad('Nume', 20)
                             || Rpad('Locatie', 20));

        LOOP
            FETCH c_dept INTO dept_rec;

            EXIT WHEN c_dept%NOTFOUND;

            dbms_output.Put_line(Rpad(dept_rec.department_name, 20)
                                 || Rpad(dept_rec.loc, 20));
        END LOOP;

        IF c_dept%isopen THEN
          dbms_output.Put_line('Cursorul este deschis, trebuie inchis!');

          CLOSE c_dept;
        END IF;
    END;
END;
```

```sql
DECLARE
    PROCEDURE Test_ref (ref_cursor IN OUT SYS_REFCURSOR)
    IS
      TYPE r_dept IS RECORD (
        department_name departments.department_name%TYPE,
        loc locations.city%TYPE);
      dept_rec R_DEPT;
    BEGIN
        dbms_output.Put_line(Rpad('Nume', 20)
                             ||Rpad('Locatie', 20));

        LOOP
            FETCH ref_cursor INTO dept_rec;

            EXIT WHEN ref_cursor%NOTFOUND;

            dbms_output.Put_line(Rpad(dept_rec.department_name, 20)
                                 || Rpad(dept_rec.loc, 20));
        END LOOP;

        IF ref_cursor%isopen THEN
          CLOSE ref_cursor;
        END IF;

        OPEN ref_cursor FOR
          SELECT first_name
                 || ' '
                 || last_name,
                 salary
          FROM   employees;
    END;
BEGIN
    DECLARE
        TYPE c_ref_cursor IS ref CURSOR;
        ref_cursor C_REF_CURSOR;
        TYPE r_emp IS RECORD (
          ename VARCHAR2(50),
          salary employees.salary%TYPE);
        emp_rec    R_EMP;
    BEGIN
        OPEN ref_cursor FOR
          SELECT department_name,
                 locations.city
          FROM   departments
                 inner join locations
                         ON locations.location_id = departments.location_id;

        dbms_output.Put_line(Rpad('Nume', 20)
                             ||Rpad('Locatie', 20));

        Test_ref(ref_cursor);

        dbms_output.Put_line(Rpad('Nume', 20)
                             ||Rpad('Salariu', 20));

        LOOP
            FETCH ref_cursor INTO emp_rec;

            EXIT WHEN ref_cursor%NOTFOUND;

```

```
            dbms_output.Put_line(Rpad(emp_rec.ename, 20)
                                 || Rpad(emp_rec.salary, 20));
        END LOOP;

        IF ref_cursor%isopen THEN
          CLOSE ref_cursor;
        END IF;
    END;
END;
```

Funcții PL/SQL

O funcție este un subprogram care acceptă parametri, poate fi apelată dintr-un program apelant și returnează o valoare. În general, funcțiile sunt folosite pentru a procesa un set de date și a întoarce un rezultat. Funcțiile și procedurile au structuri asemănătoare. O funcție trebuie să întoarcă o valoare folosind clauza **return** (și mai multe folosind parametri de tip OUT și IN OUT) în timp ce o procedură poate să întoarcă una sau mai multe valori prin intermediul parametrilor de tip OUT sau IN OUT.

Funcții declarate în cadrul unui bloc

Funcțiile declarate în cadrul unui bloc sunt funcții stocate în memorie împreună cu blocul PL/SQL în care sunt declarate și au o funcționalitate asemănătoare procedurilor. Durata de viață a unei astfel de funcții este doar pe perioada de existență a blocului. Aceste funcții pot să fie apelate doar în cadrul blocului în care sunt declarate. După terminarea execuției blocului funcțiile locale sunt șterse din memorie. Pentru a apela din nou o astfel de funcție trebuie să se execute din nou și blocul în care a fost declarată. Funcțiile pot fi folosite în blocul apelant, folosind operatorul de atribuire, în cadrul unor expresii și condiții;

Sintaxa:

```
DECLARE
  block_variables;
  var return_data_type; -- variabilă de tipul întors de funcție
  block_variables;

  FUNCTION function_name [(parameter_name [IN|OUT|IN OUT] parameter_type,...)] RETURN return_data_type
  {IS | AS}
    [function_declaration_section]
  BEGIN -- blocul funcției
    function_executable_section;
  [EXCEPTION
    function_exception_section]
  END [function_name]; -- sfârșitul declarării funcției
...
BEGIN
  block_executable_section;
  var := function_name[(parametrii)];
  block_executable_section;
[EXCEPTION
     block_exception_section]
END;
```

Unde:

- **function_name** – numele funcției
- **parameter_name** – numele unui parametru formal din lista de parametri
- **parameter_type** – reprezintă tipul parametrului formal, se specifica fără precizie
- **return_data_type** – tipul de date întors de funcție, se specifica fără precizie
- **function_declaration_section** – secțiunea de declarare a variabilelor locale folosite în funcție, poate să lipsească

- **function_executable_section** – secțiunea executabilă a funcției, trebuie să conțină cel puțin un **RETURN** care să întoarcă o variabilă care are tipul de date **return_data_type**
- **function_exception_section** – secțiune de tratare a excepțiilor din cadrul funcției, poate să lipsească, în schimb, dacă se folosește trebuie să conțină câte un **RETURN** pentru fiecare excepție tratată
- **block_variables** – secțiunea de declarare a variabilelor locale folosite în bloc
- **block_executable_section** – secțiunea executabilă a blocului
- **block_exception_section** – secțiunea de tratare a excepțiilor din cadrul blocului, este opțională
- **IN | OUT | IN OUT** – specifică dacă parametrul poate fi referit sau modificat în interiorul sau exteriorul funcției
- **var** – numele variabile în care funcția returnează o valoare, are tipul **return_data_type**

Ex. 8.Să se scrie o funcție locală care primește ca parametru un identificator de departament și returnează numărul salariaților din departamentul respectiv.

```plsql
DECLARE
    numedept departments.department_name%TYPE;
    iddept   departments.department_id%TYPE;
    nrang    NUMBER;
    FUNCTION F_salariati(p_iddept IN NUMBER)
    RETURN NUMBER
    AS
      nrang NUMBER;
    BEGIN
        SELECT Count(DISTINCT employee_id)
        INTO   nrang
        FROM   employees
        WHERE  department_id = p_iddept;

        RETURN nrang;
    END;
BEGIN
    iddept := &iddept;

    SELECT department_name
    INTO   numedept
    FROM   departments
    WHERE  department_id = iddept;

    nrang := F_salariati(iddept);

    dbms_output.Put_line('Departamentul '
                         || numedept
                         || ' are '
                         || nrang
                         || ' salariati');
EXCEPTION
    WHEN no_data_found THEN
      dbms_output.Put_line('Departament inexistent');
END;
```

Funcții stocate

Funcțiile stocate sunt funcții create în dicționarul bazei de date și pot fi accesate ca orice obiect, dacă utilizatorul are suficiente privilegii.

Sintaxa:

```
CREATE [OR REPLACE] FUNCTION function_name [(parameter_name [IN|OUT|IN OUT] parameter_type, ... )]
RETURN return_data_type
  [AUTHID {DEFINER|CURRENT_USER}]
  [PRAGMA AUTONOMOUS_TRANSACTION]
{IS|AS}
  [declaration_section]
BEGIN
  execution_section;
[EXCEPTION
  exception_section;]
END [function_name];
```

Unde:

- **function_name** – numele funcției
- **parameter_name** – numele unui parametru formal din lista de parametri
- **parameter_type** – reprezintă tipul parametrului formal, se specifica fără precizie
- **return_data_type** – tipul de date întors de funcție, se specifica fără precizie
- **declaration_section** – secțiunea de declarare a variabilelor locale folosite în funcție
- **executable_section** – secțiunea executabilă a funcției, trebuie să conțină cel puțin un **RETURN** care să întoarcă o variabilă care are tipul de date return_data_type
- **exception_section** – secțiune de tratare a excepțiilor din cadrul funcției, poate să lipsească
- **IN | OUT | IN OUT** – specifică dacă parametrul poate fi referit sau modificat în interiorul sau exteriorul procedurii
- **[AUTHID {DEFINER|CURRENT_USER}]** – specifică dacă o procedură stocată se execută cu drepturile celui care a creat-o (valoarea implicită) sau ale utilizatorului curent
- **[PRAGMA AUTONOMOUS_TRANSACTION]** – specifică că execuția procedurii suspendă tranzacția curentă care se reia după terminarea execuției procedurii, adică într-o tranzacție imbricăm o altă tranzacție cu propriile sale comenzi TCL (COMMIT și ROLLBACK)

Ex. 9 Să se scrie o funcție stocată, **f_puncte**, care calculează numărul de puncte acumulate de angajați în vederea numirii unui șef pe fiecare departament. Criteriile de punctaj și punctele aferente sunt următoarele:

1. vechimea în firmă
   - Ani vechime > 32 ⇒ 30 puncte
   - Ani vechime <= 32 ⇒ 15 puncte
2. salariul maxim pe departament
   - Salariu angajat = salariu maxim din departament ⇒ 20 puncte
   - Salariu angajat < salariu maxim din departament ⇒ 10 puncte
3. comision
   - Comision angajat > 0 ⇒ 10 puncte
   - Comision angajat = 0 ⇒ 5 puncte

Pentru stocarea punctelor acumulate de fiecare angajat, se va crea o tabelă temp_puncte care are următoarele coloane: idAng, nrPuncte, idDept.

```plsql
CREATE TABLE temp_puncte
  (
     idang    NUMBER,
     nrpuncte NUMBER,
     iddept   NUMBER
  );

CREATE OR replace FUNCTION F_puncte(p_idang IN NUMBER)
RETURN NUMBER
IS
  dataang  employees.hire_date%TYPE;
  iddept   departments.department_id%TYPE;
  salariu  employees.salary%TYPE;
  comision employees.commission_pct%TYPE;
  salmax   NUMBER;
  puncte   NUMBER := 0;
BEGIN
    -- iau informatiile de care am nevoie
    SELECT hire_date,
           department_id,
           salary,
           Nvl(commission_pct, 0)
    INTO   dataang, iddept, salariu, comision
    FROM   employees
    WHERE  employee_id = p_idang;

    -- calculez puncte in functie de vechime
    IF Months_between(SYSDATE, dataang) > 32 * 12 THEN
      puncte := puncte + 30;
    ELSE
      puncte := puncte + 15;
    END IF;

    -- iau salariul maximim din departament
    SELECT Max(salary)
    INTO   salmax
    FROM   employees
    WHERE  department_id = iddept;

    -- calculez puncte in functie de salariu
    IF salmax = salariu THEN
      puncte := puncte + 20;
    ELSE
      puncte := puncte + 10;
    END IF;

    -- calculez puncte in functie de comision
    IF comision > 0 THEN
      puncte := puncte + 10;
    ELSE
      puncte := puncte + 5;
    END IF;

    RETURN puncte;
END f_puncte;

/
DECLARE
    puncte     NUMBER;
    CURSOR c_angajati IS
      SELECT employee_id,
             first_name
             || ' '
             || last_name,
```

```
               department_id
      FROM     employees;
    numedept   departments.department_name%TYPE;
    idmgr      employees.manager_id%TYPE;
    numeang    VARCHAR2(50);
    dataang    employees.hire_date%TYPE;
    anivechime NUMBER;
    salmaxdept NUMBER;
    comision   NUMBER;
BEGIN
    -- sterg informatiile din tabelul
    DELETE FROM temp_puncte;

    -- calculez punctajul pentru fiecare angajat si
    -- inserez informatiile in tabelul temp_puncte
    FOR angajat IN c_angajati LOOP
        puncte := F_puncte(angajat.employee_id);

        INSERT INTO temp_puncte
        VALUES     (angajat.employee_id,
                    puncte,
                    angajat.department_id);
    END LOOP;

    -- antetul listei
    dbms_output.Put_line(Rpad('Departament', 20)
                         ||Rpad('Salariu maxim', 20)
                         ||Rpad('Nume angajat', 50)
                         ||Rpad('Ani vechime', 20)
                         ||Rpad('Comision', 20)
                         ||Rpad('Puncte angajat', 20));

    dbms_output.Put_line(Rpad('=', 120, '='));

    -- folosind un cursor selectez maximul de puncte pentru fiecare departament
    FOR contor1 IN (SELECT Max(nrpuncte) maxPuncte,
                           iddept
                    FROM   temp_puncte
                    GROUP  BY iddept
                    ORDER  BY iddept) LOOP
        -- folosind un cursor selectez toti angajatii care sunt din departamentul
        -- selectat de primul cursor si au maximul de puncte din departament
        FOR contor2 IN (SELECT idang
                        FROM   temp_puncte
                        WHERE  nrpuncte = contor1.maxpuncte
                               AND iddept = contor1.iddept) LOOP
            -- selectez numele departamentului
            SELECT department_name
            INTO   numedept
            FROM   departments
            WHERE  department_id = contor1.iddept;

            -- selectez salariul maxim din departament
            SELECT Max(salary)
            INTO   salmaxdept
            FROM   employees
            WHERE  department_id = contor1.iddept;

            -- selectez informatiile despre angajat
            SELECT hire_date,
                   Nvl(commission_pct, 0) * salary,
                   first_name
                   || ' '
                   || last_name
            INTO   dataang, comision, numeang
            FROM   employees
            WHERE  employee_id = contor2.idang;

```

```
            -- calculez vechimea angajatului
            anivechime := Trunc(Months_between(SYSDATE, dataang) / 12);

            -- afisez informatiile cerute
            dbms_output.Put_line(Rpad(numedept, 20)
                                 ||Rpad(salmaxdept, 20)
                                 ||Rpad(numeang, 50)
                                 ||Rpad(anivechime, 20)
                                 ||Rpad(comision, 20)
                                 ||Rpad(contor1.maxpuncte, 20));
        END LOOP;
    END LOOP;
END;
```

Observații:

- Funcția se va crea prima și dacă nu are erori de compilare se va afișa mesajul: **Function created**
- În caz contrar se va afișa mesajul: **Warning: Function created with compilation errors**
- Pentru a vedea erorile de compilare se va folosi comanda **show errors**
- Dacă în dicționarul de date există o funcție cu un nume, atunci nu se mai poate crea alta cu același nume
- Pentru a suprascrie o funcție existentă se folosește comanda **CREATE OR REPLACE**, folosiți comanda aceasta cu atenție deoarece puteți să înlocuiți din greșeală o funcție deja existentă, scrisă de un alt dezvoltator, care are același nume dar are altă funcționalitate
- Pentru a șterge o funcție stocată se folosește comanda DDL drop: **DROP FUNCTION function_name**
- Odată create și stocate în baza de date, funcțiile pot fi apelate în comenzi SQL, dar cu anumite restricții.

Funcția **f_puncte** creată anterior, care returnează numărul de puncte obținute de un angajat în vederea întocmirii unui clasament pe departamente, poate fi folosită astfel:

```
SELECT d.dname, e.ename, f_puncte(e.empno) punctaj
FROM emp e
INNER JOIN DEPT d ON d.deptno = e.deptno
WHERE d.deptno = 10;

SELECT d.dname, e.ename, f_puncte(e.empno) punctaj
FROM emp e
INNER JOIN dept d ON d.deptno = e.deptno AND f_puncte(e.empno) >= 40
ORDER BY d.dname, punctaj DESC;

SELECT
  d.dname Departament,
  (SELECT MAX(sal) FROM emp WHERE deptno = e.deptno) Salariu_Maxim,
  e.ename NumeAngajat,
  trunc(months_between(sysdate, e.hiredate)/12) Ani_Vechime,
  nvl(e.comm, 0) Comision,
  f_puncte(e.empno) Punctaj
FROM emp e INNER JOIN dept d ON d.deptno = e.deptno
WHERE f_puncte(e.empno) = (SELECT MAX(f_puncte(empno)) FROM emp WHERE deptno = e.deptno)
ORDER BY d.dname;
```

Informațiile obținute din ultimul select sunt echivalente cu cele obținute prin programul PL/SQL prezentat în exercițiul 3. Se observă că dacă utilizatorul are cunoștințe de SQL, acesta poate să creeze fără dificultăți liste simple. Comanda se bazează pe un cursor implicit creat de SGBD. Faptul că o comandă SQL poate să facă același lucru ca un bloc PL/SQL nu înseamnă că este mai ușor să scrii comenzi SQL decât blocuri PL/SQL. Totuși, în dezvoltarea aplicațiilor, un dezvoltator va încerca întotdeauna, în momentul construirii unui bloc PL/SQL, să minimizeze numărul de cereri SQL, încercând să obțină toate informațiile necesare folosind cât mai puține cereri SQL.

Considerente asupra procedurilor și funcțiilor

În general, o procedură este folosită pentru secvențe de cod mai complexe, în timp ce o funcție este folosită în cadrul procedurii pentru calcule repetate și mai simple. De exemplu, într-o aplicație de salarizare:

- O procedură se pretează pentru implementarea algoritmului de calcul al salariului net, considerând salariul de bază, pontajele lunare, sporurile, creșterile salariale, etc.
- O funcție se pretează pentru calculul taxelor și impozitelor aferente

Din punct de vedere tehnic, sunt câteva mici diferențe între proceduri și funcții:

- O funcție întoarce întotdeauna în mod direct o valoare în programul apelant și poate întoarce mai multe valori prin parametrii de tip **OUT** sau **IN OUT**
- O procedură nu returnează direct o valoare, dar poate întoarce mai multe valori în programul apelant prin parametrii de tip **OUT** sau **IN OUT**
- Funcțiile stocate pot fi apelate direct din comenzi SQL
- Procedurile stocare **NU** pot fi apelate direct din comenzi SQL
- Funcțiile pot fi folosite în operațiile de atribuire, în expresii și condiții (adică, atât în clauza **WHERE** cât și în condițiile structurilor de control)
- Procedurile **NU** pot fi folosite în operațiile de atribuire, în expresii și condiții (adică, atât în clauza **WHERE** cât și în condițiile structurilor de control)
- Procedurile și funcțiile pot fi apelate direct din programe PL/SQL, sau din interfețe specifice, de exemplu *Oracle Forms Builder* sau *Oracle Reports Builder*
- Procedurile și funcțiile pot fi apelate și din limbaje de programare de nivel înalt (Java, C++, C#, Python, etc) folosindu-se API-uri dedicate.

Funcțiile apelate din cereri SQL trebuie să îndeplinească următoarele condiții:

- Când este folosită într-o cerere **SELECT**, funcția **NU** trebuie să conțină comenzi **INSERT**, **UPDATE** sau **DELETE**
- Când este folosită în comenzile **INSERT**, **UPDATE** sau **DELETE**, funcția **NU** trebuie să modifice conținutul tabelelor afectate de comanda SQL
- Când este folosită în comenzile **SELECT**, **INSERT**, **UPDATE** sau **DELETE** , funcția **NU** trebuie să conțină comenzi **TCL** (e.g., **COMMIT** sau **ROLLBACK**).

Dacă într-o procedură sau funcție apare o excepție, tratată de sistem sau definită de utilizator, controlul este preluat de secțiunea **EXCEPTION** sau de blocul superior, iar parametrilor de tip **OUT** sau **IN OUT** nu li se transmite nicio valoare. În cazul în care apar excepții, parametrii actuali din procedura sau funcția apelată vor păstra aceleași valori pe care le-au avut înainte de a se face apelul. Execuția comenzilor **CREATE OR REPLACE** sau **DROP** pentru proceduri și funcții , precum și a comenzilor **ALTER TABLE** sau **ALTER VIEW** pentru tabele și view-uri, pot modifica starea altor obiecte din baza de date cu care sunt relaționate. De exemplu dacă într-o procedură este declarat un cursor care lucrează pe o anumită tabelă, atunci ștergerea sau modificarea structurii tabelei poate invalida procedura care apelează cursorul respectiv. Probabilitatea apariției acestor efecte, ca urmare a relaționării obiectelor, este ridicată și de aceea serverul Oracle le tratează corespunzător în timpul execuției sau apelării obiectelor corespunzătoare. La apariția unei astfel de situații serverul modifică în dicționarul bazei de date starea obiectelor afectate ca **DISABLE** și este necesară o recompilare a lor pentru a reveni în starea **ENABLE**.

Sistemul de gestionare ORACLE permite acordarea privilegiului de execuție asupra procedurilor și funcțiilor și altor utilizatori, acțiunea permisă proprietarului lor (utilizatorul care le-a creat) sau administratorului care are privilegii de DBA. Sintaxa comenzii **GRANT** este:

```
GRANT EXECUTE ON {procedure_name | function_name} TO user_name;
```

Ștergerea privilegiilor se face folosind comanda **REVOKE**, sintaxa este:

```
REVOKE EXECUTE ON {procedure_name | function_name} FROM user_name;
```

Informații din dicționarul bazei de date

Codul sursă al procedurilor și funcțiilor stocate se găsește în dicționarul bazei de date, în view-ul USER_SOURCE, care are următoarea structură:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS	INSERTABLE	UPDATABLE	DELETABLE
NAME	VARCHAR2(30)	Yes	(null)	1	Name of the object	NO	NO	NO
TYPE	VARCHAR2(12)	Yes	(null)	2	Type of the object: "TYPE", "TYP...	NO	NO	NO
LINE	NUMBER	Yes	(null)	3	Line number of this line of source	NO	NO	NO
TEXT	VARCHAR2(4000)	Yes	(null)	4	Source text	NO	NO	NO

Unde:

- **name** – este numele obiectului (procedură, funcție, etc)
- **type** – reprezintă tipul obiectului
- **line** – este numărul liniei de cod
- **text** – este codul sursă de la linia respectivă

Dacă vrem să vedem toate liniile de cod ale funcției f_puncte, putem folosi următoarea cerere SQL:

```
SELECT text FROM user_source WHERE LOWER(name) = 'f_puncte' ORDER BY line;
```

```
TEXT
FUNCTION f_puncte(p_idAng in number)

  RETURN NUMBER

  IS

    dataAng EMPLOYEES.HIRE_DATE%TYPE;

    idDept DEPARTMENTS.DEPARTMENT_ID%TYPE;

    salariu EMPLOYEES.SALARY%TYPE;

    comision EMPLOYEES.COMMISSION_PCT%TYPE;

    salMax number;

    puncte number := 0;

  BEGIN

    -- iau informatiile de care am nevoie

"    SELECT HIRE_DATE, DEPARTMENT_ID, SALARY, nvl(COMMISSION_PCT, 0)
"
"      INTO dataAng, idDept, salariu, comision
"
      FROM EMPLOYEES WHERE EMPLOYEE_ID = p_idAng;



    -- calculez puncte in functie de vechime

"    IF months_between(sysdate, dataAng) &gt; 32*12 THEN
"
      puncte := puncte + 30;

    ELSE

      puncte := puncte + 15;

    END IF;
```

```
  -- iau salariul maximim din departament
  SELECT max(SALARY) INTO salMax FROM EMPLOYEES WHERE DEPARTMENT_ID = idDept;
  -- calculez puncte in functie de salariu
  IF salMax = salariu THEN
    puncte := puncte + 20;
  ELSE
    puncte := puncte + 10;
  END IF;

  -- calculez puncte in functie de comision
  IF comision &gt; 0 THEN
    puncte := puncte + 10;
  ELSE
    puncte := puncte + 5;
  END IF;

  RETURN puncte;
END f_puncte;
```

Pentru a vizualiza toate procedurile, funcțiile și tipurile de date create de utilizatorul curent, se poate folosi următoarea cerere SQL:

```
SELECT object_name, object_type
FROM user_objects
WHERE LOWER(object_type) IN ('function', 'procedure', 'type');
```

OBJECT_NAME	OBJECT_TYPE
F_IMPOZIT	FUNCTION
F_PUNCTE	FUNCTION
SP_VENIT	PROCEDURE

bdd/laboratoare/05.txt · Last modified: 2024/10/07 14:39 by ciprian.truica

Laboratorul 06 - PL/SQL Pachete și Triggere

Conținut

- Pachete PL/SQL
- Specificațiile unui pachet
- Corpul unui pachet
- Restricții în definirea unui pachet
- Informații din dicționarul de date
- Pachete standard

Pachete PL/SQL

Un pachet (package) este o bibliotecă de obiecte stocate pe server, de tipul procedurilor stocate, funcțiilor, cursoarelor, tipurilor de date, excepțiilor, variabilelor și constantelor. Toate obiectele declarate în secțiunea de creare a unui pachete sunt globale (publice) și pot fi apelate din orice program PL/SQL, asemănător variabilelor globale din alte limbaje de programare; Un pachet este compus din două secțiuni distincte:

- O secțiune de creare (create package) care conține specificațiile globale (publice) de declarare a conținutului (structura obiectelor)
- O secțiune care cuprinde corpul pachetelor (package body) în care sunt descrise efectiv obiectele și subprograme, variabile, structuri locale (private)

Un pachet este creat ca un obiect în dicționarul bazei de date.

Principalele avantaje oferite de un pachet sunt:

- Modularitatea aplicațiilor
- Posibilitatea declarării de obiecte globale
- Îmbunătățirea performanțelor sistemului de gestiune
- Ușurința în proiectarea aplicațiilor
- Adăugarea de funcționalități noi

Specificațiile Pachetului

Specificațiile reprezintă partea publică a unui pachet și au următoarea sintaxă:

```
        CREATE [OR REPLACE] PACKAGE package_name {IS|AS}
                global (public) types AND variable declarations
                global (public) subprogram specifications
        END [package_name]
```

Unde :

- package_name – numele pachetului
- global (public) types and variable declarations – reprezintă declarațiile globale (publice) ale cursoarelor, excepțiilor, constantelor, variabilelor și tipurilor de date și descrierea acestora * din punct de vedere al structurii
- global (public) subprogram specifications – reprezintă numele procedurilor și funcțiilor cu parametrii formali aferenți. Acestea sunt globale (publice).

Subprogramele, variabilele și cursoarele care vor fi apelate din exterior trebuie să fie cuprinse în secțiunea de creare a pachetului, pe când subprogramele, cursoarele, excepțiile, constantele, variabilele și tipurile de date folosite doar în pachet vor fi declarate doar în secțiunea body a acestuia. Referirea într-un program PL/SQL a unei componente globale (publice) a unui pachet se face folosind numele componentei, având ca prefix numele pachetului:

```
package_name.type_name
package_name.item_name
package_name.subprogram_name
```

Unde:

- package_name – numele pachetului
- type_name – numele unui tip de date
- item_name – numele unui cursor, excepție, constantă sau variabilă locală
- subprogram_name – numele unei proceduri sau funcții

Dacă compilarea unui pachet se face cu succes, apare mesajul: Package created. În caz că apar erori la crearea unui pachet apare mesajul de avertizare: Warning: Package created with compilation errors.

Pentru a vedea erorile de compilare se va folosi comanda: show errors.

Corpul Pachetului

Corpul unui pachet (package body) conține descrierea efectivă a procedurilor și funcțiilor definite în specificații. Această parte poate să conțină și componente locale (private) care sunt folosite doar în interiorul pachetului.

Sintaxa este:

```
CREATE [OR REPLACE] PACKAGE BODY package_name IS|AS
        LOCAL TYPE AND variable declarations
        subprogram bodies
        [BEGIN
                initialization statements
        END;]
END [package_name]
```

Unde:

- package_name – numele pachetului
- local type and variable declarations – reprezintă declarațiile locale ale cursoarelor, excepțiilor, constantelor, variabilelor și tipurilor de date
- subprogram bodies – codul sursă al procedurilor și funcțiilor definite în specificații
- initialization statements – codul de inițializare

Cursoarele, tipurile de date, excepțiile, variabilele și constantele declarate în corpul pachetului vor avea caracter local (privat), deci vor fi accesibile numai în blocurile în care au fost definite. La primul apel al unui obiect dintr-un pachet, întregul pachet este inițializat. Inițializarea implică încărcarea unui pachet de pe disc în memorie și alocarea de spațiu în memorie pentru variabilele globale (publice).

Dacă apelul se referă la o procedură sau funcție, inițializarea este urmată de execuția codului deja compilat. Fiecare sesiune de lucru are o copie proprie a variabilelor din pachet. Sesiuni diferite pot apela același pachet, dar fiecare apel al pachetului aițializări prin atribuiri directe sau apelează proceduri de inițializare.

Ex. 1 Să se scrie un pachet p_angajare, care conține o funcție și o procedură, pentru a face o listă cu angajații care au comision și au venit în firmă înaintea șefului direct. Șeful direct al unui angajat este specificat în coloana manager_id. Procedura va afișa rezultatele.

```sql
-- crearea specificatiilor pahetului
CREATE OR REPLACE PACKAGE p_angajare AS
    CURSOR dept(did NUMBER) IS
        SELECT department_name
        FROM departments
        WHERE department_id = did
        ORDER BY 1;
    FUNCTION Vechime(hdate_ang date, hdate_sef date) RETURN boolean;
    PROCEDURE Listare;
END p_angajare;
/

-- crearea corpului pachetului
CREATE OR REPLACE PACKAGE BODY p_angajare AS
    FUNCTION Vechime(hdate_ang date, hdate_sef date)
    RETURN boolean AS
    BEGIN
        IF hdate_ang < hdate_sef THEN RETURN true;
        ELSE RETURN false;
        END IF;
    END Vechime;

    PROCEDURE Listare
    IS
        CURSOR c_ang IS
            SELECT
                e.first_name||' '||e.last_name nume_ang,
                e.hire_date hdate_ang,
                e.commission_pct comm_ang,
                e.department_id dept_ang,
                s.first_name||' '||s.last_name nume_sef,
                s.hire_date hdate_sef,
                s.department_id dept_sef
            FROM employees e
            LEFT OUTER JOIN employees s
                ON e.manager_id = s.employee_id;
        w_c c_ang%rowtype;
        d_ang departments.department_name%type;
        d_sef departments.department_name%type;
    BEGIN
        FOR i IN c_ang
        LOOP
            IF i.dept_ang IS NOT NULL THEN
                OPEN p_angajare.dept(i.dept_ang);
                LOOP
                    FETCH p_angajare.dept INTO d_ang;
                    EXIT WHEN p_angajare.dept%NOTFOUND;
                END LOOP;
                IF dept%ISOPEN THEN
                    CLOSE dept;
                END IF;
            ELSE
                d_ang := 'N/A';
            END IF;

            IF i.dept_sef IS NOT NULL THEN
                OPEN p_angajare.dept(i.dept_sef);
                LOOP
                    FETCH p_angajare.dept INTO d_sef;
                    EXIT WHEN p_angajare.dept%NOTFOUND;
                END LOOP;
                IF dept%ISOPEN THEN
                    CLOSE dept;
                END IF;
            ELSE
                d_sef := 'N/A';
            END IF;

            IF Vechime(i.hdate_ang, i.hdate_sef) AND i.comm_ang IS NOT NULL THEN
                dbms_output.put_line(rpad(i.nume_ang, 30)||' '||
                    rpad(d_ang, 30)||' '||
                    rpad(i.nume_sef, 30)||' '||
                    rpad(d_sef, 30));
            END IF;
        END LOOP;
    END Listare;
```

```
    END Listare;
END p_angajare;
/

-- bloc apelant
BEGIN
    p_angajare.Listare;
END;
/
```

Restricții

În pachete nu se permite declararea a două proceduri sau funcții cu același nume, dacă parametrii acestora diferă numai prin nume sau mod (IN, OUT, IN OUT). Trebuie ca cel puțin un parametru să fie de un alt tip, iar tipul nu trebuie să fie din aceeași familie(de exemplul tipul CHAR este din aceeași familie cu VARCHAR2). Aceeași situație este și în cazul rezultatelor returnat de o funcție.

Ex. 2 Exemplu de ambiguitate:

```
CREATE OR REPLACE PACKAGE p_test AS
  FUNCTION f_test(data_ang date, numar integer) RETURN number;
  FUNCTION f_test(data_ang date, numar decimal) RETURN number;
END p_test;
/

CREATE OR REPLACE PACKAGE BODY p_test AS
  FUNCTION f_test(data_ang date, numar integer) RETURN number IS
    nr number :=0;
  BEGIN
    IF data_ang < sysdate THEN nr := 1;
    END IF;
    RETURN nr;
  END f_test;

  FUNCTION f_test(data_ang date, numar decimal) RETURN number IS
    nr number :=2;
  BEGIN
    IF data_ang < sysdate THEN nr := 3;
    END if;
    RETURN nr;
  END f_test;
END p_test;
/

DECLARE
  contor number;
BEGIN
  contor := p_test.f_test(sysdate, 5);
END;
/
```

- Exemplul se compilează fără erori.
- Dacă executăm blocul, atunci o să apară o eroare!
- Aceast mod de funționare poate să creeze o anumită confuzie.

Dacă schimbăm tipul celui de-al doilea parametru în ceva care nu este compatibil cu tipurile numerie, VARCHAR2 să zicem, totul merge cum trebuie.

O variabilă globală (publică) poate fi folosită pentru a inițializa un parametru al subprogramelor (public/privat) cât și în interiorul acestora.

Ex. 3. Exemplu de utilizarea a unei variabile publice.

```
CREATE OR REPLACE PACKAGE apel AS
  var number := 10;
  PROCEDURE test(nr number := var);
END apel;
/

CREATE OR REPLACE PACKAGE BODY apel AS
  PROCEDURE test(nr number := var) IS
  BEGIN
    dbms_output.put_line(nr);
  END test;
END apel;
/

BEGIN
  dbms_output.put_line(apel.var);
  apel.var := 20;
  apel.test;
END;
/
```

O variabilă locală (privată) poate fi folosită pentru a inițializa un parametru al unei subprogram local (privat) și în interiorul subprogramelor (private/publice).

Ex. 4. Exemplu de utilizarea a unei variabile private.

```plsql
CREATE OR REPLACE PACKAGE apel AS
  PROCEDURE test(nr number);
END apel;
/

CREATE OR REPLACE PACKAGE BODY apel AS
  var number := 10; -- variabila privata
  -- procedura privata
  PROCEDURE test_privat(nr number := var) IS
  BEGIN
    dbms_output.put_line(var + nr);
  END test_privat;

  PROCEDURE test(nr number) IS
  BEGIN
    test_privat(nr + var);
    test_privat;
  END test;
END apel;
/

BEGIN
  apel.test(1);
END;
/
```

Informații din dicționarul bazei de date

Deoarece pachetele sunt create ca oricare alt obiect, din dicționarul bazei de date putem să aflăm informații despre ele făcând interogări pe view- urile sistemului de gestiune ORACLE. De exemplu dacă vrem să vedem toate pachetele create de userul curent, data când au fost create, data ultimei modificări și starea lor, putem să executăm următoarea cerere SQL:

```
SELECT object_name, created, last_ddl_time, STATUS
FROM user_objects
WHERE object_type = 'PACKAGE';
```

Pentru a vedea care pahete au specificații și nu au secțiunea body se execută următoarea cerere SQL:

```
SELECT object_name, created, STATUS
FROM user_objects
WHERE
        object_type = 'PACKAGE' AND
        object_name NOT IN (SELECT object_name
                            FROM user_objects
                            WHERE object_type = 'PACKAGE BODY');
```

Pentru a vedea secțiunea de specificații a pachetului p_test, executăm următoarea cerere SQL.

```
SELECT text
FROM user_source
WHERE
        LOWER(name) = 'p_test' AND
        LOWER(TYPE) = 'package'
ORDER BY line;
```

Pentru a lista codul sursă a pachetului p_test, se execută următoarea cerere SQL.

```
SELECT text
FROM user_source
WHERE
        LOWER(name) = 'p_test' AND
        LOWER(TYPE) = 'package body'
ORDER BY line;
```

Specificațiile și corpul unui pachet se pot șterge din dicționar folosind comanda DDL DROP:

```
DROP PACKAGE pakage_name;
```

Pentru a șterge numai corpul unui pachet se folosește comanda SQL:

```
DROP PACKAGE BODY package_name;
```

Pentru a da privilegii de execuție a pachetelor altor utilizatori, utilizatorul care a creat pachetul (sau administratorul) poate folosi comanda DCL, GRANT:

```
GRANT EXECUTE ON package_name TO user_name;
```

Utilizatorul grantificat poate apela un obiect din pachetul respectiv, specificând în apel userul, pachetul și obiectul. De exemplu, o procedură fără parametri poate fi apelată direct din SQL*Plus astfel:

```
EXECUTE user_name.package_nane.procedure_name;
```

O procedură poate fi apelată și dintr-un bloc SQL:

```
DECLARE
BEGIN
  ...
  user_name.package_name.procedure_name; ...
END;
```

Pentru a revoca privilegiile de execuție se folosește comanda REVOKE:

```
REVOKE EXECUTE ON package_name FROM user_name;
```

Serverul Oracle conține câteva pachete standard(de sistem), care sunt instalate odată cu serverul de baze de date. Câteva din cele mai uzuale pachete standard sunt:

- **DBMS_STANDARD** conține proceduri care ajută programatorul în interacțiunea cu Oracle. De exemplu, procedura raise_application_error este folosită pentru definirea propriilor mesaje;
- **DBMS_OUTPUT** conține proceduri pentru afișare, folosite în depanarea sau execuția programelor. De exemplu, procesura put_line este folosită pentru afișarea de mesaje în SQL*Plus;
- **DBMS_PIPE** permite comunicarea între sesiuni diferite, folosind o zonă de memorie comună (pipe), pentru schimbul de informații. O sesiune poate folosi două proceduri pack_message și send_message, pentru a împacheta și pune mesajul în zona comună și apoi a-l trimite către o altă sesiune. Sesiunea receptoare poate folosi două proceduri receive_message și unpack_message, care fac operațiunile

inverse. De exemplu, se pot scrie rutine C++, care permit serverelor externe să capteze informații și să le trimită către proceduri stocate în baza de date Oracle.

UTL_FILE – permite programatorilor PL/SQL să scrie și să citească fișiere text gestionate de sistemul de operare. Pentru aceasta, se poate folosi funcția fopen pentru deschiderea fișierului și procedura get_line pentru citirea linie cu linie;

- **UTL_HTML** permite programatorilor PL/SQL să acceseze Internetul sau Oracle Web Server, folosind protocolul HTTP; Pachetul acceptă un URL, se conectează la site-ul specificat și întoarce datele solicitate, de regulă în format HTML;
- **DBMS_SQL** permite unui programator PL/SQL să execute comenzi DDL sau comenzi SQL standard, în mod dinamic;
- **DBMS_ALERT** permite utilizarea triggerelor pentru alertare, în cazul în care intervin modificări în baza de date.

Informațiile din dicționar, legate de pachetele standard, se obțin folosind view-urile de sistem DBA_OBJECTS și DBA_SOURCE, acesibile userului system sau celor care au privilegiul DBA.

Pentru vizualizarea mai multor informații din dicționar, legate de pachetele standard, se poate folosi cererea SQL:

```
SELECT text
FROM dba_source WHERE LOWER(name) = 'package_name'
  AND LOWER(TYPE)='package'
ORDER BY line;
```

Pachetele standard pot fi activate sau dezactivate (ENABLE/DISABLE) cu comanda SET.

```
SET standard_package {ON|OFF}
```

Triggere PL/SQL

Un trigger este un bloc PL/SQL stocat pe server care se execută la apariția unui eveniment care modifică starea anumitor obiecte ale bazei de date. Termenul corespondent în literatura de specialitate românească este declanșator, dar este rar folosit și de aceea în continuare se va folosi termenul în limba engleză. Tipuri de evenimente care pot determina execuția unui trigger sunt:

- Comenzi INSERT, UPDATE, DELETE pe o tabelă
- Comenzi INSERT, UPDATE, DELETE pe un view cu opțiunea INSTEAD OF
- Comenzi CREATE, ALTER, DROP la nivel de schemă sau bază de date
- Comenzi SHUTDOWN, LOGON, LOGOFF la nivel de schemă sau bază de date

În general, triggerii se folosesc pentru:

- Gestionarea restricțiilor complexe de integritate
- Monitorizarea tranzacțiilor
- Efectuarea de replicări de tabele situate în diferite noduri ale unei baze de date distribuite
- Păstrarea semnăturii userilor care au efectuat operații pe baza de date
- Prelucrarea de informații statistice în legătură cu accesul tabelelor
- Jurnalizarea transparentă a evenimentelor

Printre avantajele utilizării triggerilor, se pot menționa:

- Declanșarea automată, la apariția evenimentului monitorizat
- Lansarea în execuție a unor proceduri stocate specifice
- Posibilitatea modificării în cascadă a mai multor obiecte corelate în baza de date
- Transparența față de utilizator

Sintaxa unui trigger este:

```
CREATE [OR REPLACE] TRIGGER [schema.]trigger_name
        {BEFORE | AFTER | INSTEAD OF}
        {DELETE | INSERT | UPDATE [OR {DELETE | INSERT | UPDATE }…]
                [OF COLUMN[, COLUMN …] ]}
        ON [schema.]tabel _name
        [referencing_clauses]
        [FOR EACH ROW]
        [WHEN (condition) ]
        DECLARE
                trigger_variables
        BEGIN
                trigger_body
        END
```

Unde:

- trigger_name – numele triggerului PL/SQL
- schema – specifică schema pe care se definește triggerul sau în care există obiectele, în mod implicit este aleasă schema utilizatorului curent
- tabel _name – numele tabelul/view-ul pe care se monitorizează evenimentul
- column – numele coloanei (coloanelor) din tabelul/view-ul pe care se monitorizează evenimentul
- condition – reprezintă o condiție pentru executarea triggerului, fiind admise corelări dar nu și interogări
- trigger_variables – secțiunea de declarare a variabilelor locale ale triggerului
- trigger_body – reprezintă corpul triggerului
- BEFORE | AFTER – specifică momentul executării triggerului: înainte sau după apariția evenimentului
- INSTEAD OF – specifică că este permisă o operație de inserare, ștergere, modificare pe view-uri, pentru care nu este permisă operația în mod firesc
- INSERT | UPDATE | DELETE – specifică evenimentul pe care se declanșează triggerul
- FOR EACH ROW – specifică dacă execuția triggerului se face pentru fiecare linie afectată, cu respectarea condiției din WHEN

Sintaxa referencing_clauses:

```
REFERENCING {OLD [AS] old_variable NEW [AS] new_variable |
                        NEW [AS] new_variable OLD [AS] old_variable}
```

Se folosesc nume corelate pentru a specifica valorile noi și vechi ale rândului curent pentru care se declanșează triggerul. Aceste valori se pot folosi atât în clauza WHEN (old_variable.row_field, new_variable.row_field) cât și în blocul PL/SQL folosindu-se prefixate de două puncte : (de exemplu :old_variable.row_field, :new_variable.row_field).

Numele implicite sunt OLD și NEW.

Sintaxa pentru crearea unui trigger de sistem este următoarea:

```
        CREATE [OR REPLACE] TRIGGER [schema.]trigger_name
                {BEFORE | AFTER}
                {DDL_event_list | DB_event_list}
                ON {DATABASE | SCHEMA}
                [WHEN (condition) ]
        DECLARE
                trigger_variables
        BEGIN
                trigger_body
        END
```

Unde:

- DDL_event_list - CREATE, DROP, ALTER

- DB_event_list - STARTUP, SHUTDOWN, LOGON, LOGOFF, SERVERERROR, SUSPEND

Crearea unui trigger

Există două tipuri de triggeri:

- Triggeri pe o comandă – sunt executați o singură dată pentru evenimentul declanșator. De exemplu dacă se execută o comandă INSERT de mai multe linii, triggerul este executat o singură dată. În acest caz, nu este limitare la numărul de linii afectate de eveniment;
- Triggeri pe o linie – este executat ori de câte ori o linie a unei tabele este afectată de evenimentul declanșator. De exemplu, dacă se execută o comandă UPDATE care actualizează k linii, atunci triggerul este executat de k ori.

Un trigger poate fi executat înainte ca un eveniment să aibă loc (opțiunea BEFORE) sau după ce evenimentul s-a consumat (opțiunea AFTER). În general, triggerii de tip BEFORE sunt folosiți pentru:

- A salva valorile coloanelor înaintea executării unei comenzi UPDATE
- A decide dacă acțiunea triggerului trebuie sau nu executată (aceasta poate îmbunătăți performanțele serverului prin eliminarea procesării inutile)

Triggeri de tip AFTER sunt, în general, folosiți atunci când:

- Se dorește ca executarea triggerului să se facă după ce comanda s-a efectuat cu succes
- Nu au apărut erori de procesare care ar impune o comandă ROLLBACK pentru respectiva tranzacție
- Trebuie alterate și alte date corelate cu cele deja afectate

Triggeri de tip BEFORE

Triggerii de tip BEFORE se declanșează la apariția unui eveniment, dar înainte ca evenimentul să se termine;

Ex. 5 Să se scrie un trigger de tip BEFORE care printează un mesaj ori de câte ori se face un insert în tabela JOBS.

```
1 CREATE OR REPLACE TRIGGER insertjob
2 BEFORE INSERT ON jobs
3 BEGIN
4   dbms_output.put_line('S-a adaugat o noua functie');
5 END;
6 /
7
```

Să verificăm cum lucrează făcând un insert în tabela JOBS:

```
INSERT INTO jobs(job_id, job_title, min_salary, max_salary)
VALUES('IT_SA', 'System Administrator', 6000, 12000);
```

Se observă că triggerul s-a declanșat și operația s-a făcut cu succes.

Să veden ce se întâmplă dacă forțăm o eroare la inserare (pe coloana JOB_TITLE este o constrângere de tip NOT NULL):

```
INSERT INTO jobs(job_id, min_salary, max_salary)
VALUES('IT_TL', 7000, 19000);
```

Se observă că triggerul s-a declanșat normal, nu a ținut cont că inserarea nu a fost efectuată și a generat o eroare.

Dacă compilarea unui trigger se face cu succes, apare mesajul: "Trigger created"

În caz că apar erori la crearea unui trigger apare mesajul de avertizare: "Warning: Trigger created with compilation errors."

Pentru a vedea erorile de compilare se va folosi comanda: "show errors;"

Triggeri de tip AFTER

Triggerii de tip AFTER se declanșează după ce evenimentul declanșator se termină.

Ex. 6 Să se creeze un trigger de tip AFTER care afișează un mesaj ori de câte ori se face o modificare în tabela JOBS.

```
1 CREATE OR REPLACE TRIGGER updatejob
2 AFTER UPDATE ON jobs
3 BEGIN
4     dbms_output.put_line('S-a modificat o functie');
5 END;
6 /
7
```

Să verificăm cum lucrează triggerul făcând o modificare a salariului maxim în tabelul JOBS:

```
UPDATE jobs SET max_salary = 14000 WHERE job_id = 'IT_SA';
```

Se observă că triggerul a fost declanșat la apariția evenimentului, în acest caz comanda UPDATE și s-a afișat mesajul de avertizare. Triggerul se declanșează chiar dacă nu este găsită nicio înregistrare care să îndeplinească condițiile din clauza WHERE.

```
UPDATE jobs SET max_salary = 14000 WHERE job_id = 'IT_TM';
```

Deoarece nu există nicio functie cu ob_id = 'IT_TM', nu a fost alterată nicio linie, totuși triggerul s-a declanșat, deoarece nu a apărut nicio eroare de execuție.

Dacă însă operația se termină cu o eroare, triggerul nu se mai declanșează.

```
UPDATE jobs SET job_id = NULL WHERE job_id = 'IT_SA';
ROLLBACK;
```

Am încercat să atribuim valoarea NULL pentru job_id, ceea ce este interzis prin definirea tabelei, ca urmare s-a generat o eroare de sistem pe constrângerea respectivă și triggerul nu s-a declanșat.

- Putem trage concluzia că un trigger de tip BEFORE se declanșează necondiționat de rezultatul comenzii SQL (chiar dacă se generează o eroare), pe când cel de tip AFTER nu se declanșează dacă comanda SQL generează o eroare.
- S-a utilizat comanda ROLLBACK pentru ca modificările să nu rămână permanente.

Restricții în clauza WHEN

Dacă vrem să introducem o restricție pentru declanșarea triggerului, putem să folosim clauza WHEN. Această clauză se poate folosi numai pentru triggerii care se declanșează pentru fiecare linie ce suferă modificări, deci poate fi folosită doar cu opțiunea FOR EACH ROW. În clauza WHEN se acceptă numai condiționări directe sau corelate, nu se acceptă cereri sau subcereri.

Ex. 7 Să se scrie un trigger de tip AFTER care se declanșează dacă salariul unui angajat cu funcția 'SH_CLERK' este majorat.

```
CREATE OR REPLACE TRIGGER majorareSal
AFTER UPDATE OF salary ON employees
FOR EACH ROW
WHEN (new.salary > old.salary and old.JOB_ID = 'SH_CLERK')
DECLARE
  nume_ang varchar2(45);
BEGIN
  nume_ang := :old.first_name||' '||:old.last_name;
  dbms_output.put_line('Salariul lui '|| nume_ang ||
    ' a fost majorat de la '||:old.salary||' la '||:new.salary);
END;
/
```

Care este rezultatul dacă se face o majorare de salariu pentru angajatul cu id-ul 182? Dar pentru cel cu id-ul 120?

```
UPDATE employees SET salary = 3000 WHERE employee_id = 182;
UPDATE employees SET salary = 3000 WHERE employee_id = 120;
ROLLBACK;
```

- În clauza WHEN, **variabilele old și new se apelează fără caracterul ':'**
- În zona de execuție a triggerului, **variabilele old și new se apelează cu caracterul ':'**

Predicate condiționate

În cazul în care se execută mai multe comenzi DML, se pot folosi predicate condiționate în corpul triggerului. Predicatele condiționate sunt:

- INSERTING – returnează TRUE dacă triggerul se declanșează pe o comandă INSERT;
- UPDATING – retunează TRUE dacă triggerul se declanșează pe o comandă UPDATE;
- UPDATING (‘column_name’) – retunează TRUE dacă triggerul se declanșează pe o comandă UPDATE care modifică coloana specificată prin column_name;
- DELETING – returnează TRUE dacă triggerul se declanșează pe o comandă DELETE.

Ex. 8 Să se scrie un trigger de tip AFTER care se declanșează în momentul în care se face un insert, delete sau update pe coloanele SALARY și COMMISSION_PCT din tabela EMPLOYEES. Să se afiseze mesaje cu operați efectuată la declanșarea triggerului.

```plsql
CREATE OR REPLACE TRIGGER monitor
  AFTER INSERT OR DELETE OR UPDATE OF salary, commission_pct
  ON employees
  FOR EACH ROW
DECLARE
  sal_old employees.salary%type := :old.salary;
  com_old employees.commission_pct%type := :old.commission_pct;
  sal_new employees.salary%type := :new.salary;
  com_new employees.commission_pct%type := :new.commission_pct;
  tip_op varchar2(30);
  operator varchar2(20);
  data varchar2(20);
BEGIN
  data := to_char(sysdate, 'dd-mm-yyyy hh:mi:ss');
  -- utilizatorul care face operatia
  operator := user;
  IF INSERTING THEN
    tip_op := 'insert';
  ELSIF DELETING THEN
    tip_op := 'delete';
  ELSIF UPDATING('salary') AND UPDATING('commission_pct') THEN
    tip_op := 'update salariu si comision';
  ELSIF UPDATING('salary') THEN
    tip_op := 'update salariu';
  ELSIF UPDATING('commission_pct') THEN
    tip_op := 'update comision';
  END IF;
  dbms_output.put_line(
    'Operator: '||operator || chr(13)|| chr(10) ||
    'Tip operatie: '||tip_op || chr(13)|| chr(10) ||
    'Data operatie: '||data || chr(13)|| chr(10) ||
    'Old Salariu: '||nvl(to_char(sal_old), 'NULL') || chr(13)|| chr(10) ||
    'New Salariu: '||nvl(to_char(sal_new), 'NULL') || chr(13)|| chr(10) ||
    'Old Commision: '||nvl(to_char(com_old), 'NULL') || chr(13)|| chr(10) ||
    'New Commision: '||nvl(to_char(com_new), 'NULL') || chr(13)|| chr(10)
    );
END;
/
```

Verificări pentru Insert, Update si Delete:

```plsql
INSERT INTO employees VALUES(1000, 'Ion', 'Ionescu', 'IIONESCU', '321.321.3214',
         TO_DATE('17-06-2015', 'dd-MM-yyyy'), 'IT_PROG', 4200, NULL, NULL, 90);

UPDATE employees SET commission_pct = nvl(commission_pct, 0) + 0.05 WHERE employee_id = 1000;

UPDATE employees SET salary = NVL(salary, 0) + 500 WHERE employee_id = 1000;

UPDATE employees SET salary = NVL(salary, 0) + 500,
    commission_pct = nvl(commission_pct, 0) + 0.05  WHERE employee_id = 1000;

DELETE FROM employees WHERE employee_id = 1000;

ROLLBACK;
```

Triggere cu opțiunea INSTEAD OF

Acest trigger se definește numai pe view-uri, nu și pe tabele. Unele view-uri nu pot fi modificate prin comenzi DML, dar folosind un trigger cu opțiunea INSTEAD OF acest lucru este realizabil. View-urile care nu pot fi modificate prin comenzile UPDATE, INSERT sau DELETE sunt cele create printr-o interogare care conține în construcție:

- Un operator SET sau DISTINCT
- O funcție de agregare sau o funcție analitică
- Clauzele GROUP BY, ORDER BY, CONNECT BY sau START WITH

- O expresie tip colecție într-o clauză SELECT
- O subcerere într-o clauză SELECT
- Unele metode de JOIN

Orice view, aflat într-un astfel de caz, se poate face modificabil folosind un trigger cu opțiunea INSTEAD OF. Acest trigger trebuie să determine ce operație trebuie executată pentru modificarea tabelei pe care este creat view-ul respectiv. Dacă view-ul conține pseudocoloane sau expresii , acestea nu pot fi modificate direct printr-o comandă UPDATE, dar pot fi modificate prin trigger.

Ex. 9 Să creăm un view care selectează angajații care nu sunt sefi (un angajat este sef dacă apare pe coloana manager_id din tabelul employees).

```
CREATE OR REPLACE VIEW notsefi AS
  SELECT
    d.department_name, e.first_name, e.last_name, j.job_title,
    e.hire_date, e.salary, e.commission_pct
  FROM employees e
    INNER JOIN departments d ON e.department_id = d.department_id
    INNER JOIN jobs j ON e.job_id = j.job_id
  WHERE e.employee_id NOT IN
    (SELECT DISTINCT manager_id
        FROM employees
        WHERE manager_id IS NOT NULL);
```

Să facem o inserare în view-ul creat

```
INSERT INTO notsefi
VALUES('Treasury', 'Vasile', 'Ionescu', 'System Administrator', sysdate, 2000, 0.15);

INSERT INTO notsefi
VALUES('Mediu', 'Ion', 'Ionescu', 'System Administrator', sysdate, 2000, 0.15);

INSERT INTO notsefi
VALUES('Mediu', 'George', 'Ionescu', 'Help Deck', sysdate, 2000, 0.15);

ROLLBACK;
```

Comanda INSERT a generat o eroare, deoarece nu se acceptă inserarea într-un astfel de view.

Să creăm triggerul t_notsefi cu opțiunea INSTEAD OF care să permită operația de INSERT pe view.

```
CREATE OR REPLACE TRIGGER t_notsefi
INSTEAD OF INSERT ON notsefi
  REFERENCING new AS n
  FOR EACH ROW
DECLARE
  nr number;
  nid employees.employee_id%TYPE;
  did departments.department_id%TYPE;
  jid jobs.job_id%TYPE;
  email employees.email%TYPE;
BEGIN
  SELECT max(employee_id) + 1 INTO nid FROM employees;
  -- verificare daca se foloseste un departament nou sau unul existent
  SELECT count(*) INTO nr FROM departments
    WHERE lower(department_name) = lower(:n.department_name);
  IF nr = 0 THEN
    SELECT max(department_id) + 10 INTO did FROM departments;
    INSERT INTO departments (department_id, department_name)
      VALUES(did, :n.department_name);
  ELSE
    SELECT department_id INTO did FROM departments
      WHERE lower(department_name) = lower(:n.department_name);
  END IF;
  -- verificare daca se foloseste o functie noua sau una existenta
  SELECT count(*) INTO nr FROM jobs WHERE lower(job_title) = lower(:n.job_title);
  IF nr = 0 THEN
    jid := upper(substr(:n.job_title, 1, 4));
    INSERT INTO jobs (job_id, job_title) VALUES(jid, :n.job_title);
  ELSE
    SELECT job_id INTO jid FROM jobs WHERE lower(job_title) = lower(:n.job_title);
  END IF;

  email := upper(substr(:n.first_name, 1, 1)) || upper(:n.last_name);
  INSERT INTO employees(employee_id, first_name, last_name,
                        email, hire_date, job_id, salary, commission_pct, department_id)
  VALUES(nid, :n.first_name, :n.last_name,
         email, :n.hire_date, jid, :n.salary, :n.commission_pct, did);
END;
/
```

Să se execute din nou comanenzile de insert de mai sus. Ce observați?

În corpul triggerului nu trebuie să fie cuprinse instrucțiuni care să afecteze starea tabelei sau view-ului pe care se monitorizează evenimentul.

Informații din dicționarul bazei de date

Informațiile despre triggeri se pot obține din dicționarul de date, ca pentru proceduri, funcții și pachete. De exemplu, dacă vrem să vedem toți triggerii creați de userul curent, data când au fost creați, data ultimei utilizări și starea lor, putem executa următoarea cerere SQL:

```
SELECT OBJECT_NAME,
  CREATED,
  LAST_DDL_TIME,
  STATUS
FROM USER_OBJECTS
WHERE OBJECT_TYPE = 'TRIGGER';
```

Un trigger se poate șterge din dicționar folosind comanda DDL :

```
DROP TRIGGER trigger_name;
```

Pentru a modifica starea unui trigger se folosească comanda DDL:

```
ALTER TRIGGER trigger_name {ENABLE | DISABLE}
```

bdd/laboratoare/06.txt · Last modified: 2024/10/07 14:39 by ciprian.truica

Laboratorul 07 - SQL Server : Interogari, Variabile si XML

Conținut

În acest laborator vom învăța numeroase noțiuni de SQL Server 2017 precum:

- Structura unei interogări
- Variabile
- Lucrul cu date de tip XML

Încercați să rulație fiecare instrucțiune din laborator chiar dacă nu vă este cerut în mod explicit acest lucru pentru a vă ajuta să înțelegeți.

Cerințe Preliminarii

1. Dorința de a învăța
2. Executarea scriptului pentru crearea tabelelor necesare
3. Noțiuni de bază despre SQL
4. Citirea scriptului de crearea a bazelor de date

Interogări

Ca și în orice alt limbaj de tip SQL avem instrucțiunea **SELECT** care are numeroase posibilități pentru a obține sau formata date. Ordinea clauzelor este foarte importantă și este următoarea:

```
[WITH] --CTE
SELECT
[INTO] -- Putem folosi rezultatul instrțiunii select pentru a popula o tabelă deja existentă
[FROM] -- Sursa sau sursele de date de date
[[[CROSS]|[INNER]|[LEFT|RIGHT|FULL OUTER]] JOIN] -- Putem folosi mai multe surse de date cu condiții speciale
[WHERE] -- Condițiile de filtrare a datelor deja existente
[GROUP BY] -- Agregarea datelor deja existente până în acest punct
[HAVING] -- Condiții pentru noile grupări
[ORDER BY] -- Ordonare, se pot folosi alias  * uri
```

Tot ce ați văzut între **[paranteze pătrate]** poate să lipsească din interogare și clauza să fie validă, spre exemplu:

```
SELECT  N'Aceasta este o clauză cât se poate de validă'
```

Ex. 1 Scrieți o clauză T-SQL care întoarce răspunsul la întrebarea: De ce am folosit litera **N** înainte șirul de caractere constant?

Comentarii

Ca în orice alt limbaj de programare avem dreptul de a folosi explicații pentru codul nostru. Se pot face în 2 moduri:

```
-- Primul
-- mod
-- Aici trebuie să avem caractele speciale '--' pe fiecare rând.

/*
Al
doilea
```

```
mod

Aici trebuie doar să deschidem comentariul înainte de primul rând
și să-l închidem după ultimul.
Nu este nevoie să avem vreun string special pe fiecare rând.
*/
```

Coloane, TOP și ALIAS

Putem să avem oricâte coloane în clauza noastră și va trebui să le declarăm prin una din următoarele 2 metode:

- Explicit: listă delimitată prin virgule
- Implicit pe toate, folosind operatorul *

Următoarele 2 clauze sunt echivalente (și generate de SQL Management Studio):

```sql
/****** Script for SelectTopNRows command from SSMS  ******/
SELECT TOP (1000) [employee_id]
      ,[first_name]
      ,[last_name]
      ,[email]
      ,[phone_number]
      ,[hire_date]
      ,[job_id]
      ,[salary]
      ,[commission_pct]
      ,[manager_id]
      ,[department_id]
  FROM [hr].[dbo].[employees]
```

```sql
SELECT TOP (1000) *
FROM [hr].[dbo].[employees]
```

Ne lovim de necunoscut, clauza **TOP**, care reduce numărul de linii returnate la **minimumul(numărul specificat, câte rezultate sunt într-adevăr)**. Aceasta nu are nevoie în mod deosebit de **(paranteze rotunde)** pentru a delimita numărul dar ne este mai ușor să ne dăm seama când avem de aface și cu constante.

După cum puteți observa avem o sintaxă clasică de SQL, până acum. Majoritatea bazelor de date relaționale folosesc același trunchi comun, SQL, duoă care fiecare adaugă "jucării noi".

Alias-urile sunt un mod de a oferi un nume unei coloane (câteodata este obigatoriu să avem unul, vom vedea asta). Dacă dorim să avem spațiu pentru un alias este nevoie să folosim (N)VARCHAR. Un ALIAS se poate folsi astfel:

```
SELECT  'Un Alias poate fi specificat în acest mod'  [Modul1]--Cu sau fără spațiu/Paranteze Drepte

,'Sau așa'  AS  'Modul 2'
```

Ex. 2 Scrieți o instrucțiune care întoarce 2 coloane, cu alias, și răspunde următoarei întrebări: De ce nu mai avem diacritice în clauza de mai sus?

Surse de date, WHERE și JOIN

Clauza **WHERE** este la fel ca în toate limbajele, filtrează date. Ce este important este că NU ține cont de ALIAS-uri și este foarte importană ordinea operatorilor booleani. În funcție de natura clauzei procesul din spate poate să difere:

- Datele sunt aduse și după filtrate
- Datele aduse sunt deja filtrate

În T-SQL avem mai multe tipuri de **JOIN** (+- cele teoretice care se reduc tot la acestea):

- **CROSS JOIN**, produs cartezian
- **INNER JOIN**, produs cartezian și condițiile
- **OUTER JOIN**, cu cele 3 variante **LEFT RIGHT** și **FULL**

Produsul cartezian se poate genera în următorul mod:

```
1 SELECT E.[first_name] + ' ' + E.[last_name] [NumeCompletAngajat]
2     ,M.[first_name] + ' ' + E.[last_name] [NumeCompletManager]
3 FROM [dbo].[employees] E
4     ,[dbo].[employees] M
```

Aici pentru fiecare angajat am adus fiecare manager, ne-am dori să aducem doar managerul lui așa că vom scrie un (SELF) **INNER JOIN** în 2 moduri:

```
1 SELECT E.[first_name] + ' ' + E.[last_name] [NumeCompletAngajat]
2     ,M.[first_name] + ' ' + E.[last_name] [NumeCompletManager]
3 FROM [dbo].[employees] E
4     ,[dbo].[employees] M
5 WHERE E.[manager_id] = M.[employee_id]
```

```sql
SELECT E.[first_name] + ' ' + E.[last_name] [NumeCompletAngajat]
    ,M.[first_name] + ' ' + E.[last_name] [NumeCompletManager]
FROM [dbo].[employees] E
JOIN [dbo].[employees] M ON E.[manager_id] = M.[employee_id]
```

După cum puteți observa, cuvântul **INNER** poate să lipsească cu desăvârșire. Același lucru se aplică și pentru clauza **OUTER**, fiind nevoie doar de **LEFT**, **RIGHT** sau **FULL**.

Ex. 3 Ne dăm seama că nu sunt aduși toți angajații, de ce? Cine lipsește? De ce lipsește? Scrieți o clauză care rezolvă acest lucru!

CTE

CTE provine de la Common Table Expression și este un mod de a ne aduce date pentru o clauză înainte de a se executa efectiv acea clauză. Putem aveam oricâte astfel de expresii, le separăm prin virgulă. Exemplu:

```sql
/*
Ne dăm seama că nu ne ajung informațiile precedente, mai dorim
și departementele pentru angajați și pentru manageri așa
că ne vom aduce și resul de date prin join, iar pentru angajați
vom folosi un CTE. Putem să-l folosim chiar pe același.
*/
WITH EMP_CTE
AS (
    SELECT E.[employee_id]
        ,E.[first_name] + ' ' + E.[last_name] [NumeCompletAngajat]
        ,DE.[department_name] AS [NumeDepartament]
        ,E.[manager_id]
    FROM [dbo].[employees] E
    INNER JOIN [dbo].[departments] DE ON E.[department_id] = DE.[department_id]
    )
SELECT E.[NumeCompletAngajat]
    ,E.[NumeDepartament] AS [NumeDepartamentAngajat]
    ,M.[NumeCompletAngajat] AS [NumeCompletManager]
    ,M.[NumeDepartament] AS [NumeDepartamentManager]
FROM EMP_CTE E
INNER JOIN EMP_CTE M ON E.[manager_id] = M.[employee_id]
```

Ex. 4 Aceeași problemă ca mai sus. Se rezolvă la fel? Rescrieți clauza, în mod corect și complet, fără CTE.

Subclauze

O subclauză este o clauză aflată în interiourl altei clauze și se poate folosi de valorile din clauzele superioare.. Nivelul 2 știe de Nivelul 1, Nivelul 3 știe de nivelele 1 și 2… Nivelul 32 știe de nivelele 1,2..31 și STOP.

32 este nivelul maxim, în cam tot ce presupune o oarecare recursivitate și T-SQL, nu doar pentru subclauze.

Ex. 5 Scrieți o clauză care întoarce numele angajaților care sunt manageri.

Grupări, Agregate și HAVING

Avem o mulțime de funcții de agregare predefinite pe care le găsiți aici [https://docs.microsoft.com/en-us/sql/t-sql/functions/aggregate-functions-transact-sql?view=sql-server-ver15]. Pe lângă aceste funcții mai sunt și cele fare funcționează DOAR când apare clauza **GROUP BY**.

Gruparea este la fel ca și în celelate limbaje de tip SQL: dispare noțiunea de individ și apre noțiunea de întreg. Pentru a filtra, în acest caz, se folosește **HAVING** care se referă la proprietățile unui grup. Se pot folosi funcții și în acest caz.

Exemplu:

```sql
/*
Pentru ficare departament care are un număr impar de angajați
ne dorim să aflăm următoarele:
- Numele departamentului
- Numărul de angajați din acel departament
- Numele angajaților din acel departament
*/
SELECT D.[department_name] NUME_DEP
    ,COUNT(D.[department_name]) NUMAR_ANG
    ,STRING_AGG(E.[first_name] + ' ' + E.[last_name], ' | ') NUME_ANG
FROM [departments] D
INNER JOIN [employees] E ON D.[department_id] = E.[department_id]
GROUP BY D.[department_name]
HAVING COUNT(D.[department_name]) % 2 = 1;
```

Ex. 6 Dorim să îmbunătățim clauza prin adăugarea unei coloane în care vom preciza numele managerilor din acel departament. (**HINT: SUBCLAUZE**)

ORDER BY

Această clauză sortează datele după obținerea lor. Clauza **TOP** ține cont de această clauză, de aceea veți observa diferențe de performanță neliniare când aceastea două sunt combinate.

Sortarea este fie:

- Ascendentă, modul implicit, **ASC**
- Descendentă, trebuie specificat explicit, **DESC**

Se poate sorta după mai multe criterii, lista fiind separată prin virgulă.

Acestă clauză ține cont de ALIAS, deoare ce este executată tocmai la final, și de aceea ne putem folosi de ele. De exemplu dacă dorim să sortăm rezultatele din exemplul precedent în mod descrescător după numărul de angajați vom adăuga **ORDER BY NUMAR_ANG DESC** la final. Se poate folosi și o condiție specială cum ar fi **ORDER BY COUNT(D.DNAME) DESC**, care în cazul nostru face fix același lucru.

UNION, EXCEPT, INTERSECT

Avem și posibilitatea de de a folosi operatori definiți special pentru mulțimi, având în vedere că rezultatul unei clauze este o mulțime de elemente. Singurul lucru care trebuie respectat este structura, adică:

- Fiecare coloană trebuie să fie de același tip
- Ordinea coloanelor este foarte importantă, trebuie să coincidă pentru a avea rezultatul așteptat
- Rezultatul operației va avea drep coloane (numele coloanelor) ce a avut primul tabel din operație

Exemplu:

```
-- Ordinea este vitală, de aici parantezele
(
  (
    SELECT 'Ana are Mere'
    UNION
    SELECT 'Dar are si Pere'
  )
  EXCEPT
  (
    SELECT 'Ana nu are Mere'
  )
)
INTERSECT
(
  SELECT 'Ana are Mere'
)
```

Ex. 7 Să genereze textul, folosind **UNION** de 3 ori:

```
Ana are Mere
Ana are Mere
Ana are MERE
Ana are MERE
Ana are Mere
Ana are Mere
```

Exercițiu final Clauze

Ex. 8. Scrieți o clauză care pentru ficare departament în care deviația standard a grilei salariale este între 1 și 1.5 obține următoarele: - Numele departamentului - Numele angajaților din acel departament - Numele managerului cu cel mai mare salariu din departament - Grila salarială minimă din departament - Grila salarială maximă din deprtament

Variabile

În T-SQL putem să declarăm variabile oriunde, nu doar în blocuri speciale, dar trebuie să ținem cont de **SCOPE**, ca în orice alt limbaj de programare.

```
-- Putem să declarăm variabile fără valoare
DECLARE @VAR_WITH_NO_VALUE NVARCHAR(MAX);
-- Sau care au deja valoare
DECLARE @VAR_WITH_VALUE NVARCHAR(MAX) = N'Se știe';

-- Se poate seta în mod explicit valoare unei variabile
SET @VAR_WITH_NO_VALUE = NULL;-- Atenție la tip

-- Se poate seta ca rezultat al unei clauze, oricare ar fi el
SELECT TOP 1 @VAR_WITH_VALUE = N'Se știe'
FROM [hr].[dbo].[EMPLOYEES];

-- Putem afișa valoare unei variabile
PRINT (@VAR_WITH_NO_VALUE);
PRINT (@VAR_WITH_VALUE);
```

XML

SQL Server ne permită să facem diverse operațiuni cu date de tip XML, în afară de stocarea lor în mod optim în tabele.

Din tabele în XML

După cum ați văzut în scriptul de creare, putem să generăm un XML prin sipla adăugare a clauzei **FOR XML AUTO** la finalul instrucțiuniilor din cladrul cererii **SELECT** (aici nu avem voie să folosim INTO). Rezultatul este un XML formatat în modul preferat de SQL Server, de aici provine **AUTO**.

Puteți să vă definiți și voi schema/structura pentru XML, dar pentru început este recomandat să o folosiți pe cea implicită.

Exemplu Creare XML din Tabele:

```
DECLARE @XML_STRING NVARCHAR(MAX);

SET @XML_STRING = '<ROOT> ' + (
        SELECT E.[employee_id]
            ,E.[first_name] + ' ' + E.[last_name] [EmployeeFullName]
            ,DE.[department_name] [EmployeeDepartment]
            ,M.[first_name] + ' ' + M.[last_name] [ManagerFullName]
            ,DM.[department_name] AS [ManagerDepartment]
        FROM [employees] E
        INNER JOIN [employees] M ON E.[manager_id] = M.[employee_id]
        INNER JOIN [departments] DE ON E.[department_id] = DE.[department_id]
        INNER JOIN [departments] DM ON M.[department_id] = DM.[department_id]
        WHERE E.[department_id] = M.[department_id]
        FOR XML AUTO
        ) + '</ROOT> ';

PRINT (@XML_STRING);
```

Întrebare 1 Ce face această clauză, dacă ignorăm partea de XML?

Interogare de XML pentru tabele

Putem să interogăm conținutul unui XML pentru a obține date tabelare. În exemplul de mai jos vom lua XML-ul generat anterior și ne vom folosi doar de primul nivel pentru a obține datele legate direct de angajat.

```sql
DECLARE @idoc INT
    ,@doc XML;

SET @doc = CONVERT(XML, @XML_STRING);

EXEC sp_xml_preparedocument @idoc OUTPUT
    ,@doc;

SELECT *
FROM OPENXML(@idoc, '/ROOT/E', 1) WITH (
        [employee_id] INT
        ,[EmployeeFullName] NVARCHAR(255)
        );
```

Din XML în tabele

Putem să revenim din XML înapoi la o tabelă (dacă ne permite structura) folosind operatorul **OUTER APPLY**, despre care puteți să citiți mai multe aici [https://docs.microsoft.com/en-us/sql/t-sql/queries/from-transact-sql?view=sql-server-ver15], dar el aplică anumite operațiuni pe TABELE, mai exact ce este în stânga lui și ce este în dreapta lui, la fel ca și **JOIN**. În exemplul de mai jos puteți să vedeți cum refacem datele tabelare transformate în XML.

```sql
DECLARE @idoc INT
    ,@doc XML;

SET @doc = CONVERT(XML, @XML_STRING);

EXEC sp_xml_preparedocument @idoc OUTPUT
    ,@doc;

SELECT E.X.value('@employee_id', 'varchar(max)') AS EMPLOYEE_ID
    ,E.X.value('@EmployeeFullName', 'varchar(max)') AS EMPLOYEE_NAME
    ,DE.X.value('@EmployeeDepartment', 'varchar(max)') AS EMPLOYEE_DEPARTMENT
    ,DE.X.value('@ManagerFullName', 'varchar(max)') AS MANAGER_NAME
    ,DM.X.value('@ManagerDepartment', 'varchar(max)') AS MANAGER_DEPARTMENT
FROM @doc.nodes('/ROOT/E') AS E(X)
OUTER APPLY E.X.nodes('DE') AS DE(X)
OUTER APPLY DE.X.nodes('DM') AS DM(X)
```

Întrebare 2 Ce credeți că returnează clauza dacă stricăm ordinea/nodurile?

bdd/laboratoare/07.txt · Last modified: 2024/10/08 01:44 by alex.petrescu

Laboratorul 08 - SQLServer: DML, Cursori si Programabilitate

Conținut

În acest laborator vom învăța numeroase noțiuni de SQL Server 2019 precum:

- DML și DDL
- Cursori
- SQL Dinamic
- Funcții, Proceduri Stocate și DML TRIGGER

Încercați să rulație fiecare instrucțiune din laborator chiar dacă nu vă este cerut în mod explicit acest lucru pentru a vă ajuta să înțelegeți.

Cerințe Preliminarii

1. Dorința de a învăța
2. Executarea scriptului pentru crearea tabelelor necesare
3. Noțiuni de bază despre SQL
4. Citirea scriptului de crearea a bazelor de date

Data Manipulation Language (DML) și Data Definition Language (DDL)

Precum majoritatea limbajelor de tip SQL, T-SQL oferă instrucțiuni precum:

- **INSERT** - DML
- **UPDATE** - DML
- **DELETE** - DML
- **TRUNCATE** - DDL
- **MERGE** - DML

Instrucțiunea INSERT

Poate fi folosită atât în modul clasic unde inserăm un set de valori definite de noi, cât și care rezultat al unei clauze. **INSERT INTO** va încerca să introducă datele într-o tabelă deja existentăm, în timp ce **SELECT ... INTO** va crea o nouă tabelă, ceea ce funcționează maxim o singură dată fără eroare.

```
-- EXEMPLU CLASIC
-- În ambele cazuri trebuie să avem grijă la constrângeri

-- Exemplu clasic, valorile sunt inserate în ordinea de creare a coloanelor
INSERT INTO [dbo].[employees] -- Dacă omitem coloanele, trebuie să respectăm tipul de date
VALUES (
        3 -- Valoare coloana 1
        ,'NEW' -- Valoare coloana 2
        ,'EMPLOYEE' -- Valoare coloana 3
        ,'newemployee@exhample.com' -- Valoare coloana 4
        ,'+40712345679' -- Valoare coloana 5
        ,'2020-11-22' -- Valoare coloana 6
        ,'FI_MGR' -- Valoare coloana 7
        ,15002 -- Valoare coloana 8
        ,NULL -- Valoare coloana 9
        ,NULL -- Valoare coloana 10
        ,100 -- Valoare coloana 11
        )
GO

-- Când specificăm numele coloanei, ordinea o alegem noi
INSERT INTO [dbo].[employees]
           ([employee_id]
           ,[first_name]
           ,[last_name]
           ,[email]
           ,[phone_number]
           ,[hire_date]
           ,[job_id]
           ,[salary]
           ,[commission_pct]
           ,[manager_id]
           ,[department_id]) -- Putem să precizăm una sau mai multe coloane
VALUES (
        2 -- Valoare coloana 1
        ,'NEW' -- Valoare coloana 2
        ,'EMPLOYEE' -- Valoare coloana 3
        ,'new.employee@exhample.com' -- Valoare coloana 4
        ,'+40712345678' -- Valoare coloana 5
        ,'2020-11-22' -- Valoare coloana 6
        ,'FI_MGR' -- Valoare coloana 7
        ,15000 -- Valoare coloana 8
        ,NULL -- Valoare coloana 9
        ,NULL -- Valoare coloana 10
```

```
        ,100 -- Valoare coloana 11
        )
GO
```

Ca să nu stricăm și mai tare structura bazei de date, ne vom clona tabela **[dbo].[EMPLOYEE]** folosing codul de mai jos.

```
-- Tabela clonă va avea coloane care au numele și tipul de date al rezultatului
-- Toate constrângerile și toți indecșii se pierd (nu se știe de existența lor)
SELECT *
INTO [dbo].[EMPLOYEE_CLONE]
FROM [dbo].[EMPLOYEES]
```

Dacă dorim să repopulăm tabela clonă cu datele noi din cea originală am putea folosi o instrucțiune de tipul celei de mai joi.

```
INSERT INTO [dbo].[EMPLOYEE_CLONE]
SELECT *
FROM [dbo].[EMPLOYEES]
```

Ex. 1 Corectați clauza astfel încât să insereze doare ce este nou, din punct de vedere al **[EMPLOYEE_ID]**.

Instrucțiunea UPDATE

Această instrucțiune modifică datele dintr-o tabelă deja existentă, respectând constrângerile. Acceptă și **JOIN**-uri pentru satisfacerea unor condiții mai complicate.

Ex. 2 Să se scrie o cerere care mărește salariul angajaților (din tabela **[dbo].[EMPLOYEE]**) cu 15% doar dacă sunt într-un departament ce conține un număr par de angajați .

Instrucțiunea DELETE

Ca și în alte limbaje, este o instrucțiune folosită pentru a șterge date dintr-o tabelă. Acceptă și **JOIN**-uri pentru satisfacerea unor condiții mai complicate. Este destul de important să avem condiție când folosim instrucțiunea **DELETE** pentru că altfel obținem un **TRUNCATE** suboptim care doar poate fi executat pe tabele ce nu au chei primare.

Ex. 3 Să se scrie o cerere care șterge angajații (din tabela **[dbo].[EMPLOYEE_CLONE]**) care au departamentul situat în "US".

Instrucțiunea TRUNCATE

Această instrucțiune este un **DELETE** total și mult mai rapid. Dezavantajul este că nu poate să șteargă tabele unde există chei primare.

```
TRUNCATE TABLE <NAME>
```

Ex. 4 Clonați tabela **[dbo].[DEPARTMENT]** doar pentru departamentele care au litera **E** în numele lor. Folosiți instrucțiunea **TRUNCATE** pe tabela rezultată.

Instrucțiunea MERGE

Este o instrucțiune foarte interesantă deoarece, în funcție de condiția de potrivire, în cazul în care există rânduri ce pot fi legate se poate face **UPDATE** pe acestea sau **DELETE**, iar în caz contrar se poate face **INSERT**.

Dorim să aducem datele noi în tabela clonă, **[dbo].[EMPLOYEE_CLONE]**, iar în cazul în care angajatul deja există îi da un super bonus salarial, mai exact salariul pe care îl are în tabela **[dbo].[EMPLOYEE]**.

```
MERGE INTO [dbo].[EMPLOYEE_CLONE] AS [Target  ]
USING [dbo].[employees] AS [Source]
        ON Target.[employee_id] = [Source].[employee_id]
WHEN MATCHED
        THEN
                UPDATE
                SET [Target].[salary] = [Source].[SALARY]
WHEN NOT MATCHED
        THEN
                INSERT
                VALUES ([employee_id]
           ,[first_name]
           ,[last_name]
           ,[email]
           ,[phone_number]
           ,[hire_date]
           ,[job_id]
           ,[salary]
           ,[commission_pct]
           ,[manager_id]
           ,[department_id]);
-- Din păcate este nevoie să precizăm toate coloanele/valorile pentru INSERT
```

Ex. 5 Dorim să aducem datele modificate înapoi în tabela **[dbo].[EMPLOYEE]** (doar salariul și bonusul), folosind **MERGE** clonă, din tabela **[dbo].[EMPLOYEE_CLONE]**, doar pentru angajații ce au (în tabela **[dbo].[EMPLOYEE]**) un [JOB] ce conține cel puțin un **A**, iar în caz contrar să nu facem nimic.

Cursori + SQL Dinamic

Cursorul este răspunsul la întrebarea "Există instrțiuni repeptitive în limbajul SQL?", el fiind un "WHILE". Cum ne aflăm într-o paradigmă centrată pe date, cursorii se folosesc niște tabele temporare stocate în memorie și, din motive de utilizare optimă a resurselor, este foarte important să închidem și dealocăm orice cursor pe care îl creăm.

SQL-ul dinamic este o metodă prin care putem să scriem ceva generic, folosindu-ne de variabile de tip **NVARCHAR**. Codul rezultat trebuie să respected toate regulile unui SQL clasic, dar avantajul este că poate fi scris în mod iterativ și folosind variabile. Un dezavantaj al acestei metode este performanța.

Exemplu: Ne dorim să facem curățenie în baza de date, adică să scăpăm de clone. Pentru asta vom scrie un SQL Dinamic, folosind un cursor, și vom aplica comanda **TRUNCATE** tuturor tabelelor clonă.

Ex. 6 Ne dăm seama că nu este suficient să ștergem tabelele, dorim și să scăpăm de ele. Scrieți o clauză care face asta. (folosiți **DROP**)

```
DECLARE @SQL NVARCHAR(500)

DECLARE @Cursor CURSOR

SET @Cursor = CURSOR FAST_FORWARD FOR SELECT  'DROP TABLE [' + TABLE_SCHEMA + '].[' +  TABLE_NAME + ']' FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_NAME LIKE '%CLONE'


OPEN @Cursor FETCH NEXT FROM @Cursor INTO @SQL

WHILE (@@FETCH_STATUS = 0)
BEGIN
print(@SQL)
EXEC sp_executesql @SQL
FETCH NEXT FROM @Cursor INTO @SQL
END

CLOSE @Cursor

DEALLOCATE @Cursor

GO
```

Funcții, Proceduri Stocate și DML TRIGGER

Ca și în alte limbaje de tip SQL, avem funcții și proceduri stocate. Funcțiile sunt READ-ONLY, nu au voie să modifice date, iar procedurile stocate au drepturi depline în ceea ce privește accesul la date. În T-SQL nu avem blocuri speciale pentru a declara variabile, așa că putem declara orice oriunde, singura restricție fiind vizibilitatea, ceea ce înseamnă că pentru o fucție sau o procedură vom încerca să declarăm mediul de lucrul la început.

În SQL Server, începând cu 2016, avem disponibilă sintaxa **CREATE OR ALTER** care oferă posibilitatea de a crea sau modifica, în caz că deja există, entitatea la care se face referință. Funcționează pentru:

- **Funcții**
- **Proceduri Stocate**
- **TRIGGER-e**
- **VIEW-uri**

Funcții

Funcțiile pot să primească sau nu parametrii care pot avea valori implicite (constante). Trebuie să precizăm tipul valorii returnate de funcție și este indicat să să folosim variablile și să delimităm instrucțiunile cu **BEGIN** și **END**.

Vom crea o funcție care care primește ca parametru ID-ul unui angajat și un salariu (parametru opțional care are valoarea default egală cu salariul angatului respectiv) și întoarce numărul angajaților subordonați lui, care au salariul mai mic sau egal cu salariul dat:

Ex. 7 Scrieți o funcție care întoarce salariul unui angajat și modificați funcția de mai sus.

Proceduri stocate

Procedurile stocate sunt un mod optim de a păstra un cod care va fi executat de mai multe ori, acestea fiind și optimizate în spate, în funcție de planul lor de execuție. Procedurile stocate conferă și proprietatea de "black-box" deoarece putem să oferim drepturi doar de execuție pentru a anonimiza baza noastră de date și de oferi acces granular.

Pentru proceduri stocate avem ca unele auxiliare blocuri de tipul **BEGIN TRY \ END TRY** și **BEGIN CATCH \ END CATCH** care ne vor proteja de excepții la care ne așteptăm în timpul execuției, ca de exemplu existența unei tabele care este șterasă destul de frecvent.

Vom crea o procedură numită **[dbo].[DEPT_MGR_INFO]** care are ca input id-ul unui departament și ca output numele departamentului și inserează într-o tabelă istorică urmatoarele:

- ID-ul fiecărui manager care lucrează în acel departament (manager ⇒ are cel

puțin un subaltern)

- Numele managerului
- Numărul angajaților subordonați lui, care au salariul mai mic sau egal ca el

(obținut prin intermediul funcției **[dbo].[GET_SUBORDINATES]**)

- Data executiei

Ex. 8 De ce credeți că este nevoie de SQL Dinamic?

DML TRIGGER

Sunt o metodă de a executa anumite instrucțiuni în momentul în care este accesată o tabelă cu una din următarele instrucțiuni **DML**:

- **INSERT**, iar datele noi pot fi referite cu **INSERTED**
- **DELETE**, iar datele șterse pot fi referite cu **DELETED**
- **UPDATE**, iar datele noi pot fi referite cu **INSERTED** și cele șterse pot fi referite cu **DELETED**

SQL Server permite două tipuri de **TRIGGER**:

- **AFTER**, se execută după operația propriu-zisă și schimbările devin permanente doar dacă ambele au reușit
- **INSTEAD OF**, se execută în locul operației

Dorim să ne folosim de procedura creată anterior așa că o vom atașa de tabela **[dbo].[EMPLOYEE]**.

```
CREATE TRIGGER [dbo].[INSERARE_ISTORIC_E] ON [dbo].[EMPLOYEE]
AFTER INSERT, UPDATE, DELETE
AS
EXEC [DEPT_MGR_INFO];
GO
```

Ex. 9 Are sens să o mai atașăm și de altă tabelă?

bdd/laboratoare/08.txt · Last modified: 2024/10/07 11:27 (external edit)

Laboratorul 09 - Conectare la o Baza de Date

Conectare OracleDB din Python

Configurare mediu de lucru

Windows

1. Descărcați și instalați managerul de pachete Miniconda pentru Python 3 de aici [https://docs.conda.io/en/latest/miniconda.html].
2. Deschideți programul Anaconda Prompt.
3. Instalați pachetul cx_Oracle.

```
conda install -c anaconda cx_oracle
```

4. Descărcați Oracle Instant Client, urmând instrucțiunile de aici [https://oracle.github.io/odpi/doc/installation.html#windows].

Linux

1. Instalați Python.
2. Instalați pachetul cx_Oracle.

```
pip install cx_Oracle
```

3. Descărcați Oracle Instant Client, urmând instrucțiunile de aici [https://oracle.github.io/odpi/doc/installation.html#linux].

Pachet Oracle

Vom crea un pachet care expune o funcție și o procedură. Funcția va primi ca parametru id-ul unui job și va returna salariul mediu pentru acel job, expunând ca parametri de tip OUT salariul minim și salariul maxim pentru jobul respectiv. Procedura va primi ca parametru un id de departament și va expune ca parametri de tip OUT numele departamentului și un cursor ce va întoarce angajații acelui departament.

```
CREATE OR REPLACE package pck_example
AS
    PROCEDURE exemplu_procedura(idDept IN departments.department_id%TYPE,
        numeDept OUT departments.department_name%TYPE, angajati OUT sys_refcursor);
    FUNCTION exemplu_functie(jobId IN jobs.job_id%TYPE,
        salLow OUT jobs.min_salary%TYPE, salHigh OUT jobs.max_salary%TYPE) RETURN NUMBER;
END pck_example;
/

CREATE OR REPLACE package body pck_example
AS
    FUNCTION exemplu_functie(jobId IN jobs.job_id%TYPE,
        salLow OUT jobs.min_salary%TYPE, salHigh OUT jobs.max_salary%TYPE) RETURN NUMBER
    IS
        avgVenit NUMBER;
    BEGIN
        SELECT DISTINCT min_salary, max_salary INTO salLow, salHigh FROM jobs WHERE job_id = jobId;
        SELECT round(avg(salary * (1 + nvl(commission_pct, 0))), 2) INTO avgVenit
            FROM employees WHERE job_id = jobId;
        RETURN avgVenit;
    END exemplu_functie;

    PROCEDURE exemplu_procedura(idDept IN departments.department_id%TYPE,
        numeDept OUT departments.department_name%TYPE, angajati OUT sys_refcursor)
    IS
    BEGIN
        SELECT department_name INTO numeDept FROM departments WHERE department_id = idDept;
        OPEN angajati FOR
            SELECT e.first_name || ' ' || e.last_name, e.hire_date, j.job_title, e.salary * (1 + nvl(e.commission_pct, 0)) AS venit
            FROM employees e INNER JOIN jobs j ON e.job_id = j.job_id WHERE e.department_id = idDept;
    END exemplu_procedura;
END pck_example;
/
```

Scriptul poate fi descărcat de aici.

Conexiune din Python

Vom crea o clasă **OracleConnection** pentru a ne conecta la baza de date Oracle, salvând în constructor pe câmpuri ale clasei adresa bazei de date, numele utilizatorului și parola lui:

```
import cx_Oracle

class OracleConnection:

    def __init__(self, host, port, schema, username, password):
        self.host = host
        self.port = port
        self.schema = schema
        self.username = username
        self.password = password
```

Această clasă va expune o metodă pentru inițializarea conexiunii către baza de date Oracle și o metoda pentru deconectare:

```
    def openConnection(self):
        try:
            dsn_tns = cx_Oracle.makedsn(host=self.host, port=self.port, service_name=self.schema)
```

```
            self.db = cx_Oracle.connect(self.username, self.password, dsn_tns)
            self.cursor = self.db.cursor()
            print("Connection open!")
        except Exception as e:
            print("Connection not open!")
            print(e)

    def closeConnection(self):
        try:
            self.cursor.close()
            self.db.close()
            print("Connection close!")
        except Exception as e:
            print("Connection not closed!")
            print(e)
```

Urmează implementarea interacțiunii cu procedura și functia din pachetul definit mai sus. Pentru aceasta vom expune o metodă care primește ca parametru un id de departament pentru a apela procedura și o metodă care primește ca parametru id-ul unui job pentru a apela funcția:

```
    def exempluProcedura(self, idDept):
        try:
            numeDept = self.cursor.var(cx_Oracle.STRING)
            angajati = self.cursor.var(cx_Oracle.CURSOR)
            self.cursor.callproc("pck_example.exemplu_procedura", [idDept, numeDept, angajati])
            for elem in angajati.getvalue():
                print(elem[0], elem[1].strftime("%Y-%m-%d"), elem[2], elem[3], numeDept.getvalue())
        except Exception as e:
            print(e)

    def exempluFunctie(self, jobId):
        try:
            salLow = self.cursor.var(cx_Oracle.NUMBER)
            salHigh = self.cursor.var(cx_Oracle.NUMBER)
            avgVenit = self.cursor.callfunc("pck_example.exemplu_functie", cx_Oracle.NUMBER, [jobId, salLow, salHigh])
            print(jobId, avgVenit, salLow.getvalue(), salHigh.getvalue())
        except Exception as e:
            print(e)
```

Putem observa că valorile parametrilor întorși de procedură și de funcție pot fi accesate apelând metoda **.getvalue()**, iar pentru a itera peste un cursor este de ajuns sa iterăm peste valoarea parametrului de tip cursor, folosind o buclă **for**.

Vom adăuga și comportamentul de funcție **main** pentru script, unde vom deschide o conexiune către baza de date, apelând procedura cu id-ul de departament 10 și funcția cu id-ul de job 'SA_REP', la final închizând conexiunea:

```
if __name__ == "__main__":
    oc = OracleConnection('localhost', 1521, 'XEPDB1', 'system', 'parolaAiaPuternic4!')
    oc.openConnection()
    oc.exempluProcedura(10)
    oc.exempluFunctie('SA_REP')
    oc.closeConnection()
```

Dacă rulăm scriptul obținem:

```
Connection open!
Jennifer Whalen 2003-09-17 Administration Assistant 4400 Administration
SA_REP 10184.67 6000.0 12008.0
Connection close!
```

Scriptul complet poate fi găsit aici.

Conectare MSSQL din Python

Configurare mediu de lucru

Configurati mediul de lucru urmarind pasii de aici [https://learn.microsoft.com/en-us/sql/connect/python/pyodbc/step-1-configure-development-environment-for-pyodbc-python-development?view=sql-server-ver16] in functie de platforma pe care va aflati.

Subprograme MSSQL

Pentru a testa conexiunea din Python la MSSQL vom folosi doua subprograme, o procedura si o functie.

Procedura va intoarce rezultatul unei interogari SELECT care preia numele complet, data angajarii, titlul jobului si venitul angajatilor din departamentul cu id-ul primit ca parametru.

Functia calculeaza numarul de subordonati ai unui manager cu id-ul primit ca parametru, care au salariul mai mic decat o valoare data ca parametru, care are valoarea default salariul managerului.

```
CREATE OR ALTER FUNCTION dbo.GET_SUBORDINATES_WITH_LOWER_SALARY
(@ManagerId INT, @Salary REAL = NULL)
RETURNS INT AS
BEGIN
    IF @Salary IS NULL
        SELECT @Salary = salary
        FROM employees
        WHERE employee_id = @ManagerId;

    DECLARE @result INT;

    SELECT @result = COUNT(*)
    FROM employees EMP
    LEFT OUTER JOIN employees MGR ON EMP.manager_id = MGR.employee_id
    WHERE EMP.manager_id = @ManagerId AND EMP.salary <= @Salary;

    RETURN @result;
END
GO

CREATE OR ALTER PROCEDURE dbo.exemplu_procedura
(@idDept INT)
AS
```

```
    SELECT e.first_name + ' ' + e.last_name AS full_name, e.hire_date, j.job_title, e.salary * (1 + isnull(e.commission_pct, 0)) AS venit
    FROM employees e INNER JOIN jobs j ON e.job_id = j.job_id WHERE e.department_id = @idDept;
RETURN 0
GO
```

Conexiune din Python

Pentru conexiunea din python vom crea o clasa MSSQLConnection cu urmatorul constructor:

```
import pyodbc

class MSSQLConnection:
    def __init__(self, host, port, database, username, password):
        self.host = host
        self.port = port
        self.database = database
        self.username = username
        self.password = password
```

Vom defini apoi doua functii, **open_connection** si **close_connection** care vor deschide, respectiv inchide, conexiunea la baza de date.

```
    def openConnection(self):
        try:
            self.db = pyodbc.connect('DRIVER={ODBC Driver 18 for SQL Server};SERVER='+self.host+';DATABASE='+self.database+';ENCRYPT=yes;TrustServerCertificate=yes;UID='+self.username+';P
            self.cursor = self.db.cursor()
            print("Connection open!")
        except Exception as e:
            print("Connection not open!")
            print(e)

    def closeConnection(self):
        try:
            self.cursor.close()
            self.db.close()
            print("Connection close!")
        except Exception as e:
            print("Connection not closed!")
            print(e)
```

Acum mai ramane de adaugat metode care sa apeleze subprogramele definite in baza de date mai sus:

```
    def exempluProcedura(self, idDept):
        try:
            cmd = "EXEC dbo.exemplu_procedura ?;"
            values = (idDept)
            self.cursor.execute(cmd, values)
            for elem in self.cursor.fetchall():
                print(elem)
        except Exception as e:
            print(e)

    def exempluFunctie(self, manager_id, salary=None):
        try:
            cmd = """\
            SET NOCOUNT ON;
            DECLARE @RC int;
            EXEC @RC = dbo.GET_SUBORDINATES_WITH_LOWER_SALARY ?, ?;
            SELECT @RC AS rc;
            """
            values = (str(manager_id), (str(salary or "NULL"))
            self.cursor.execute(cmd, values)
            emp_count = self.cursor.fetchval()
            print('Managerul cu id-ul ' + str(manager_id) + ' are ' + str(emp_count) + ' angajati cu salariul mai mic decat ' + (str(salary) or 'el'))
        except Exception as e:
            print(e)
```

Aici sunt cateva lucruri de observat. Accesul la date in afara contextului de T-SQL necesita o instructiune de tip SELECT la final. Astfel, in cazul procedurii, care facea un singur SELECT, este de ajuns sa apelam procedura folosind **exec numele_procedurii lista_parametri**. Pentru functie, in schimb, va trebui sa rulam un bloc de instructiuni T-SQL care se termina in **SELECT rezultat_apel_functie**.

In ambele cazuri, cursorul va contine acum valorile dorite. Pentru a accesa valorile multiple intoarse de procedura putem folosi **cursor.fetchall()**, iar pentru functie putem folosi **cursor.fetchval()**.

Vom si testa clasa nou creata cu o functie main:

```
if __name__ == "__main__":
    oc = MSSQLConnection('localhost', 1433, 'master', 'sa', 'parolaAiaPuternic4!')
    oc.openConnection()
    oc.exempluProcedura(10)
    oc.exempluFunctie(100, 20000)
    oc.closeConnection()
```

Dupa rulare putem vedea urmatorul output:

```
Connection open!
('Jennifer Whalen', datetime.date(2003, 9, 17), 'Administration Assistant', 4400.0)
Managerul cu id-ul 100 are 14 angajati cu salariul mai mic decat 20000
Connection close!
```

Conectare MongoDB din Python

Configurare mediu de lucru

Pentru a va conecta la MongoDB din python aveți nevoie de pachetul `pymongo`. Acesta se poate instala fie cu `pip` fie folosinf `conda`:

- `pip install pymongo` sau
- `conda install pymongo`

Clasa de conectare

Pentru a vă conecta la baza de date, aveți nevoie de urmatorele informații:

- `hostname` numele sau adreasa de IP pe care se afla serverul de MongoDB
- `port` portul pe care asculta MongoDB
- `dbname` numele bazei de date la care doriti sa va conectatii

Toate aceste informații se găsesc în constructorul classei `MongoDBConnector`.

Pentru a vă conecta la instanța de mongo, folosiți `client = pymongo.MongoClient(host, port)`. Pentru a vă baza de date, folosit `client[dbname]`. Aceste sunt implementate în metoda `openConnection(…)`.

Pentru a interoga baza de date, se pot folosi funcțiile `find_one()` sau `find()`. Pentru a ușura munca, vom defini două metode: `getRecords(…)` și `getRecord(…)`. Parametrul obligatoriu primit de aceste metode este `collection` care specifică colecția pe care să se facă căutare. Aceste metode primesc parametrii opționali:

- `filter` - pentru a pune condiții și a filtra înregistrările întoarse
- `projection` - pentru a spune ce câmpuri să se întoarcă.

Pentru a folosi Aggregation Pipeline, este definită metoda `execAggreation(…)` care primește doi parametrii:

- `collection` - coleția pe care să se facă agregarea
- `pipeline` - operațiile de agregare sub forma unei liste de dicționare.

Metoda `execMapReduce(…)` este folosită pentur a trimite către server intrucțiuni de tip MapReduce. primește următorii parametrii:

- `collection` - colecția pe care să se execute MapReduce
- `mapper` - funcția pentru Map
- `reducer` - funcția pentru Reduce
- `out` - colecția unde să se salveze rezultatul
- `query` (opțional) - cerere de filtrare a întregistrărilor înainte de să facă MapReduce.

```
import pymongo

class MongoDBConnector:
    def __init__(self, dbname, host='localhost', port=27017):
        self.host = host
        self.port = port
        self.dbname = dbname

    def openConnection(self):
        self.client = pymongo.MongoClient(host=self.host, port=self.port)
        self.db = self.client[self.dbname]

    def closeConection(self):
        self.client.close()

    def getRecords(self, collection, filter=None, projection=None):
        if projection is None and not filter is None:
            return self.db[collection].find(filter=filter)
        elif not projection is None and filter is None:
            return self.db[collection].find(projection=projection)
        elif projection is None and filter is None:
            return self.db[collection].find()
        return self.db[collection].find(filter=filter, projection=projection)

    def getRecord(self, collection, filter=None, projection=None):
        if projection is None and not filter is None:
            return self.db[collection].find_one(filter=filter)
        elif not projection is None and filter is None:
            return self.db[collection].find_one(projection=projection)
        elif projection is None and filter is None:
            return self.db[collection].find_one()
        return self.db[collection].find_one(filter=filter, projection=projection)

    def execAggreation(self, collection, pipeline):
        return self.db[collection].aggregate(pipeline=pipeline)

    def execMapReduce(self, collection, mapper, reducer, out, query=None):
        self.db[out].drop()
        if query is None:
            return self.db[collection].map_reduce(mapper, reducer, out)
        return self.db[collection].map_reduce(mapper, reducer, out, query=query)
```

Utilizare

Pentru următoarele exemple să se importe documentele din fișierul bd2_mongo care conține baza de date în format JSON. Baza de date se va numi **BD** și colecția se va numi **documents**. Folosiți mongoimport pentru a popula baza de date:

```
mongoimport --db=BD --collection=documents --type=json --file=bd2_mongo.json
```

După ce creați baza de date, puneți următorii indecși pentur a funcționea exemplele

```
db.documents.createIndex({author: 1})
db.documents.createIndex({gender: 1, age: 1})
db.documents.createIndex({"words.word": 1})
db.documents.createIndex({"geoLocation": "2d"})
db.documents.createIndex({"lemmaText": "text"})
db.documents.createIndex({"date": "hashed"})
```

Se folosesc host-ul și portul default al bazei de date MongoDB. Nu uitați să imporați clasa `MongoDBConnector` sau să o puneți în același fișier

Interogarea bazei de date

Următorul exeplu folosește folosește indexul de tip `text` pentru a găsi un document care are cuvântul "coffee" în interiorul șirului de caractere din câmpul `lemmaText`.

```
from bson.code import Code

if __name__ == '__main__':
    dbname = "BD"
    db = MongoDBConnector(dbname=dbname)
    collection = "documents"

    filter = { "$text":
        {
            "$search": "coffee",
            "$language": "english",
            "$caseSensitive": False,
            "$diacriticSensitive": False
        }
    }
    projection = { "lemmaText": 1 }
    db.openConnection()
    print("Test 1")
    doc = db.getRecord(collection, filter, projection)
    print(doc)
    print("Test 2")
    doc = db.getRecord(collection, filter=filter)
    print(doc)
    print("Test 3")
    doc = db.getRecord(collection, projection=projection)
    print(doc)
    print("Test 4")
    doc = db.getRecord(collection)
    print(doc)
    db.closeConection()
```

Următorul exeplu găsește toate documentele care se află în aproierea coordonatelor [25,25]. Se folosește indexul de tip `geoLocation` pentru a folosi operatorul `$near`.

```
from bson.code import Code

if __name__ == '__main__':
    dbname = "BD"
    db = MongoDBConnector(dbname=dbname)
    collection = "documents"

    filter = { "geoLocation": { "$near": [25, 25] } }
    projection = {"geoLocation": 1, "lemmaText": 1 }
    db.openConnection()
    docs = db.getRecords(collection, filter, projection)
    for doc in docs:
        print(doc)
    db.closeConection()
```

Următorul exemplu folosește Aggregation Pipeline pentru a număra de câte ori apar un cuvânt în documentele care au ca autor un bărbat. Pipeline-ul este format din următoarele câmpuri:

- `$match` - este folosit pentru filtrare
- `$project` - folosit pentru proiecție. La cest pas se extrac cuvintele din câmpul `$lemmaText`. Cuvintele sunt salvate în vectorul `words`
- `$unwind` - este folosit pentru a despacheta vectorul `words` și a lua fiecare element din el care este folosit mai departe în `$group`
- `$group` - este folosti pentru a asigna ca "_id" un cuvânt din `$words` și a calcula numărul de apariții ale unui cuvânt `"counts": { "$sum": 1 }`

Filtrarea se face folosind `q`.

```
from bson.code import Code

if __name__ == '__main__':
    dbname = "BD"
    db = MongoDBConnector(dbname=dbname)
    collection = "documents"

    q = {"gender": "male"}
    pipeline = [
                    { "$match": q },
                    { "$project": { "words": { "$split": ["$lemmaText", " "]}}},
                    { "$unwind": "$words" },
                    { "$group": { "_id": "$words", "counts": { "$sum": 1 } } }
        ]

    db.openConnection()
    docs = db.execAggreation(collection, pipeline)
    for doc in docs:
        print(doc)
    db.closeConection()
```

Următorul exemplu folosește MapReduce pentru a număra de câte ori apar un cuvânt în documentele care au ca autor o femeie. Trebuie să scriem funcțiile map și reduce în limbnajul JavaScript. Funcția `mapper` ia textul care se găsește în câmpul `lemmaText` și extrage cuvintele. Apoi, pentru fiecare cuvânt se "emite' o pereche cheie-valoare care cuprinde cuvântul (cheia) și numărul de apariții initializat la 1 (valoare). Funcția `reducer` ia fiecare set cuvânt (`key`) și lista cu numărul de apariții (`values`) și însumează valorile. Rezultatele sunt salvate în colecția `out_collection = "wordCounts"`. Filtrarea se face folosind `q`.

```
from bson.code import Code

if __name__ == '__main__':
    dbname = "BD"
    db = MongoDBConnector(dbname=dbname)
    collection = "documents"

    mapper = Code("""
        function() {
```

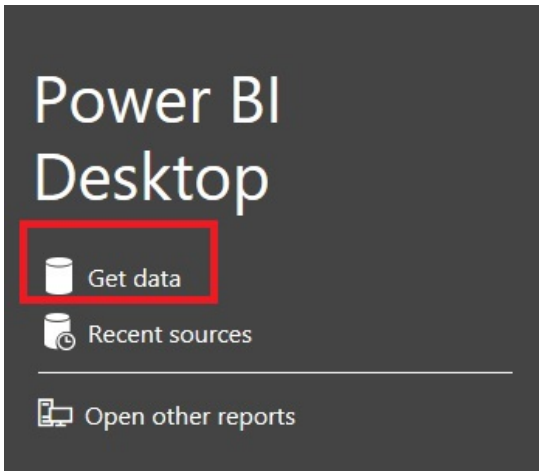
```
            var tokens = this.lemmaText.split(" ");
            for (var idx=0; idx<tokens.length; idx++){
                emit(tokens[idx], 1);
            }
        }
        """)

    reducer = Code("""
        function(key, values) {
            return Array.sum(values);
        }
        """)

    q = {"gender": "female"}
    out_collection = "wordCounts"
    db.openConnection()
    docs = db.execMapReduce(collection, mapper, reducer, out_collection, q)
    for doc in docs.find():
        print(doc)
    db.closeConection()
```

Codul se găsește main.py. Nu uitați să schimbați extensia.

PowerBI - Windows

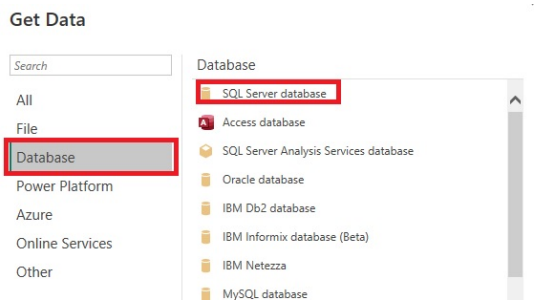
Pentru a descarca Microsoft PowerBI, pe Windows, accesati acest link [https://www.microsoft.com/en-us/download/details.aspx?id=58494]. Dupa ce ati selectat versiunea pentru arhitectura voastra, vom deschide aplicatia si o vom conecta la bazele noastre de date (Oracle si Microsoft din Docker, dar principiul este acelasi si pentru unele hostate complet).

In toate scenariile, dupa deschiderea aplicatiei vom apasa pe butonul “Get Data”.



Conectarea la BD Microsoft

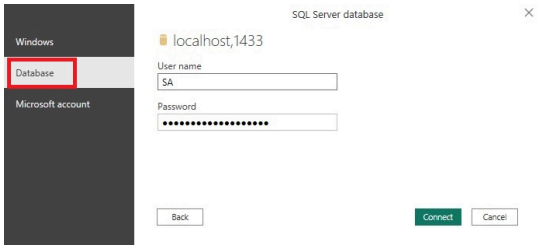
Pentru a selecta SQL Server ca baza de date vom selecta meniul **Database** > **SQL Server Database** > **Connect**.



Dupa care vom introduce datele de conectare. Daca folosim exact datele din laborator vom folosi **localhost,1433** (port-ul in PBI este precizat cu virgula, nu cu:).

Vom selecta modul Import.

Dupa ce apasam **OK** va trebui sa ne autentificam. Daca folosim exact datele din laborator vom folosi **SA@parolaAiaPuternic4!** la Database Credentials.



Conectarea la BD Oralce

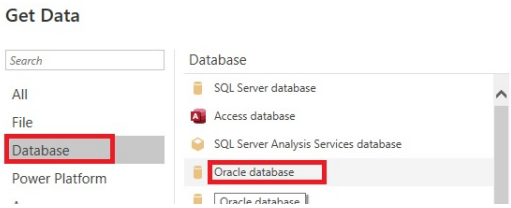
Inainte de a conecta PBI la o baza de date Oracle, trebuie sa instalam clientul. Detaliile complete se gasesc aici [https://learn.microsoft.com/en-us/power-bi/connect-data/desktop-connect-oracle-database#installing-the-oracle-client].

Setup - TLDR

1. Descarcam ODAC x64 [https://www.oracle.com/database/technologies/odac-downloads.html] sau x32 [https://www.oracle.com/database/technologies/dotnet-utilsoft-downloads.html] (nu cel cu copy_, cel mare)
2. Rulam Setup-ul (avem grija sa fie bifat **Configure ODP.NET nad/or Oracle Providers for ASP.NET at machine-wide level** in sectiunea de **ODP.NET**)

Conectare PBI

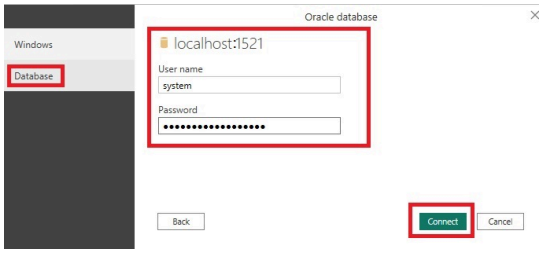
Pentru a selecta Oracle ca baza de date vom selecta meniul **Database** > **Oracle Database** > **Connect**.



Dupa care vom introduce datele de conectare. Daca folosim exact datele din laborator vom folosi **localhost:1521**.

Vom selecta modul Import.

Dupa ce apasam **OK** va trebui sa ne autentificam. Daca folosim exact datele din laborator vom folosi **system@parolaAiaPuternic4** la Database Credentials.



Link-uri Utile

- Get started with Power BI Desktop [https://learn.microsoft.com/en-us/power-bi/fundamentals/desktop-getting-started]
- Sample App With Microsoft Tech [https://blog.alexpetrescu.net/2022/01/20/building-my-cool-site-the-microsoft-way/]

Laboratorul 10 - MongoDB I

Obiective

- Comenzi MongoDB
- Operații DML

Linkuri utile

- MongoDB: https://www.mongodb.com/try/download/community [https://www.mongodb.com/try/download/community]
- Mongo database tools: https://www.mongodb.com/try/download/database-tools [https://www.mongodb.com/try/download/database-tools]
- MongoDB in Docker:

```
docker pull mongo
docker run -d -p 27017-27019:27017-27019 --name mongodb mongo:latest
```

Path pentru MongoDB pe Windows - Masinile din Laborator

Calea unde se găsesc utilitarele: `C:\Program Files\mongodb*\bin`, restul comenzilor sunt ca în laborator.

Path pentru MongoDB pe Windows

Dacă ați instalat tool-urile de mai sus, utilitarele se găsesc la calea `C:\Program Files\MongoDB\Tools\100\bin`, puteți să o includeți ca să lucrați mai ușor cu anumite comenzi.

Comenzi MongoDB

Pornire MongoDB

Pentru a porni baza de date MongoDB se va folosi demonul **mongod**(mongod [https://docs.mongodb.com/manual/reference/program/mongod/]).

Procesul demon **mongod** poate porni cu următorii paramterii:

- **--config <filename>** specifică un fisier de configurare
- **--port <port>** pentru a specifica portul pe care asculta procesul demon, implicit 27017
- **--bind_ip <hostnames|ipaddresses|Unix domain socket paths>** pentru a specifica addresa de
- **--dbpath <path>** calea către directorul unde se vor stoca fișierele de stocare.

Exemplu utilizare **mongodb**

```
mongod --port 27018 --bind_ip 192.100.10.10 --dbpath /data
```

Ex. 1. - LAB Să creeze un director pe discul `Student` numit `mongo_data`. Să se pornească baza de date cu folosind ca director de stocare noul director creat. Restul valorilor rămân cele implicite.

Ex. 1. - Docker Intrați în container folosind comanda: `docker exec -it mongodb bash`.

Conectare MongoDB

Pentru a va conecta la un server MongoDB, se folosește interpretorul în linie de comandă **mongo** (Documentație [https://docs.mongodb.com/manual/mongo/]).

Exemplu de utilizare:

```
# se conectează implicit la localhost pe portul 27017
mongo

# se conectează la hostname folosind portul implicit 217017
mongo --host hostname

# se conectează implicit la localhost folosind portul port
mongo --port 27017

# se conectează la hostname folosind portul port
mongo --host hostname:27017

mongo --host hostname --port 27017
```

Ex. 2. Conectați-vă la MongoDB.

Comenzi CLI MongoDB

Pentru a vedea baza de date curentă se folosește comanda **db**.

Pentru a vedea toate bazele de date existente se folosește comanda **show dbs**.

Pentru a vă conecta la o baza de date sau a crea o nouă bază de date se folosește comanda **use DATABASE_NAME**. În cazul în care se folosește comanda **use** pentru a crea o bază de date, aceasta va deveni persistentă numai dacă se crează o colecție sau se inserează o înregistrare.

Pentru a vedea toate colecțiile se folosește comanda **show collections**.

Puteți folosi una din următoarele metode pentru a crea o nouă colecție:

- **db.createCollection("COLLECTION_NAME")**, unde **COLLECTION_NAME** este numele noii colecții
- inserați o înregistrare și colecția se va crea automat.

Pentru a șterge o colecție se folosește comanda **db.COLLECTION_NAME.drop()**.

Pentru a șterge o baza de date se folosește comanda **db.dropDatabase()**. Trebuie să fiți conectați la baza de date (folosing **use**) pentru a o șterge.

Numele bazelor de date este case sensitive.

Ex. 3. Creați o bază de date numită **faculty** care să conțină colecția **students**.

Operații DML

Inserarea datelor

Pentru a adăuga date în MongoDB puteți sa folosiți una din metodele:

- Insert sau Bulk Insert (Documentație [https://docs.mongodb.com/manual/reference/method/db.collection.insert/])
- **mongoimport** (Documentație [https://docs.mongodb.com/manual/reference/program/mongoimport/])

insert()

Comanda Insert are următoare sintaxa

```
db.collection.insert(
   <document or array of documents>,
   {
     writeConcern: <document>,
     ordered: <boolean>
   }
)
```

Un document dintr-o colecție este în format BSON:

```
{
    "student":
        {
            "firstname": "Ion",
            "lastname": "Popescu"
        },
    "an": 4,
    "grupa": "342C3",
    "materii":
        [
            {"nume": "BD2",  "an": 4},
            {"nume": "Comp", "an": 4},
            {"nume": "SO2",  "an": 4}
        ],
    "cunostinte": ["SQL", "Java", "PL/SQL"]
}
```

Pentru a insera documentul în colecția students:

```
db.students.insert(
{
    "student":
        {
            "firstname": "Ion",
            "lastname": "Popescu"
        },
    "an": 4,
    "grupa": "342C3",
    "materii":
        [
            {"nume": "Comp", "an": 4},
            {"nume": "BD2" , "an": 4},
            {"nume": "SO2" , "an": 4}
        ],
    "cunostinte": ["SQL", "Java", "PL/SQL"]
}
)
```

Pentru a va ușura munca puteți sa va creați variabile la nivelul consolei. Dacă se dorește inserarea mai multor înregistrari, se poate declara un vector.

```
var stud = {"student": { "firstname": "Andrei", "lastname": "Ionescu"}, "an": 4, "grupa": "341C4", "materii": [{"nume": "BD2", "an": 4}, {"nume": "IA", "an": 4},{"nume": "IAUT", "an": 4}]

db.students.insert(stud)

var studs = [
    {"student": { "firstname": "George", "lastname": "Popescu"}, "an": 4, "grupa": "341C2", "materii": [{"nume": "BD2", "an": 4}, {"nume": "IOCLA", "an": 2}], "cunostinte": ["Python", "SQ
    {"student": { "firstname": "Georgiana", "lastname": "Petrescu"}, "an": 4, "grupa": "341C2", "materii": [{"nume": "BD2", "an": 4}, {"nume": "SCAD", "an": 4}], "cunostinte": ["Python",
    {"student": { "firstname": "Valentina", "lastname": "Vasilescu"}, "an": 3, "grupa": "331CA", "materii": [{"nume": "BD", "an": 3}, {"nume": "RL", "an": 3}, {"nume": "APD", "an": 2}], "
    {"student": { "firstname": "Grigore", "lastname": "Ionescu"}, "an": 4, "grupa": "342C2", "materii": [{"nume": "BD2", "an": 4}, {"nume": "VLSI", "an": 4}, {"nume": "SRIC", "an": 4}, {"
    {"student": { "firstname": "Andrei", "lastname": "Popescu"}, "an": 3, "grupa": "332CA", "materii": [{"nume": "CN1", "an": 2}, {"nume": "CN2", "an": 2}, {"nume": "RL", "an": 3}], "cuno
    {"student": { "firstname": "Ana", "lastname": "Georgescu"}, "an": 4, "grupa": "342C5", "materii": [{"nume": "BD2", "an": 4}, {"nume": "UBD", "an": 4}, {"nume": "SRIC", "an": 4}, {"num
]

db.students.insert(studs)
```

Ex. 4. Inserați în baza de date **faculty** în colecția **students** folosind structura JSON de mai sus doi colegi și pe dumneavoastră.

mongoimport

Pentru a insera un fișier în format CSV sau JSON se folosește comanda **mongoimport**. Parametrii comenzii sunt:

- `--host=<hostname><:port>` - specifică numele host-ului și portul. Dacă nu se specifică se vor lua valorile implicite
- `--port=<port>` - specifică portul. Dacă nu se specifică se va lua valoarea implicită
- `--db=<databasename>` - specifică numele bazei de date unde se vor insera datele
- `--collection=<collection>` - specifică numele colecției care va stoca datele
- `--fields=<field1[,field2]>` - specifică câmpurile din fișier care for fi inserate în colecție
- `--columnsHaveTypes` - se utilizează dacă se dorește specificarea tipurilor de date pentru coloanele din lista de coloane specificată cu opțiunea `--fields`
- `--type=<json|csv|tsv>` - specifică formatul fișierului cu date
- `--file=<filename>` - specifică numele fișierului cu date

```
mongoimport --host=192.100.10.2 --port=28018 --db=BD --collection=students --type=json --file=data_dump.json

mongoimport --host=192.100.10.2 --port=28018 --db=BD --collection=students --columnsHaveTypes --fields="name.string(),birthdate.date(2006-01-02),contacted.boolean(),followerCount.int32()
```

Ex. 5. Să se insereze fișierul bd2_mongo care conține baza de date în format JSON. Noua bază de date se va numi **BD2** și colecția se va numi **documents**.

Când vă folosiți de docker sunteți local (127.0.0.1) și port-ul vi-l arată interfața. (sau comanda pe care ați rulat-o pentru a crea acel container)

Selectarea datelor

Select

În MongoDB operația de selecție a datelor este **find()** (Documentație [https://docs.mongodb.com/manual/reference/method/db.collection.find/]).

Sintaxa comenzii este:

```
db.collection.find(query, projection)
```

Unde

- `query` reprezintă lista de filtre
- `projection` reprezintă câmpurile care vor fi afișate

Dacă parametrul `query` lipsește, atunci se selectează toate documentele. Dacă parametrul `projection` lipsește atunci se afișează toate câmpurile documentului.

Comanda **find()** face paginație, arătând în consolă 10 înregistrări.

Să se selecteze toți studenții din colecția **students**, baza de date **faculty**

```
db.students.find()
```

- Dacă se dorește întoarcerea unei singur document se folosește **findOne()**

```
db.students.findOne()
```

- Dacă se dorește întoarcerea unui număr n de documente se folosește **limit(n)**

```
db.students.find().limit(3)
```

- Dacă se dorește sortarea rezultatelor după valoare unui anumit câmp se folosește `sort({label_1: order, label_2: order,…, label_n:order})` unde `order` este `1` pentru ascendent și `-1` pentru descendent.

```
db.students.find().sort({"grupa": 1})
```

- Dacă se dorește afișarea într-un mod citibil în consolă se folosește **pretty()**

```
db.students.find().pretty()
```

- **pretty**, **limit** și **sort** se pot folosi împreună

```
db.students.find().sort({"student.firstname": 1}).limit(3).pretty()
```

Ex. 6. Să se selecteze primele 4 rezultate ale unei cereri care întoarce toți studenți ordonați descrescător după nume.

Filtrarea cererilor

Pentru a filtra rezultatele se folosește parametru **query**.

Pentru comparație avem:

- `:` - egalitate
- `$ne` - diferit de (not equal)
- `$gt` - mai mare
- `$gte` - mai mare sau egal
- `$lt` - mai mic
- `$lte` - mai mic sau egal
- `$in` - căutare într-o listă
- `$all` - căutare cu egalitate pe toate elementele dintr-o listă

```
db.students.find({"student.firstname": "Ion"})
db.students.find({an: {$gte: 4}})
db.students.find({"materii.nume": {$in: ['SCAD', 'IA']} })
db.students.find({"grupa": /^341/ })
db.students.find({"cunostinte": {$all: [/^J/, /^C/ ]}})
```

Operatori logici

- `$or` - sau logic
- `$and` - și logic
- `$not` - negare logic

```
db.students.find({$or:  [{"student.lastname": "Ionescu"}, {"cunostinte": /^C/}]})
db.students.find({$and: [{"student.lastname": "Ionescu"}, {"cunostinte": /^C/}]})
```

Operatori utili pentru vectori

- `$size` pentru a verifica dimensiune
- `0, 1, 2,…` pentru poziționare (indexare vectorilor începe de la 0)

```
db.students.find({"cunostinte": {$size: 2}})
db.students.find({"cunostinte.2": "SQL"})
db.students.find({"materii.0.nume": "BD2"})
```

Pentru a verifica existența unui câmp se folosește `$exists`

```
db.students.find({"sef": true})
db.students.find({"sef": {$exists: true}})
db.students.find({"sef": false})
db.students.find({"sef": {$exists: false}})
```

Acești operatori pot să fie folosiți împreună.

Ex. 7. Explicați ce fac cererile din acest subcapitol.

Ex. 8. Să se selecteze toți studenții din anul 4 care au restanțe și știu limbajele de programare C și SQL.

Proiecția

Pentru a proiecta doar anumite câmpuri se se folosește parametrul **projection**. În cazul în care nu se folosește filtrare se pun acolade fără goale pentru **query**.

```
db.students.find({}, {"student.firstname": 1})
```

O cerere cu **projection** va întoarce doar câmpurile specificate și `_id`. Dacă se dorește să se elimine câmpul `_id` atunci proiecția trebuie să conțină și `“_id”: 0`

```
db.students.find({}, {"_id": 0, "student.firstname": 1})
```

Ex. 9. Să se selecteze studenții care nu sunt șefi și au cunoștințe de “Python” pe a doua poziție din vectorul de cunoștințe. Afișați doar numele, prenumele și vectorul de cunoștințe.

Modificarea datelor

Pentru a modifica datele se poate folosi una din comenzile:

- `db.collection.update(<query>, <update>, <options>)` (Documentație [https://docs.mongodb.com/manual/reference/method/db.collection.update/])

Unde:

- `query` - filtrează datele
- `update` - setează date care trebuie modificate
- `options` - diferite opțiuni, e.g.:
  - `upsert: <boolean>` - crează un nou document în cazul în care nu se găsește nici un document cu filtrul
  - `multi: <boolean>` - modifică mai multe documente

Pentru a seta o valoare se folosește `$set`:

```
db.students.update({"student.firstname": "George"}, {$set: {"grupa": "342C1"}})
db.students.update({an: 4, "student.firstname": "Grigore"}, {$set: {cunostinte: ["C++"]}})
```

Folosind `$set` se suprascrie valoarea pentru câmp. În cazul în care dorim sa modificăm o structură de tip vector folosiți `$push`. Dacă se dorește să se adauge mai multe valori, se poate folosi `$push` împreună cu `$each`.

```
db.students.update({an: 4, "student.firstname": "Grigore"}, {$push: {"cunostinte": "C"}})
db.students.update({an: 4, "student.firstname": "Grigore"}, {$push: {"cunostinte":{$each: ["Python", "SQL", "Ruby"]}}})
```

Dacă se dorește să se modifice valoare unui element dintr-o listă sau a unui câmp dintr-un document imbricat care se găsește în listă se poate folosi `$`. Trebuie să selectați și valoare/câmpul care se dorește să fie modificat în `query`:

```
db.students.update({"student.firstname": "Grigore", "cunostinte": "C"}, {$set: {"cunostinte.$": "Java"}})
db.students.update({"student.firstname": "Grigore", "materii.nume": "SRIC"}, {$set: {"materii.$.nume": "SPG"}})
db.students.update({"student.firstname": "Grigore", "materii.nume": "SPG"}, {$set: {"materii.$.nume": "SRIC", "materii.$.an":4}})
```

Ex. 10. Să se adauge la toți studenți de anul 3 materiile EGC și LFA (nume si an) folosind o singură comandă.

Alte funcții care modifică datele sunt:

- `db.collection.updateOne(<query>, <update>, <options>)`
- `db.collection.updateMany(<query>, <update>, <options>)`

Ștergerea datelor

Pentru a șterge un document dintr-o colecție se folosește comanda:

- `db.collection.remove(<query>, <justOne>)` Documentație [https://docs.mongodb.com/manual/reference/method/db.collection.remove/]

Unde

- `query` - filtrează datele
- `justOne` - boolean pentru a șterge unul (`true`) sau mai multe (`false` – implicit) documente

```
db.students.remove({"an": {$lt: 4}})
```

Pentru a șterge toate documentele se folosește `db.collection.remove({})`

```
db.students.remove({})
```

Alte comenzi care pot fi utilizare pentru a șterge documente dintr-o colecție sunt:

- `db.collection.deleteOne(<filter>)`
- `db.collection.deleteMany(<filter>)`

Ex. 11. Să se șteargă doar un student care are materia BD2.

bdd/laboratoare/11.txt · Last modified: 2024/10/08 01:45 by alex.petrescu

Laboratorul 11 - MongoDB II

Obiective

- Indexare
- Funcții
- Agregarea datelor

Indexare

MongoDB suport diferite tipuri de indecși. Indecși sunt utilizați pentru a crește performanțele operațiilor de căutare. Indecși sunt creați la nivel de colecție. Pentru a crea un index se folosește comanda `createIndex()`. La creare se specifică numele câmpului (i.e., `label`) și modul de ordonare a indexului (i.e., `order`)

- Indecși simpli `db.collection.createIndex({“label”: order})`
- Indecși compusi `db.collection.createIndex({“label_1”: order_1, “label_2”: order_2, …})`
- Indecși pe documente imbricate: `db.collection.createIndex({“label_1.nested_label”: order_1, “label_2.nested_label”: order_2, …})`
- Indecși geo-spațiali `db.collection.createIndex({“label_1.nested_label”: “2d”})`
- Indecși textuali (o colecție poate să aibă un singur index textual, dar acesta poate fi compus)`db.collection.createIndex({“label_1”: “text”, “label_2”: “text”, …})`
- Indecși hashed (utili pentru sharding): `db.collection.createIndex({“label”: “hashed”})`

Pentru următoarele exemple să se importe documentele din fișierul bd2_mongo care conține baza de date în format JSON. Noua bază de date se va numi **BD2** și colecția se va numi **documents**.

Ex. 1. Analizați schema documentelor și înțelegeți structura

```
db.documents.createIndex({author: 1})
db.documents.createIndex({gender: 1, age: 1})
db.documents.createIndex({"words.word": 1})
db.documents.createIndex({"geoLocation": "2d"})
db.documents.createIndex({"lemmaText": "text"})
db.documents.createIndex({"date": "hashed"})
```

Ex. 2. Descrieți tipul indecșilor construiți anterior.

Pentru a vedea toți indecși se folosește comanda `db.collection.getIndexes()`

Pentru a șterge un index se folosește comanda `db.collection.dropIndexes(“index_name”)`

Pentru a șterge toți indecși (cu excepția indexului de pe cheia primară) se folosește comanda `db.collection.dropIndexes()`

Utilizare indecși geo-spațiali

- Pentru a găsi documentele de lângă un punct `p=[x, y]` se folosește `$near`
- Pentru a găsi documentele care se găsesc în diferite figuri geometrice se folosește `$within`:
  - Pentru un cerc cu centrul într-un punct `p=[x, y]` și o rază `r` se folosește `$center`
  - Pentru un dreptunghi definit prin punctele din stânga jos și dreapta sus se folosește `$box`
  - Pentru un poligon dat de o listă puncte se folosește `$polygon`

```
db.documents.find( { geoLocation: { $near: [25, 25] } }, {geoLocation: 1} )
db.documents.find( { geoLocation: { $within: { $center: [[25, 10], 3] } } }, {geoLocation: 1} )
db.documents.find( { geoLocation: { $within: { $box: [[25, 40], [30, 45]] } } }, { geoLocation: 1 } )
db.documents.find( { geoLocation: { $within: { $polygon: [[20, 40], [40, 50], [30, 30]] } } },  { geoLocation: 1 } )
```

Utilizare indecși textuali

Pentur a utiliza un index textual se folosește următoare sintaxă:

```
{
  $text:
    {
      $search: <string>,
      $language: <string>,
      $caseSensitive: <boolean>,
      $diacriticSensitive: <boolean>
    }
}
```

Unde

- `$search` - este șirul de cuvinte căutat sau fraza căutată
- `$language` - este limba pentru cuvintele căutate
- `$caseSensitive` - dacă se caută case sensitive sau nu
- `$diacriticSensitive` - dacă se ține cont de diacritice sau nu

```
# Caută un cuvânt
db.documents.find( { $text: { $search: "coffee", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } ,  { lemmaText: 1 } )

# Caută mai multe cuvinte (se folosește operatorul or)
db.documents.find( { $text: { $search: "coffee cup", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } ,  { lemmaText: 1 } )

# Caută o frază
```

```
db.documents.find( { $text: { $search: "\"heaven tonight\"", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } ,  { lemmaText: 1 } )

# Caută o frază cu un scor de relevanță
db.documents.find( { $text: { $search: "\"heaven tonight\"", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } ,  { score: { $meta: "textScore" }, lemmaText: 1
```

Ex. 3. Să se găsească toate documentele care se află într-un dreptunghi dat de punctele `[20, -100]` și `[40, 90]`, care să conțină cuvintele **tech** și **engineering**. Cererea să se folosească de indexul textual. Afișeți câmpul **rawText**. Să se refacă cererea utilizând alt câmp din document.

Funcții

MongoDB permite dezvoltatorilor sa scrie funcții folosind limbajul JavaScript.

Ex. 4. Să se scrie o funcție care primește un **query** pentru filtrare și împarte **lemmaText** în cuvinte. Funcția va întoarce un vector.

```
tokenization = function(q){
    var cursor = db.documents.find(q,{"_id": 0, lemmaText: 1});
    var tokens = Array();
    cursor.forEach(function(elem){
        tokens = tokens.concat(elem["lemmaText"].split(" "));
    });
    return tokens;
}

// utilizate
var q = {gender: "male"}
tokens = tokenization(q)
```

Ex. 5. Să se scrie o funcție numită **countWords** care primește un **query** pentru filtrare și numără aparițiile unui cuvânt. Funcția va întoarce un obiect de forma `{word_1: count, word_2: count, …}`. Folosiți funcția `tokenization`.

Funcții stocate

Funcțiile pot fi stocate la nivelul MongoDB. Pentru acest lucru se folosește o colecție specială numită `system.js`. Stocarea se face cu comanda: `db.system.js.save(_id: “function_name”, value: “function_body”)`. Pentru a lucra cu funcțiile stocare (a le încărca în sesiune) se folosește comanda `db.loadServerScripts()`.

```
db.system.js.save(
   {
     _id: "tokenization",
     value : function(q){
                var cursor = db.documents.find(q,{"_id": 0, lemmaText: 1});
                var tokens = Array();
                cursor.forEach(function(elem){
                    tokens = tokens.concat(elem["lemmaText"].split(" "));
                });
                return tokens;
            }
   }
)

db.loadServerScripts()
```

Ex. 6. Salvați și apoi încărcați în sesiune funcția **countWords**.

Agregarea datelor

În MongoDB putem să agregăm datele folosind mai multe metode:

- Funcții de agregare
- Modulul nativ de Aggregation Pipeline
- MapReduce

Funcții de agregare

MongoDB oferă utilizatorilor două funcții pentru agregare:

1. Count: `db.collection.count(query)`

```
// Cate documente au lungimea textului lematizat 10
db.documents.count({lemmaTextLength: 10})
```

2. Distinct: db.collection.distinct(field, query)

```
// afiseaz varstele distincte
db.documents.distinct("age")

// afiseaz varstele distincte pentru female
db.documents.distinct("age", {gender: "female"}).sort()
```

Ex. 7. Afișați toate cuvintele distincte. Folosiți câmpul **words**.

Aggregation Pipeline

Pentru agregare, MongoDB oferă și **Aggregation Pipeline** (Documentație [https://docs.mongodb.com/manual/core/aggregation-pipeline/]).

```
db.collection.aggregate(
    { $match: <query> },
    { $unwind: <array> },
    { $project: <projection>},
```

```
    { $group: <aggregation_group, aggregation_functions> },
    { $sort: <sorting_fields> },
)
```

Unde:

- `$match` - filtrează documentele de intrare după un **query**
- `$unwind` - procesează elementele unui vector
- `$project` - proiecția
- `$group` - face grupuri și a aplica funcții agregare
- `$sort` - reordonează documentele de intrare după o cheie

Contează ordinea în care sunt folosiți operatorii.

Ex. 8. Să se utilizeze Aggregation Pipeline pentru a calcula numărul de apariții ale cuvintelor.

```
db.documents.aggregate([
    { $match: q },
    { $project: { words: { $split: ["$lemmaText", " "]}}},
    { $unwind: "$words" },
    { $group: { _id: "$words", counts: { $sum: 1 } } }
])
```

Ex. 9. Să se utilizeze Aggregation Pipeline pentru a calcula numărul de apariții ale cuvintelor folosind coloana **words**.

MapReduce

MongoDB oferă un framework de MapReduce. Pentru a utiliza acest framework trebuie să scriem două funcții,i.e., Map și Reduce. Pentru a utilza funcțiile pe o colecție se folosește `mapReduce` (Documentație [https://docs.mongodb.com/manual/core/map-reduce/]):

```
db.collection.mapReduce(
    mapFunction,
    reduceFunction,
    {
        finalize: <function>,
        out: <output>,
        query: <document>,
        sort: <document>,
        limit: <number>,
        scope: <document>
    }
)
```

Unde:

- mapFunction - funcția de map
- reduceFunction - funcția de reduce
- finalize - (opțional) funcție pentru finalizare, se aplică după ce se termină procesul și datele sunt agregate
- out - (opțional) colecția unde va fi salvat rezultatul
- query - (opțional) filtru pentru documentele de intrare
- sort - (opțional) ordonează documentele de intrare după cheie
- limit - (opțional) limitează numărul de documente de intrare
- scope - (opțional) variabile din exterior care pot fi folosite în funcțiile map, reduce și finalize

Comanda `mapReduce` este o mapare peste comanda **mapReduce** din MongoDB (Documentație [https://docs.mongodb.com/manual/reference/command/mapReduce/])

```
db.runCommand(
    {
        mapReduce: <collection>,
        map: <function>,
        reduce: <function>,
        finalize: <function>,
        out: <output>,
        query: <document>,
        sort: <document>,
        limit: <number>,
        scope: <document>
    }
)
```

Ex. 10. Să se utilizeze MapReduce pentru a calcula numărul de apariții ale cuvintelor.

```
mapFunction = function() {
    var tokens = this.lemmaText.split(" ");
    for (var idx=0; idx<tokens.length; idx++){
        emit(tokens[idx], 1);
    }
}

reduceFunction = function(key, values) {
    return Array.sum(values);
};

var q = {"gender": "female"}
db.documents.mapReduce(mapFunction, reduceFunction, {query: q, out: "wordCounts"});

db.wordCounts.count()

db.wordCounts.find({"_id": "0"})

db.wordCounts.drop()
```

Ex. 11. Să se utilizeze MapReduce pentru a calcula numărul de apariții ale cuvintelor folosind câmpul **words**.

bdd/laboratoare/12.txt · Last modified: 2024/10/08 01:46 by alex.petrescu

Laboratorul 12 - Cassandra

Obiective

- Comenzi Cassandra
- Operatii DML
- Tipuri colectie
- Tipuri definite de utilizator

Instalare

Pentru a utiliza Cassandra folosind Docker:

1. Descarcati imaginea Docker:

```
docker pull cassandra
```

2. Porniti un container bazat pe imaginea descarcata:

```
docker run --name cassandradb -d cassandra:latest
```

Comenzi Cassandra

Conexiune la Cassandra

Pentru a ne conecta la un server Cassandra putem folosi utilitarul **cqlsh** (Documentație [https://cassandra.apache.org/doc/latest/tools/cqlsh.html]).

Printre parametrii pe care cqlsh ii permite avem:

- `--user` - specifica utilizatorul cu care sa se conecteze la Cassandra
- `--password` - specifica parola utilizatorului
- `--keyspace` - precizeaza keyspace-ul la care sa se conecteze
- `--file` - specifica un fisier din care sa se execute comenzi, urmate de exit

Deschideti o conexiune la Cassandra ruland comanda:

```
docker exec -it cassandradb cqlsh
```

Comenzi CLI Cassandra

Cateva dintre comenzile ce tin de administrarea **keyspace-urilor** sunt:

```
-- listare keyspace-uri disponibile
DESCRIBE KEYSPACES;

-- afisare keyspace curent
DESCRIBE KEYSPACE;

-- conectare la un keyspace existent
USE <keyspace_name>;

-- creare keyspace nou
CREATE KEYSPACE <keyspace_name> WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 3 };
```

```
-- creare keyspace nou fara a scrie operatiile si in commit log
CREATE KEYSPACE <keyspace_name> WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 3 } AND durable_writes = TRUE;

-- stergere keyspace
DROP KEYSPACE <keyspace_name>;
```

Ex. 1. Creati un keyspace cu numele **students_keyspace** cu durable_writes setat pe true si conectati-va la el.

Pentru a crea un tabel in keyspace-ul curent:

```
CREATE TABLE <table_name> (
   <column_name1> datatype PRIMARY KEY,
   <column_name2> datatype,
   <column_name3> datatype,
   ...
)
```

Pentru a sterge un tabel in keyspace-ul curent:

```
DROP TABLE <table_name>
```

Ex. 2. Creati o tabela cu numele **students** care are coloanele: *id* de tip INT ca PRIMARY KEY, *firstname* de tip VARCHAR, *lastname* de tip VARCHAR, *year* de tip INT, *group* de tip VARCHAR.

Operatii DML

Inserarea datelor

Pentru a insera date in Cassandra se poate folosi comanda **INSERT**, asemanator limbajului SQL:

```
INSERT INTO <table_name> (<column_name1>, <column_name2>....)
VALUES (<value1>, <value2>....);
```

Inserati in tabela students urmatorii studenti:

```
INSERT INTO students (id, firstname, lastname, YEAR, GROUP) VALUES (1, 'Ion', 'Popescu', 4, '342C3');
INSERT INTO students (id, firstname, lastname, YEAR, GROUP) VALUES (2, 'Andrei', 'Ionescu', 4, '341C4');
INSERT INTO students (id, firstname, lastname, YEAR, GROUP) VALUES (3, 'George', 'Popescu', 4, '341C2');
INSERT INTO students (id, firstname, lastname, YEAR, GROUP) VALUES (4, 'Georgiana', 'Petrescu', 4, '341C2');
INSERT INTO students (id, firstname, lastname, YEAR, GROUP) VALUES (5, 'Valentina', 'Vasilescu', 3, '331CA');
INSERT INTO students (id, firstname, lastname, YEAR, GROUP) VALUES (6, 'Grigore', 'Ionescu', 4, '342C2');
INSERT INTO students (id, firstname, lastname, YEAR, GROUP) VALUES (7, 'Andrei', 'Popescu', 3, '332CA');
INSERT INTO students (id, firstname, lastname, YEAR, GROUP) VALUES (8, 'Ana', 'Georgescu', 4, '342C5');
```

Ex. 3. Inserati in tabela students 2 colegi.

Selectarea datelor

Operatia de selectie a datelor in Cassandra se poate realiza prin comanda **SELECT**:

```
SELECT * | select_expression
FROM [keyspace_name.] <table_name>
[WHERE filters]
[ORDER BY column_name1, column_name2... ASC|DESC]
[LIMIT N]
[ALLOW FILTERING];
```

Pentru a putea impune conditii de filtrare pe coloane care nu sunt indexate trebuie precizat explicit ALLOW FILTERING.

Limitari ale comenzii SELECT:

- Nu se pot folosi subcereri.
- Unii operatori nu sunt acceptati. Ex.: BETWEEN
- Unii operatori functioneaza doar pe chei primare. Ex.: IN

- Nu exista operatia JOIN.

Ex. 4. Selectati toti studentii de anul 4 care au prenumele 'Andrei'.

Modificarea datelor

Pentru a modifica datele in Cassandra se foloseste comanda **UPDATE**:

```
UPDATE [keyspace_name.] <table_name>
SET assignment [, assignment] . . .
WHERE row_specification
```

Clauza WHERE poate contine numai conditii ce tin de chei primare.

Ex. 5. Promovati toti studentii din tabela students de anul 3 in anul 4, prin actualizarea coloanei *year* si *group*.

Stergerea datelor

Pentru a sterge datele in Cassandra se poate folosi comanda **DELETE**:

```
DELETE [column_name (term)][, ...]
FROM [keyspace_name.] <table_name>
WHERE PK_column_conditions;
```

Clauza WHERE poate contine numai conditii ce tin de chei primare.

Alternativ, se poate folosi comanda **TRUNCATE** pentru a sterge toate valorile dintr-un tabel:

```
TRUNCATE [TABLE] [keyspace_name.] <table_name>
```

Ex. 6. Stergeti din tabela students studentii cu id-urile 1, 3 si 5.

Tipuri colectie

Tipurile colectie sunt utile in cazurile in care avem nevoie sa stocam un numar variabil de elemente intr-o singura coloana.

Cassandra pune la dispozitie urmatoarele tipuri colectie:

- Set
- List
- Map

Tipul Set

Tipul de date set stocheaza o multime de elemente neordonate, care pot fi de tipuri simple, tipuri definite de utilizator sau alte colectii.

Pentru a adauga o coloana de tip set intr-o tabela putem folosi:

```
ALTER TABLE <table_name> ADD <column_name> SET<data_type>;
```

La interogari, **cqlsh** intoarce elementele din set sortate.

Un avantaj al utilizarii setului este posibilitatea de a adauga elemente fara a fi nevoiti sa citim continutul setului intai.

Pentru a adauga un nou element in set se foloseste **+**:

```
my_set = my_set + { new_element }
```

Pentru a sterge un element din set se foloseste **-**:

```
my_set = my_set - { old_element }
```

Pentru a seta un set gol se foloseste **{}**:

```
my_set = {}
```

Adaugati la tabela students coloana *hobbies* de tipul SET<TEXT>:

```
ALTER TABLE students ADD hobbies SET<TEXT>;
```

Apoi populati coloana hobbies pentru studentii predefiniti:

```
UPDATE students SET hobbies = {'Tennis', 'Chess'} WHERE id = 2;
UPDATE students SET hobbies = {'Programming', 'Design'} WHERE id = 4;
UPDATE students SET hobbies = {'Table Tennis', 'Backgammon'} WHERE id = 6;
UPDATE students SET hobbies = {'Football'} WHERE id = 7;
UPDATE students SET hobbies = {'Swimming'} WHERE id = 8;
```

Ex. 7. Adaugati hobby-uri pentru studentii inserati in tabela students la exercitiul 3.

Tipul List

Tipul de date list contine o lista de elemente ordonate. In mod standard, ele sunt ordonate dupa ordinea de inserare.

Pentru a adauga o coloana de tipul list putem folosi:

```
ALTER TABLE <table_name> ADD <column_name> LIST<data_type>;
```

Pentru a adauga un element in lista se poate folosi **+**:

```
my_list = my_list + [ new_element ]
```

Pentru a sterge un element din lista se poate folosi **-**:

```
my_list = my_list - [ old_element ]
```

Pentru a seta o lista goala se poate folosi **[]**:

```
my_list = []
```

Se pot folosi indecsi pentru a accesa elemente specifice in lista folosind **[index_number]**, indexarea pornind de la 0.

Adaugati o coloana de tip lista in tabela students, cu numele *languages* si tipul LIST<TEXT>:

```
ALTER TABLE students ADD languages LIST<TEXT>;
```

Apoi populati coloana languages pentru studentii predefiniti:

```
UPDATE students SET languages = ['C', 'C++'] WHERE id = 2;
UPDATE students SET languages = ['Java', 'Scala'] WHERE id = 4;
UPDATE students SET languages = ['Python', 'Go'] WHERE id = 6;
UPDATE students SET languages = ['Ruby'] WHERE id = 7;
UPDATE students SET languages = ['PHP', 'Javascript'] WHERE id = 8;
```

Ex. 8. Adaugati limbajul C# pentru studentul cu id-ul 2 la lista de limbaje de programare cunoscute si goliti lista de limbaje de programare cunoscute pentru studentul cu id-ul 8.

Tipul Map

Tipul de date map contine o multime de perechi cheie-valoare. Cheile si valorile pot avea orice tip cu exceptia counter.

Pentru a adauga o coloana de tip map intr-o tabela putem folosi:

```
ALTER TABLE <table_name> ADD <column_name> MAP<data_type_key, data_type_value>;
```

Cheile map-ului pot fi accesate folosind **[key_name]**:

```
my_map[my_key] = my_value
```

Pentru a verifica daca un map contine o valoare putem folosi **CONTAINS**:

```
SELECT * FROM my_table WHERE map_column CONTAINS map_value ALLOW FILTERING;
```

Pentru a verifica daca un map contine o cheie putem folosi **CONTAINS KEY**:

```
SELECT * FROM my_table WHERE map_column CONTAINS KEY map_key ALLOW FILTERING;
```

Adaugati la tabela students coloana *grades* de tipul MAP<TEXT, DOUBLE>:

```
ALTER TABLE students ADD grades MAP<TEXT, DOUBLE>;
```

Apoi adaugati note la studentii predefiniti:

```
UPDATE students SET grades = {'Baze de date 2': 10, 'Utilizarea sistemelor de operare': 8} WHERE id = 2;
UPDATE students SET grades = {'Programare web': 9, 'Calculatoare numerice': 8} WHERE id = 4;
UPDATE students SET grades = {'Baze de date 2': 9} WHERE id = 6;
UPDATE students SET grades = {'Matematica 1': 8, 'Matematica 2': 9} WHERE id = 7;
UPDATE students SET grades = {'Metode numerice': 10, 'Baze de date 2': 7} WHERE id = 8;
```

Ex. 9. Selectati studentii care au note la materia 'Baze de date 2'.

Tipuri definite de utilizator

Cassandra suporta si tipuri definite de utilizator (User defined types - **UDT**). Acestea pot fi folosite atat de sine statator, cat si in colectii.

La utilizarea unui UDT intr-o colectie, acesta trebuie declarat ca **frozen**, ceea ce inseamna ca modificarea valorii nu se poate face accesand individual campurile UDT-ului, ci doar modificata ca intreg obiect.

Pentru a defini un UDT se poate folosi:

```
CREATE TYPE [IF NOT EXISTS]
[keyspace_name.] <type_name>(
<field_name> data_type[,]
[<field_name> data_type] [,...]
)
```

Pentru a adauga o coloana de tip UDT se poate folosi:

```
-- coloana UDT non-colectie
ALTER TABLE <table_name> ADD <column_name> udt_data_type;

-- coloana UDT colectie
ALTER TABLE <table_name> ADD <column_name> collection<frozen<udt_data_type>>;
```

Pentru a seta o valoare de tip UDT se poate folosi:

```
my_udt_column = {field_1: value_1, field_2: value_2, ...}
```

Definiti tipul nou *Course* care are campurile *name* de tip VARCHAR si *year* de tip INT:

```
CREATE TYPE course(
  name VARCHAR,
  YEAR INT
);
```

Adaugati in tabela students coloana *classes* cu tipul LIST<FROZEN<COURSE»:

```
ALTER TABLE students ADD classes LIST<FROZEN<COURSE>>;
```

Apoi adaugati cursuri studentilor predefiniti:

```
UPDATE students SET classes = [{name: 'Baze de date 2', YEAR: 4}, {name: 'Utilizarea sistemelor de operare', YEAR: 1}] WHERE id = 2;
UPDATE students SET classes = [{name: 'Programare web', YEAR: 4}, {name: 'Calculatoare numerice', YEAR: 2}] WHERE id = 4;
UPDATE students SET classes = [{name: 'Baze de date 2', YEAR: 4}] WHERE id = 6;
UPDATE students SET classes = [{name: 'Matematica 1', YEAR: 1}, {name: 'Matematica 2', YEAR: 1}] WHERE id = 7;
UPDATE students SET classes = [{name: 'Metode numerice', YEAR: 1}, {name: 'Baze de date 2', YEAR: 4}] WHERE id = 8;
```

Ex. 10. Adaugati cate 2 cursuri studentilor inserati in tabela students la exercitiul 3.

bdd/laboratoare/13.txt · Last modified: 2024/10/07 11:27 (external edit)